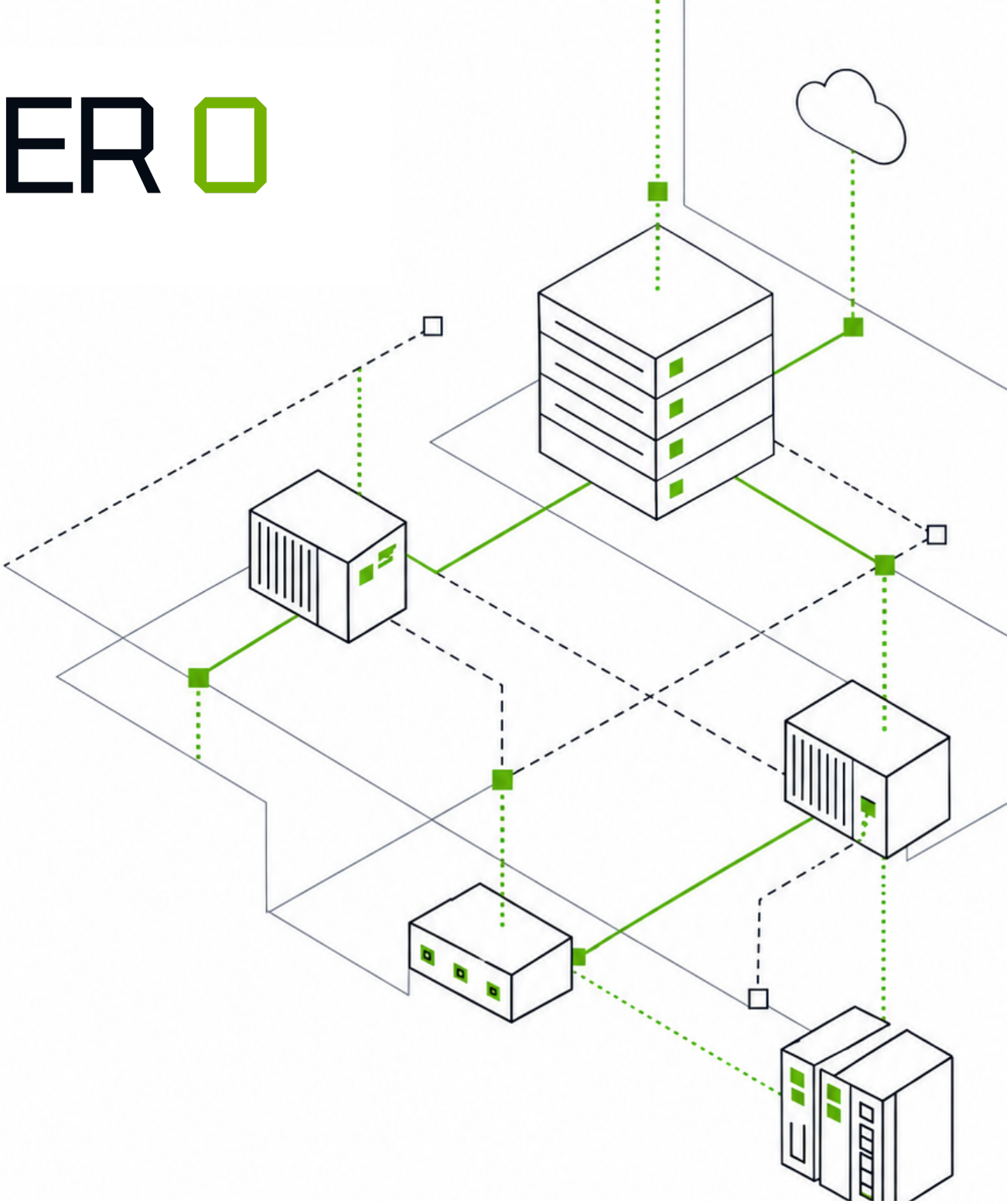
TIER 0



Tier0 技术指导手册

目录

快速开始

1. 选择最合适的版本
2. 安装
3. 在 Tier0 中体验 Demo 工厂

使用 TIER0

1. 工厂数据基础
2. 准备数据基础
3. 从中心到边缘管理数据
4. 操作工厂数据
5. 构建应用
6. 构建数字孪生
7. 分析数据
8. 用 Agents 操作 Tier0

最佳实践

1. 工厂数据建模
2. 连接工业协议
3. 安全与可靠性
4. 构建分析应用
5. 用 Agent 构建车间工作流
6. 使用数字孪生展示数据

参考资料

1. API 参考
2. CLI 命令参考
3. 内置 Builder Skills
4. 用户管理
5. Tier0 云服务版网络与数据安全说明
6. ISA-95 设备层级
7. 标准部署方案
8. tier0-enterprise 三节点部署方案 V2
9. SLA 与高可用边界
10. 标准端口清单
11. 运维 Runbook

附录

1. 完整 OpenAPI 参考

快速开始

选择最合适的版本

Edge、Tier0 云服务版或 Enterprise：通过定位、套餐和能力矩阵选择适合你的 Tier0 版本。

查看 这里 了解每个产品的详细功能，并选择最适合你的版本。

下一步

- 在工厂数据上构建应用 - 构建并在 Enterprise 上部署 Tier0 应用
- 安装

快速开始

安装

按版本安装 Tier0。

Tier0 提供三个版本：

- **Edge** - 开源版，可通过 Docker 在本地运行。
- **Tier0 云服务版** - 云端托管 SaaS。
- **Enterprise** - 面向企业环境的私有化部署版本。

请参阅 选择最合适的版本 来判断哪个版本最符合用户需求。

安装 Tier0

标签页：TIER0 云服务版

在体验 Tier0 云服务版之前，需要申请试用账号。

1. 在 注册 Tier0 页面注册账号。
2. 用发到邮箱的验证码进行验证后，使用注册账号登录。
3. 创建一个工作空间并开始体验 Tier0.

标签页：ENTERPRISE

环境要求

- 单机部署
  - 最低：4C / 8G / 100G
  - 推荐：8C / 16G / 500G
- 标准部署
  - 最少：4 台虚拟机
  - 各 VM 类型的基线配置：
    - App：8C / 32G / 100G
    - Database：4C / 8G / 500G

        - Witness：2C / 4G / 40G
- 高可用部署
    - 最少：6 台虚拟机
    - 各 VM 类型的基线配置：
        - App (Swarm)：8C / 32G / 100G x 3
        - Database：4C / 8G / 500G x 2
        - Witness：2C / 4G / 40G

> 硬件要求可能会变化
> Enterprise 的硬件要求会随部署场景变化。

> 注意
> 详细部署信息见标准部署方案和高可用部署方案。

操作步骤

1. 从供应商处获取安装包，并完成校验。

```
sha256sum -c tier0-enterprise-offline-standard-
YYYYmmddTHHMMSS.tar.gz.sha256
```

2. 解压安装包。

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. 根据部署信息编辑 `install.env`，然后开始部署。

```
vi install.env
bash install.sh
```

常用维护操作

- 在不安装 Enterprise 的情况下检查部署环境。

```
./install.sh --check-only
```

- 部署失败后清理所有节点。

```
./full-cleanup.sh --env install.env --yes
```

> ⚠ **影响说明**
> 此命令会清除全部 Tier0 运行时数据和数据库备份，但会保留虚拟机、操作系统以及已安装的 Docker 软件。

- 安装完成后会生成部署报告。

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> ⚠ **报告中出现 FAIL 时应该怎么做**
> 如果报告中包含任何 **FAIL** 项，请不要交付项目，请先联系支持团队。

维护参考

- 标准端口列表
- 维护与故障排查指南
- 高可用边界说明

标签页：EDGE

环境要求

- 至少 4 核 CPU、8 GB RAM、100 GB 磁盘
- Ubuntu 24.04，或安装了 Docker 和 Git 的 Windows 10/11

操作步骤

1 从 GitHub 克隆 Tier0-Edge 仓库到你的服务器，并进入项目目录。

```
git clone https://github.com/FREEZONEX/Tier0-Edge
cd Tier0-Edge/deploy
```

2 根据你的环境，在 `.env.default` 中按需编辑下表列出的参数。

```
cp .env.default .env
vi .env
:wq!
```

ⓘ 需要配置的参数

字段	类型	说明	示例
VOLUMES_PATH	string	项目数据的存储路径	Linux 上为 `/srv/tier0/data`；Windows Bash 中为 `D:/tier0/data` 或 `/d/tier0/data`
ENTRANCE_DOMAIN/ENTRANCE_PORT	string	Tier0 访问入口（IP/域名 + 端口）	`192.168.1.10` / `8088`
LANGUAGE	enum	zh-CN / en-US	`zh-CN`
ADMIN_INIT_PASSWORD	string	管理员用户的初始密码	一个私有强密码
COMPOSE_PROJECT_NAME/	string	隔离同一主机上运行的多个	`edge_lab` / `100`

字段	类型	说明	示例
PORT_OFFSET		实例	

3 运行安装脚本，并检查安装状态。

```
bash bin/install.sh
bash bin/compose.sh ps # 检查容器状态
bash bin/compose.sh logs -f backend # 查看 backend 容器日志
curl -fsS http://127.0.0.1:8088/readyz # 检查 backend 服务是否 ready
```

> ⚠ 警告
> 不要编辑 `deploy/.env.runtime`；它包含生成的密钥和解析后的运行时配置值。

4 使用配置好的访问入口和默认管理员账号访问 Tier0：

- 用户名：`tier0`
- 密码：`.env` 中 `ADMIN_INIT_PASSWORD` 的值；如果未设置，则为 `tier0`。

下一步

- 体验 Demo Factory - 通过体验一个 demo 工厂快速了解 Tier0。
- 选择最合适的版本 - 比较各个版本并选择最适合你的版本。

快速开始

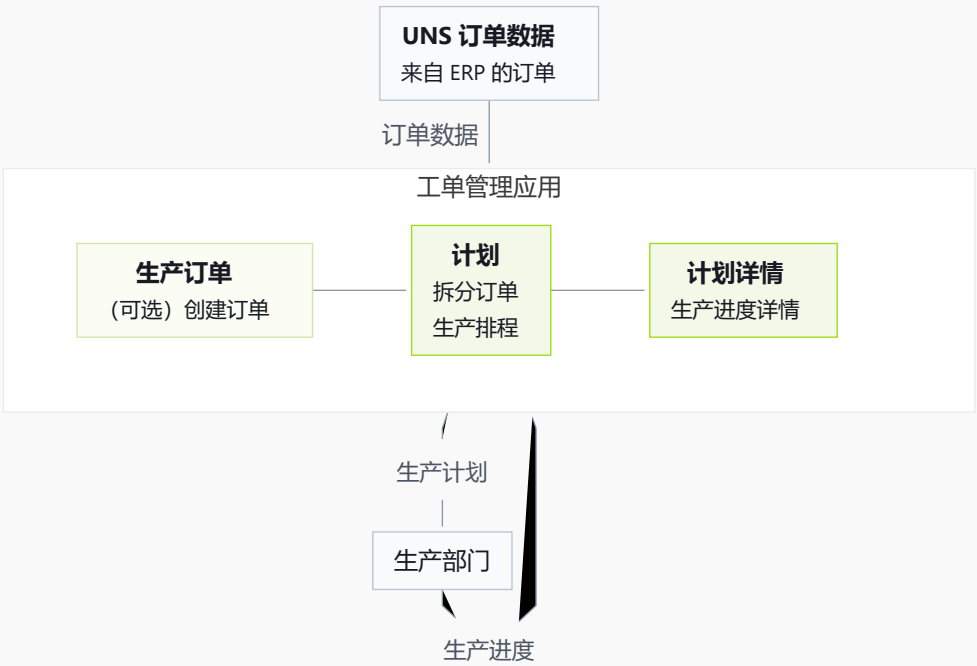
在 Tier0 中体验 Demo 工厂

在 Tier0 云服务版试用工作空间中探索一个预置的可运行工厂，包括 namespace、flows、notebooks 和应用。

理解 Tier0 最快的方式，是直接查看一个已经可以运行的工厂。

业务背景

Demo 工厂会从 ERP 接收订单，拆分订单，安排生产计划，将计划发送给生产部门，并在生产过程中接收生产进度数据。



流程

1. 在 Tier0 中进入 **UNS**，查看来自 ERP 的订单详情。
   - `DemoFactory/ERP/ProductionOrders/State/UpsertProductionOrder`：订单。
   - `DemoFactory/ERP/ProductionOrders/State/OrderList`：当前订单列表快照。

> 🚀 数据如何采集到 UNS?
> 进入 **数据流程** > **数据连接** > **DemoFactory-Flow** 查看数据采集详情。

2. （可选）进入 **应用中心**，打开 **Work Order Management** 应用，并在 **Production Orders** 页面创建订单。
3. 在 **Work Order Management** 中，在 **Plans** 页面拆分订单，并为拆分后的工单安排生产计划。
4. 将计划发送给生产部门，并在 **UNS** 的以下 topics 中查看计划详情。
   - `DemoFactory/ERP/WorkOrderPlan/Metric/SplitCount`：拆分后的工单数量。
   - `DemoFactory/ERP/WorkOrderPlan/State/PlanStatus`：当前计划状态。
   - `DemoFactory/ERP/WorkOrderPlan/State/WorkOrderList`：生产计划排程后的工单列表。

> ⓘ 注意
> 应用会直接将排产计划和工单数据写回 **UNS**。

5. 在应用的 **Plan Details** 页面查看生产进度数据。

> 🚀 详情数据来自哪里?
> 生产进度数据来自生产部门，由 **数据连接** 中的 **DemoFactory-Flow** 采集并发布到 **UNS**，然后展示在 **Plan Details** 页面。
>
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/CurrentWorkOrder`：正在执行的订单。
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/WorkOrderStatus`：当前订单的执行状态。
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Target_Qty`：当前订单的目标产量。

- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Produced_Qty`：当前订单已完成的产品数量。
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Defect_Qty`：当前订单的不良品数量。
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Completion_Rate`：当前订单的完成率。

下一步

- 选择最合适的版本 - 比较各版本并选择最适合你的版本
- UNS 概念 - 了解 Unified Namespace 数据建模

使用 Tier0

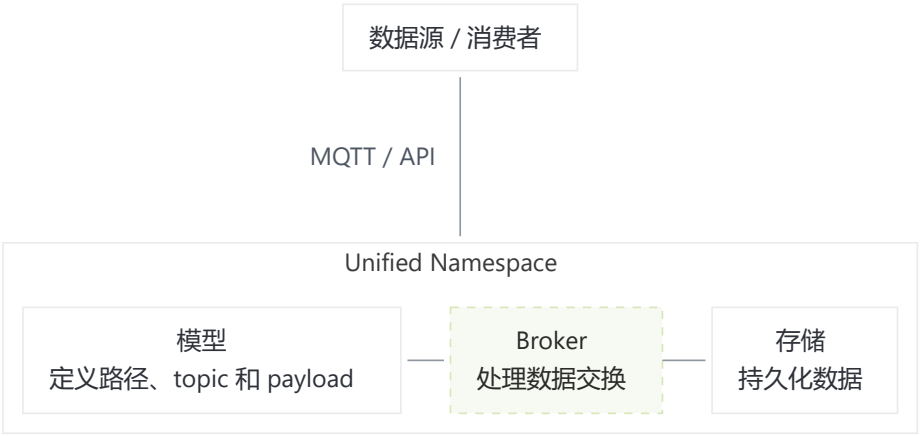
工厂数据基础

Unified Namespace 模型：Metric、State、Action，以及 UNS 如何存储工厂数据。

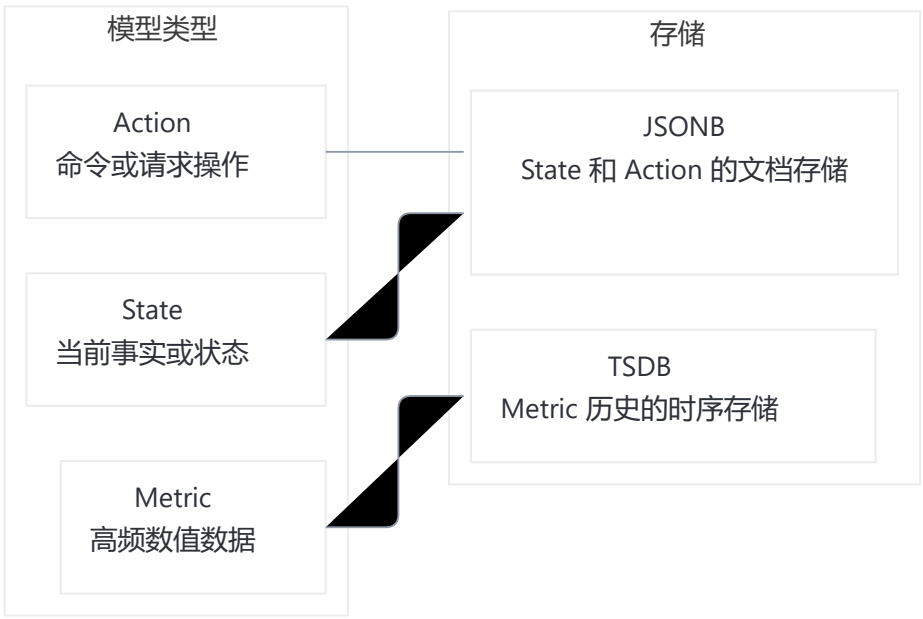
在 Tier0 中，工厂数据会基于 UNS（Unified Namespace）方法组织成清晰的树形模型，形成统一的数据基础。

数据基础架构

- Broker 通过模型 topic 接收和发送数据。



- 模型分为 **Metric**、**State** 和 **Action** 三种类型。
- 数据会根据模型类型存储到不同数据库中。
  - **Action/State**: JSONB (PostgreSQL)
  - **Metric**: 实时数据（TSDB）



Broker

在 Tier0 中，broker 是平台组件，它会基于数据模型，通过 MQTT 或 OpenAPI 接收和发送数据。

- **通过 MQTT**：模型树中每个节点的完整路径会作为 topic 地址，用于传递消息内容。
- **通过 OpenAPI**：通过 HTTP 请求对数据模型执行 CRUD 操作。

> **API 参考**
> API 详情见 **API 参考**。

模型

> **建模时应该参考哪种方式?**
> 根据数据代表的对象选择建模方式。

- **物理数据**：来自物理设备的实时数据。它通常基于 **ISA-95** 标准组织，并强依赖位置层级。
- **业务数据**：与生产需求、计划、任务和调度相关的数据。它通常按 **业务域和对象** 组织。

Metric、State、Action

工业数据变化频率不同，目的也不同，因此 Tier0 将 UNS 模型固定分为三类。

类型	对象	示例
`METRIC`	面向连续采集、需要时序存储的高频数据。	设备温度/压力/电压
`STATE`	面向变化频率较低、用于描述对象当前状态或事实的数据。	设备运行状态
`ACTION`	面向低频命令数据，用于记录外部系统或操作员应执行的动作。	启动批次、停止设备

Path/Topic、消息内容

每个 UNS 模型都包含 path、topic 和消息内容。它们共同形成一棵**树**，既承载业务上下文，也传输工业数据。

- **Path/Topic**：path 表示数据归属，topic 承载消息内容。二者共同定义 *模型地址*。
- **消息内容**：topic 中保存的消息内容。它是 **键值对** 格式的 JSON，用来定义 *模型数据*。

例如，UNS 中的模型可以像下面这样表示：

标签页：物理数据

```
Factory_A
└── Site_01
    └── SMT_Line_1
```

```
        └── Metric
            └── Machine_001
```

- **Path**：`Factory_A/Site_01/SMT_Line_1/Metric`
- **Topic**：`/Machine_001`。
- 消息内容

```
"temperature": 85,
"vibration": 2.8,
```

标签页：业务数据

```
Factory_A
└── ERP
    └── ProductionOrders
        └── State
            └── OrderList
```

- **Path**：`Factory_A/ERP/ProductionOrders/State`
- **Topic**：`/OrderList`。
- 消息内容

```
"orders": "0123569876",
"updated_at": 2026-7-13,
```

> 构建模型
> 参见 如何构建数据模型。

Storage

Tier0 使用 PostgreSQL 存储数据。不同数据类型变化频率不同，也有不同的存储要求。

数据类型	存储	说明
Metric	TSDB	针对温度、压力等高频测量数据优化。每个 topic 默认包含 `timestamp` 和 `quality` 字段。
State	PostgreSQL	以灵活的 `JSONB` 格式存储设备或系统状态。
Action	PostgreSQL	以灵活的 `JSONB` 格式存储命令或动作。

下一步

- 连接数据到 UNS - 如何将工厂建模到 UNS，并把数据连接到模型。
- 使用 UNS 数据 - 如何通过 MQTT 和 API 操作你的 UNS 模型。

使用 TIER0

准备数据基础

构建 namespace 模型，并通过数据连接接入数据。所有版本均可使用。

先在 **UNS** 中构建模型，然后通过 **数据连接** 将数据接入模型，形成后续使用的数据基础。



构建数据模型

根据以下内容定义数据层级树：

- 数据来源的物理位置（参考 ISA-95）；
- 数据遵循的业务逻辑。

手动构建模型

> 先使用草稿验证模型
> 在 **Draft** 中创建或导入模型，先验证模型是否符合建模原则，确认有效后再部署。

1. 设计模型结构，先按顺序创建路径，再创建 topic。
2. 根据数据特征，在 **Metric**、**State** 和 **Action** 中选择 **Topic Type**。
3. 选择 **Mock Data** 向模型发送模拟数据，并启用 **Enable History** 将数据存入数据库。

> ⓘ 注意
> 系统会在 **数据流程** 中创建一个 flow，用来向模型发送模拟数据。

4. 通过 **Auto Parsing** 或 **Pre-defined** 为 topic 添加正确数据类型的字段。

   - **Auto Parsing**：从 JSON 文本中批量自动转换字段。

     ```
     {
       "temp":85.5
     }
     ```

     > ⚠ 警告
     > 只能解析单层 JSON；对于多层 JSON，只会解析第一层。

   - **Pre-defined**：逐个手动设置 topic 字段。

导入模型

> 提示
> 可以使用 ChatGPT 等 LLM 辅助导入模型。

1. 将 **Import** 中的模板 JSON 发给 AI，并使用类似的 prompt。

   ```
   根据模板，生成一个用于 xx 工厂 xx 场景的 UNS 模型，包括 xx 设备和数据源。
   ```

2. 将生成结果导入 UNS。

连接数据到 UNS

数据流程特性

- 基于 Node-RED。
- 每个 flow 都以 `mqtt out` 节点结束。它作为 MQTT client 将数据发布到 UNS broker。
- UNS broker 内置在与 flow 同名的 `mqtt out` 和 `mqtt in` 节点中。

- 当使用 UNS 模型作为 topic 时，数据会直接进入 **UNS** 中对应的模型。

步骤

1. 根据数据源类型使用节点构建 flow，并以 `mqtt out` 节点结束。

   > 🚀 如果需要当前没有的节点
   > 可以从 Node-RED 社区安装所需节点。官方社区节点通常以 `node-red-contrib` 开头。

2. 确保 `mqtt out` 节点的 **Server** 设置为 UNS broker，并且 topic 是 **UNS** 中的模型。

Flow 版本管理

保存数据流程版本，这样你可以回滚到早期版本，也可以基于历史版本启动新的 flow。

> ⚠ 警告
> Edge 版本不支持版本回滚。

下一步

- 基于工厂数据构建应用 - 使用 UNS 数据构建工业应用。
- 分析数据 - 使用 Marimo Notebook 和 Python 分析 UNS 数据。

使用 TIER0

从中心到边缘管理数据

从中心节点无缝管理已连接边缘节点上的数据。

在真实工厂中，数据通常由边缘设备采集，并从中心节点统一管理，从而减少网络带宽占用并提升数据安全性。 Tier0 提供 Fleet Management 解决方案，让你可以从中心节点管理边缘节点上的数据。

将边缘节点连接到中心节点

1. 先在中心节点上创建 Fleet 节点。

   > ⓘ 注意
   > 可连接的边缘节点数量取决于许可证。

2. 使用中心节点生成的 Token 连接每个边缘节点。

在边缘与中心之间同步数据

- UNS 数据只能从边缘节点选择并同步到中心节点。
- 你可以从中心节点为边缘节点分配用户，以便进行节点管理。

下一步

- 工厂数据建模 — 了解如何在 UNS 中构建工厂数据模型。
- 准备数据基础 — 构建 namespace 模型并连接源数据。

使用 TIER0

操作工厂数据

通过 MQTT 和 API 操作 UNS 数据，并查看 API 参考。

你可以通过 MQTT 和 API 操作 UNS 数据。

MQTT

MQTT 客户端可以使用 Tier0 生成的凭据连接到 UNS broker。

> ⓘ 注意
> 你可以使用 **MQTTX**、**MQTT Explorer** 和 **Node-RED** 等 MQTT 客户端连接 UNS broker，并发布/订阅消息。

生成 MQTT 凭据

在 Tier0 的 **Edge** 页签创建新的凭据，并记录下来。

连接 MQTT 客户端

> 提示
> 以下客户端作为示例，用来展示如何使用凭据连接 UNS broker。

标签页：MQTTX

1. 安装并打开 MQTTX。
2. 使用 Tier0 UNS broker 信息和生成的凭据添加连接。

```
{
  "Host": "mqtt.tier0.dev",
  "Port": 8883,
  "Client ID": "<generated client ID>",
  "Username": "<generated username>",
  "Password": "<generated password>"
```

```
}
```

标签页：MQTT EXPLORER

1. 安装并打开 MQTT Explorer。
2. 使用 broker 信息和生成的凭据添加新连接。

```
{
  "Protocol": "mqtt://",
  "Host": "mqtt.tier0.dev",
  "Port": 8883,
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

3. 连接前，在 **ADVANCED** 中将 client ID 改为凭据 ID。

标签页：NODE-RED

- 使用 事件流程 - Tier0 中基于 Node-RED 的数据处理模块。
  1. 在 事件流程 中创建数据流，并用 `mqtt in` 节点作为起点。
  2. 在节点中选择 UNS broker 作为 **Server**（与 flow 同名），并将 **Topic** 设置为 UNS 模型。
- 使用独立 Node-RED 实例
  1. 安装并打开 Node-RED 实例。
  2. 在 `mqtt in` 节点中使用以下信息添加 server。

```
{
  {/* under Connection */}
  "Server": "mqtt://mqtt.tier0.dev",
  "Port": 8883,
  "Client ID": "<generated clientId>",
  {/* under Security */}
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

测试连接

1. 确认 client 状态正常。
2. 复制一个 UNS 模型 topic，并通过 client 向它发布一条 message。

API

使用 **REST API** 操作 UNS 数据。

> 🚀 **如何使用 API**
> OpenAPI base path 是 `/openapi/v1`，例如 `https://your-workspace.tier0.app/openapi/v1`，API key 通过 `x-api-key` header 传入。

获取 API Key

API keys 位于 **Settings > API Keys**。不同 account type 允许创建的 key type 不同。

- **Service Key**：只能由 **Admin** 和 **Owner** 创建。
- **Personal Key**：任何人都可以创建，permission scope 与该 account 相同。

> ⚠ **Key permission vs Account permission**
> Account permission 优先于 key permission。即使 key permission 设置为 **Full Access**，account 无权访问的数据仍然无法通过该 key 访问。

示例

- `/uns/create`

  创建 UNS path 或 topic。使用 `namespace` 定义 node tree.

  标签页: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;
```

```
const response = await fetch(`${BASE_URL}/uns/create`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    namespace: [
      {
        name: 'DemoFactory',
        type: 'PATH',
        children: [
          {
            name: 'Site_01',
            type: 'PATH',
            children: [
              {
                name: 'Production',
                type: 'PATH',
                children: [
                  {
                    name: 'Produced_Qty',
                    type: 'TOPIC',
                    topicType: 'METRIC',
                    displayName: 'Produced Quantity',
                    description: 'Produced quantity from the production
line.',
                    enableHistory: true,
                    fields: [
                      { name: 'value', type: 'DOUBLE', unit: 'pcs' },
                    ],
                  },
                ],
              },
            ],
          },
        ],
      },
    ],
  }),
});

const result = await response.json();
console.log(result.data.results);
```

标签页: PYTHON

```python
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/create",
    headers={"x-api-key": api_key},
    json={
        "namespace": [
            {
                "name": "DemoFactory",
                "type": "PATH",
                "children": [
                    {
                        "name": "Site_01",
                        "type": "PATH",
                        "children": [
                            {
                                "name": "Production",
                                "type": "PATH",
                                "children": [
                                    {
                                        "name": "Produced_Qty",
                                        "type": "TOPIC",
                                        "topicType": "METRIC",
                                        "displayName": "Produced
Quantity",
                                        "description": "Produced quantity
from the production line.",
                                        "enableHistory": True,
                                        "fields": [
                                            {"name": "value", "type":
"DOUBLE", "unit": "pcs"},
                                        ],
                                    },
                                ],
                            },
                        ],
                    },
                ],
            },
        ],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
```

```
print(result["data"]["results"])
```

标签页: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/create" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "namespace": [
      {
        "name": "DemoFactory",
        "type": "PATH",
        "children": [
          {
            "name": "Site_01",
            "type": "PATH",
            "children": [
              {
                "name": "Production",
                "type": "PATH",
                "children": [
                  {
                    "name": "Produced_Qty",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "displayName": "Produced Quantity",
                    "description": "Produced quantity from the production
line.",
                    "enableHistory": true,
                    "fields": [
                      { "name": "value", "type": "DOUBLE", "unit": "pcs"
}
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }'
```

- `/uns/read`

读取一个或多个 UNS topics 的最新值和元数据。

标签页: JAVASCRIPT

```javascript
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/read`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    include_leaf_value: true,
    include_metadata: true,
  }),
});

const result = await response.json();
console.log(result.data.results);
```

标签页: PYTHON

```python
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/read",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "include_leaf_value": True,
        "include_metadata": True,
    },
```

```
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

标签页: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/read" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "include_leaf_value": true,
    "include_metadata": true
  }'
```

- `/uns/update`

  通过 `path` 更新 UNS 节点。使用 `updateMask` 指定需要变更的字段。

标签页: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/update`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    path: 'DemoFactory/Site_01/Production/Produced_Qty',
    displayName: 'Produced Quantity',
    description: 'Updated production output quantity.',
    updateMask: ['displayName', 'description'],
  }),
});
```

```
const result = await response.json();
console.log(result.data);
```

标签页: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/update",
    headers={"x-api-key": api_key},
    json={
        "path": "DemoFactory/Site_01/Production/Produced_Qty",
        "displayName": "Produced Quantity",
        "description": "Updated production output quantity.",
        "updateMask": ["displayName", "description"],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

标签页: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/update" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "path": "DemoFactory/Site_01/Production/Produced_Qty",
    "displayName": "Produced Quantity",
    "description": "Updated production output quantity.",
    "updateMask": ["displayName", "description"]
  }'
```

- `/uns/delete`

  删除一个或多个 UNS 模型。使用 `hard_delete: false` 将模型移到回收站。

标签页: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/delete`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    hard_delete: false,
  }),
});

const result = await response.json();
console.log(result.data);
```

标签页: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/delete",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "hard_delete": False,
    },
```

```
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

标签页：CURL

```
curl -X POST "$TIER0_BASE_URL/uns/delete" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "hard_delete": false
  }'
```

API 参考

- Tier0 OpenAPI 参考

> ⚠ **Tier0 Edge 上的 API 密钥限制**
> 从 Tier0 Edge 获取的 API 密钥仅适用于 UNS 和 Flows API。

下一步

- 在 UNS 上构建应用 — 使用 UNS 数据构建工业应用。
- 分析数据 — 使用 Marimo Notebook 和 Python 分析 UNS 数据。

构建应用

通过 AI 对话构建自定义工业应用，并使用工厂数据、权限控制、版本管理和部署能力。

通过 AI 对话构建自定义工业应用。应用可以使用工厂数据，并内置基于角色的访问控制、版本管理和外部部署等能力。

构建 Tier0 应用

构建应用前，请先创建一个项目。

- 项目用于归类应用，并管理应用访问权限。
- 同一个项目中的应用可以作为一个复杂系统中的功能模块单独存在。
- 每个项目中的数据彼此独立。

编写 Prompt

为了让 agent 生成更符合需求的应用，请确保 prompt *至少* 覆盖以下内容。

- 应用目标：这个应用的目的是什么？

  ```
  为工厂技术人员创建一个设备维护应用。
  ```

- 应用数据：应用使用什么数据，数据在哪里？

  标签页：PLATFORM 套餐

  在 prompt 中说明 UNS topic 后，应用可以从 UNS 消费数据，也可以向 UNS 发布数据。

  ```
  该应用应使用 xxx UNS 模型中的设备状态、报警记录和维护历史数据，并将更新发布到 xxx UNS topic。
  ```

标签页：BUILDER 套餐

应用不会连接 UNS。应用使用过程中产生的数据会存储在数据库中。

- 应用功能：应用需要做什么？

  该应用应允许用户监控设备状态、提交维护请求，并跟踪维修进度。

⚠ 当你使用 Builder 套餐时

Builder 套餐目前不支持 UNS，但可以使用模拟数据。如需更具体的数据集成能力，请通过 sales@freezonex.io 联系我们。

🚀 更快构建应用

- 常见工业应用模板已提供在 **模板库** 中。你可以基于模板继续修改功能，从而更快构建应用。
- 使用 内置 Builder Skills 帮助构建应用。
- 当对话过长时，使用 `/clear` 清空上下文，或使用 `/compact` 压缩上下文。

访问控制

基于角色的访问控制会同时应用于项目和应用。

- 项目访问级别
  - **Owner**：拥有项目的完整访问权限。
  - **可编辑**：拥有项目的完整访问权限，但不能删除项目。
  - **可启动**：只能在 **应用中心** 中看到该项目，并使用已被授权的应用。
- 应用角色
  - 如果没有显式定义，agent 会根据应用的业务逻辑自动生成。其他应用中已有且职责匹配的角色也可以复用。
  - 只能分配给项目成员。
  - 如果 prompt 明确说明不需要角色，则该应用会对所有项目成员开放。

> 🚀 如果你想自行指定角色
> 请在 prompt 中说明角色和权限范围，agent 会将其与其他应用已生成的现有角色进行比较。如果存在冲突，agent 会向你确认。

> ⚠ 受角色控制的应用必须分配应用角色
> 要在 **应用中心** 中访问受角色控制的应用，必须分配应用角色。即使是 workspace/project 所有者，如果没有应用角色，也无法访问该应用。

版本管理

应用在 Tier0 中有 3 种状态。

- **草稿**：应用的最新状态。如果没有部署，对该版本的修改不会影响其它版本。
- **快照**：保存创建快照时的应用状态。部署快照版本会替换当前 **启用** 版本。
- **启用**：当前在线运行的应用版本。

> 🚀 版本切换
> - 部署草稿时，启用应用会更新到较新的版本。
> - 部署快照时，启用应用会回滚到较早的版本。

导出到外部

应用准备好后，可以在 **预览** 页面导出，用于外部第三方环境或 Tier0 Enterprise。

- **下载源代码**：导出应用源代码，用于二次开发。
- **导出运行包**：专门用于在 **Tier0 Enterprise** 本地导入。

> 🚀 下载快照的源代码
> 保存快照后再下载，即可获取该快照版本的源代码。

下一步

- 分析数据 - 使用 Marimo Notebook 和 Python 分析 UNS 数据。

- 最佳实践：连接工业协议 - 常见数据源连接示例。

使用 TIER0

构建数字孪生

将实时 UNS 数据绑定到 3D 模型，并组合成完整的工厂场景进行数字孪生可视化。

连接到 Tier0 的工厂数据可以通过 Digital Twin 可视化为具有真实动作和环境的 3D 场景。整体流程是先把数据映射到 3D 模型，再把多个模型组合起来，构建完整的工厂场景。

> ⓘ 需要做 2D 看板？
> 本页介绍的是 **3D 数字孪生可视化** —— 把 UNS 数据绑定到 3D 模型并组合工厂场景。 如果要做 **2D 分析看板**（图表、表格、KPI 面板）请使用 Notebook，参考 构建分析应用。

将数据映射到 3D 模型

1. 在 **模型库** 中上传模型后，选择需要用 UNS 数据驱动实时动作的节点。
2. 在同一个模型上添加绑定不同 UNS 数据的实例，用来模拟不同机器。
3. 将数据绑定到模型，并使用 **预览** 检查模型是否会随着数据变化而移动和旋转。
   - **位置**：模型会沿所选坐标方向移动。
   - **旋转**：模型会围绕坐标原点旋转。
4. 使用 **数据标签** 添加用于静态信息展示的标签。

可视化数据

1. 从 **实例库** 中添加场景里应存在的实例。
2. 使用顶部工具移动、缩放和旋转模型，让模型适配场景，并调整视角和坐标位置。
3. 可以通过右侧的 **对象** 确认模型位置，其中会显示绑定的 UNS 数据。
4. 视口和光照用于调整环境细节。

外部分享

只有实例可以按模型或按场景分享。

- **分享模型**：生成一个 Tier0 平台链接，用于展示带有实时 UNS 数据的模型。

- **分享场景**：生成一个 Tier0 平台链接，用于展示带有实时 UNS 数据和环境设置的模型及其场景。

使用 TIER0

分析数据

在 Tier0 Notebook 中查询 live namespace 数据，并构建交互式分析或机器学习应用。Tier0 云服务版和 Enterprise 可用。

Tier0 中使用 Marimo Notebook，基于已建模并连接到 UNS 的工厂数据，通过 **Python** 脚本进行高级数据分析。

> 开始前须知
> 只有开启 **Enable History** 的 UNS 数据，才会作为可用数据源出现在 **Notebook** 中。

1. 在 UNS 数据中选择 **VARIABLES** > **PostgreSQL** > **tenant_id** > **uns**。
2. 单击 **GENERATE WITH AI**，然后使用 @ 选择要分析的变量。Notebook 支持 fuzzy match 来帮助查找变量。
3. 根据需要继续分析，或回滚生成结果。

> Notebook 中的 AI 辅助
> - Tier0 云服务版可以直接使用 AI 辅助。
> - Tier0 Enterprise 需要配置自己的 AI key。

下一步

- 基于工厂数据构建应用 - 使用 UNS 数据构建工业应用。
- 工厂数据基础 - 理解 Unified Namespace 的数据建模基础。

使用 TIER0

用 Agents 操作 Tier0

使用 AI agents 通过 Skills 和 CLI 命令操作 Tier0。

Tier0 提供 Skills 和 CLI 命令，让 AI agents 可以操作平台。

使用你偏好的 **AI agent** 操作 **Tier0**。

Claude Code　Codex　Gemini CLI　Cursor　GitHub Copilot

安装 Skills 和 CLI

请选择与你的环境匹配的安装方式。

有 Node.js

如果环境中有 Node.js 16 或更高版本，可以使用 `npx` 安装 CLI 和 skills。

```
npx -y @tier0/cli@latest install
```

该命令会运行完整安装器，并完成两件事：

- 将 `tier0` CLI binary 安装到 `~/.tier0/bin/`。
- 安装内置的 Tier0 Skill 基线，并同步到检测到的 agents，例如 Codex、Claude Code 等。

没有 Node.js

1. 安装 `tier0` CLI binary。

标签页：CURL

```
curl -fsSL https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.sh | bash
```

标签页：IWR

```
iwr https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.ps1 | iex
```

2 下载最新 Tier0 Skills，并同步到检测到的 agents。

```
tier0 skills update
```

3 验证已安装的 Skill 版本和健康状态。

```
tier0 skills status
```

管理 Skills

- 检查已安装 Skill 的版本和健康状态：

```
tier0 skills status
```

- 修复缺失或损坏的内置 Skill 基线：

```
tier0 skills install
```

- 更新到最新 Tier0 Skill：

```
```

```
tier0 skills update
```

Skills 和 CLI 如何协作

Tier0 CLI 提供可执行命令，Skills 指导 agents 选择并安全使用这些命令。

> ⓘ 注意
> Tier0 Skills 设计为由 prompt 自动触发，并指导 agents 应该做什么、使用哪些命令。

示例

- Prompt：检查 `Plant/Line1/Metric/Temperature` 的当前温度。
- Skill
  - `tier0-uns`
  - 它会告诉 agent 当前值应从完整 topic path 读取，并选择合适的 UNS read 操作。
- CLI：`tier0 uns read Plant/Line1/Metric/Temperature --json`

Skills

Skill	用途
`tier0`	将 Tier0 任务路由到合适的 skill，并提供共享的设置、认证和安全规则。
`tier0-uns`	指导 agents 执行 UNS 操作，包括 browse、read、write、history、search、create、update、delete 和 restore。
`tier0-flow`	指导 agents 管理 SourceFlow 和 EventFlow，包括检查、导出、创建、更新、部署和删除。
`tier0-mqtt`	指导 agents 管理 MQTT 凭证、发布消息，以及进行有边界的实时订阅。

Skill	用途
`tier0-files`	指导 agents 执行 Tier0 文件操作，包括上传、下载、生成访问 URL 和删除。

> 🚀 让 agent 读取 skill repo
> 可以让你的 agent 读取 Tier0 Skill repo 了解详情。

CLI 操作

设置操作实例

> ⓘ 注意
> Tier0 云服务版默认使用 `https://tier0.dev`，无需配置 base URL。

私有化部署必须在登录前配置 base URL。配置完成后，后续所有命令都会在该实例上执行。

```
tier0 config --base-url https://your-tier0.example.com
tier0 login # interactive browser login
```

认证

标签页：已有 API 密钥

1. 设置你从当前 Tier0 实例获取的 API 密钥。

```
tier0 config --api-key <api-key>
```

2. 验证 API 密钥身份和权限。

```
tier0 auth whoami --json
```

3 使用 `tier0` CLI 执行其他操作。

标签页：没有 API 密钥

1 获取浏览器认证链接。

```
tier0 login --no-wait --json
```

2 访问返回的 `verification_url`，在浏览器中完成认证。

3 使用返回的 `setup_code` 轮询登录结果。

```
tier0 login --setup-code <setup_code>
```

> ⚠ 重要
> 在配置 API 密钥或开始浏览器登录前，先运行 `config` 设置操作实例；否则 CLI 可能会操作到错误实例。

预览操作

使用 `--dry-run` 验证并预览支持的写入操作，不向 Tier0 发送请求。

支持的操作

- UNS：`write/create/update/delete/restore`
- Flow：`create/update/delete/deploy`
- API：`tier0 api`
- MQTT：`auth create/auth delete/publish`
- Files：`delete`

示例

```
tier0 uns write --topic demo --value '{"value":1}' --dry-run
tier0 flow deploy --id 1 --flows-file flows.json --dry-run --json
tier0 api /openapi/v1/uns/write --body-file body.json --dry-run --json
tier0 mqtt auth create --name my-client --save my-client --dry-run --json
tier0 assets delete --file-path workspace/path/to/file.csv --dry-run --json
```

> ⓘ 注意
> `--json` 选项会以 JSON 格式输出请求。

> ⚠ 请求 **body** 必须包含有效 **JSON**
> 如果没有提供有效 JSON，CLI 不再猜测或重写格式错误的 JSON。

返回的预览

> ⓘ 注意
> dry-run 输出不会泄露 headers 和 API keys。

```
{
  "ok": true,
  "dry_run": true,
  "data": {
    "api": [
      {"method": "POST", "url": "https://tier0.dev/openapi/v1/uns/write", "body":
{}}
    ]
  }
}
```

处理错误

使用 `--json` 时，命令失败会返回包含错误消息和错误代码的 JSON object。

```
```

```
{"ok":false,"error":
{"type":"validation","subtype":"invalid_argument","param":"--qos","message":"-
-qos must be 0, 1, or 2"}}
```

卸载 Skills 和 CLI

- 卸载命令

```
# 移除 Tier0 CLI，但保留本地配置和 skills。
npx @tier0/cli@latest uninstall

# 移除 Tier0 CLI，并删除本地配置、凭证和缓存，但保留 skills。
npx @tier0/cli@latest uninstall --purge

# 移除 skills
npx @tier0/cli@latest uninstall --remove-skills

# 移除以上所有内容
npx @tier0/cli@latest uninstall --purge --remove-skills
```

- 全局 NPM 卸载

```
npm uninstall -g @tier0/cli
```

- 手动清理

```
rm -rf ~/.tier0/bin/tier0 ~/.tier0/skills
npx -y --package=skills -- skills remove tier0 -y -g
```

CLI 参考

Tier0 CLI 参考 列出了全部 CLI 命令。

下一步

- 操作工厂数据 — 如何通过 MQTT 和 API 操作你的工厂数据模型。

最佳实践

工厂数据建模

Demo UNS 设计：避免 topic explosion 和 payload structure 的最佳实践。

Demo UNS 如何组织

> 建模前要做什么
> 建模之前，先回答以下两个问题：
>
> - 哪些数据会从 UNS 被消费？
> - 哪些数据会发布回 UNS？

物理数据

来自有物理位置的机器和设备的工业数据，通常基于 ISA-95 标准建模。

示例模型

标签页：ISA-95 层级

```
Enterprise (Level 4)
└── Site (Level 3/4)
    └── Area (Level 3)
        └── Work Center / Process Cell (Level 2/3)
            └── Equipment / Unit (Level 1/2)
                └── Control Module / Device (Level 0/1)
```

标签页：树形结构

```
DemoFactory
└── Site_01
    └── Production
        └── Line_01
            └── WorkOrderExecution
```

```
                ├── Metric
                │   ├── Target_Qty
                │   ├── Produced_Qty
                │   ├── Defect_Qty
                │   └── Completion_Rate
                └── State
                    ├── CurrentWorkOrder
                    └── WorkOrderStatus
```

标签页：JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_9e50bbcd833d",
      "type": "PATH",
      "children": [
        {
          "name": "Site_01",
          "alias": "DemoFactory__6d1fcad14e86",
          "type": "PATH",
          "children": [
            {
              "name": "Production",
              "alias": "__Site_01_Pro_91523d2ad304",
              "type": "PATH",
              "children": [
                {
                  "name": "Line_01",
                  "alias": "__Production__acd5841c1ada",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "WorkOrderExecution",
                      "alias": "__Line_01_Wor_54904ff56e91",
                      "type": "PATH",
                      "children": [
                        {
                          "name": "Metric",
                          "alias": "metric_233fd84305c4",
                          "type": "PATH",
                          "children": [
                            {
                              "name": "Target_Qty",
                              "alias": "Target_Qty_8646b092e543",
                              "type": "TOPIC",
```

```
                    "topicType": "METRIC",
                    "description": "Target quantity for the current
Work Order.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "work_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "value",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "unit",
                        "type": "STRING"
                      },
                      {
                        "name": "timestamp",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  },
                  {
                    "name": "Produced_Qty",
                    "alias": "Produced_Qty_1f8cd3e11f77",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Produced quantity for the
current Work Order.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
```

```
        },
        {
          "name": "plan_number",
          "type": "STRING"
        },
        {
          "name": "work_order_id",
          "type": "STRING"
        },
        {
          "name": "value",
          "type": "DOUBLE"
        },
        {
          "name": "unit",
          "type": "STRING"
        },
        {
          "name": "timestamp",
          "type": "DATETIME"
        }
      ],
      "enableHistory": "TRUE",
      "mockData": "FALSE"
    },
    {
      "name": "Defect_Qty",
      "alias": "Defect_Qty_b48fb3563ec1",
      "type": "TOPIC",
      "topicType": "METRIC",
      "description": "Defect quantity for the current
Work Order.",
      "fields": [
        {
          "name": "production_order_id",
          "type": "STRING"
        },
        {
          "name": "plan_id",
          "type": "STRING"
        },
        {
          "name": "plan_number",
          "type": "STRING"
        },
        {
          "name": "work_order_id",
          "type": "STRING"
        },
        {
          "name": "value",
```

```
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Completion_Rate",
        "alias": "Completion_R_7294e38e1530",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Completion percentage for the
current Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
```

```
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              }
            ]
          },
          {
            "name": "State",
            "alias": "state_539a0f331fe0",
            "type": "PATH",
            "children": [
              {
                "name": "CurrentWorkOrder",
                "alias": "CurrentWorkO_a638320784dd",
                "type": "TOPIC",
                "topicType": "STATE",
                "description": "Current active Work Order on
Line_01.",
                "fields": [
                  {
                    "name": "production_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_number",
                    "type": "STRING"
                  },
                  {
                    "name": "work_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "target_line",
                    "type": "STRING"
                  },
                  {
                    "name": "workstation",
                    "type": "STRING"
                  },
                  {
                    "name": "product_code",
                    "type": "STRING"
                  },
                  {
                    "name": "target_qty",
                    "type": "DOUBLE"
                  },
                  {
```

```
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "started_at",
            "type": "DATETIME"
          },
          {
            "name": "updated_at",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "WorkOrderStatus",
        "alias": "WorkOrderSta_e24e3bf113d1",
        "type": "TOPIC",
        "topicType": "STATE",
        "description": "Execution status of the current
Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "status",
            "type": "STRING"
          },
          {
            "name": "reason",
            "type": "STRING"
          },
          {
            "name": "updated_at",
            "type": "DATETIME"
          }
```

```
                        ],
                        "enableHistory": "TRUE",
                        "mockData": "FALSE"
                    }
                ]
            }
        ]
    }
]
}
```

业务数据

对于 ERP 中的生产订单、工单计划等业务计划和流程对象，应根据业务上下文进行建模。

> 如何识别业务数据
> 没有实际物理位置的数据。

业务数据建模指南

```
Factory / Business Domain / Business Object / Topic Type / Topic
```

- 业务域：确认数据所属的业务流程或系统。

```
ERP
MES
WMS
QMS
EAM
LIMS
Planning
```

```
Inventory
Maintenance
Quality
Logistics
...
```

- **业务对象**：业务域中被管理的核心对象。

```
ProductionOrders
WorkOrderPlan
MaterialInventory
QualityInspection
MaintenanceOrders
PurchaseOrders
Shipments
...
```

- **Topic 类型**：从 **Metric**、**Action** 和 **State** 中选择数据类型。

Topic 类型	用途	示例
STATE	当前状态或当前事实	工单状态、订单状态、设备状态
METRIC	连续变化的数值	温度、速度、产量、OEE、能耗
ACTION	给外部执行者的命令	启动设备、停止设备、重置报警、请求下发工单

示例模型

标签页：树形结构

```
DemoFactory
└── ERP
    ├── ProductionOrders
    │   └── State
    │       ├── UpsertProductionOrder
    │       └── OrderList
```

```
    └── WorkOrderPlan
        ├── Metric
        │   └── SplitCount
        └── State
            ├── PlanStatus
            └── WorkOrderList
```

标签页: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_f925c1ec5041",
      "type": "PATH",
      "children": [
        {
          "name": "ERP",
          "alias": "DemoFactory__e504805b1973",
          "type": "PATH",
          "children": [
            {
              "name": "ProductionOrders",
              "alias": "__ERP_Product_1e7984ac3a1d",
              "type": "PATH",
              "children": [
                {
                  "name": "State",
                  "alias": "state_c2616c8698fa",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "UpsertProductionOrder",
                      "alias": "UpsertProduc_d5f166da35c6",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Event for creating or updating a
Production Order from WOM or ERP simulation.",
                      "fields": [
                        {
                          "name": "event_id",
                          "type": "STRING"
                        },
                        {
                          "name": "event_type",
                          "type": "STRING"
                        },
```

```
                    {
                      "name": "source",
                      "type": "STRING"
                    },
                    {
                      "name": "production_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "order_number",
                      "type": "STRING"
                    },
                    {
                      "name": "product_code",
                      "type": "STRING"
                    },
                    {
                      "name": "planned_qty",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "unit",
                      "type": "STRING"
                    },
                    {
                      "name": "status",
                      "type": "STRING"
                    },
                    {
                      "name": "due_date",
                      "type": "DATETIME"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                },
                {
                  "name": "OrderList",
                  "alias": "OrderList_11fbca51a2bd",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Retained compatibility snapshot of
current Production Orders.",
                  "fields": [
                    {
                      "name": "orders",
                      "type": "STRING"
```

```
                        },
                        {
                          "name": "updated_at",
                          "type": "DATETIME"
                        }
                      ],
                      "enableHistory": "TRUE",
                      "mockData": "FALSE"
                    }
                  ]
                }
              ]
            },
            {
              "name": "WorkOrderPlan",
              "alias": "__ERP_WorkOrd_19f04960daf5",
              "type": "PATH",
              "children": [
                {
                  "name": "Metric",
                  "alias": "metric_17258b809a4f",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "SplitCount",
                      "alias": "SplitCount_032e86f5fbfc",
                      "type": "TOPIC",
                      "topicType": "METRIC",
                      "description": "Number of executable Work Orders
generated from a Work Order Plan.",
                      "fields": [
                        {
                          "name": "production_order_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_number",
                          "type": "STRING"
                        },
                        {
                          "name": "split_count",
                          "type": "INTEGER"
                        },
                        {
                          "name": "updated_at",
                          "type": "DATETIME"
                        }
```

```
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  }
                ]
              },
              {
                "name": "State",
                "alias": "state_884cee548bf7",
                "type": "PATH",
                "children": [
                  {
                    "name": "PlanStatus",
                    "alias": "PlanStatus_2e21c1327c49",
                    "type": "TOPIC",
                    "topicType": "STATE",
                    "description": "Current lifecycle status of a Work Order
Plan.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "status",
                        "type": "STRING"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  },
                  {
                    "name": "WorkOrderList",
                    "alias": "WorkOrderLis_887b92f26fc2",
                    "type": "TOPIC",
                    "topicType": "STATE",
                    "description": "Split result list generated from a sent
Work Order Plan.",
                    "fields": [
```

```
                    {
                      "name": "production_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_number",
                      "type": "STRING"
                    },
                    {
                      "name": "split_count",
                      "type": "INTEGER"
                    },
                    {
                      "name": "lines",
                      "type": "STRING"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

消息结构

设计 topic 消息内容时，需要同时考虑要消费的数据和要回写的数据。

- **Action**

  Action 消息内容会自动包含 `_timestamp`，用于记录数据存入 Tier0 数据库的时间。

标签页: START_JOB

```
{
  "light_turn_on": true,
  "height_adjust": 12.5,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

标签页: STOP_JOB

```
{
  "water_pump_off": true,
  "window_close": true,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

- Metric

  Metric 消息内容会自动包含 `_timestamp` 和 `_quality`：前者记录数据存入 Tier0 数据库的时间，后者用于映射数据质量。

标签页: ERP / SPLITCOUNT

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "split_count": 3,
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```

标签页: SITE_01 / TARGET_QTY

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "value": 1200,
  "unit": "pcs",
  "timestamp": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```


- **State**

  State 消息内容会自动包含 `_timestamp`，用于记录数据存入 Tier0 数据库的时间。

标签页: ERP / UPSERTPRODUCTIONORDER


```
{
  "event_id": "EVT-001",
  "event_type": "UPSERT",
  "source": "ERP",
  "production_order_id": "PO-20260710-001",
  "order_number": "PO-001",
  "product_code": "SKU-1001",
  "planned_qty": 1200,
  "unit": "pcs",
  "status": "Released",
  "due_date": "2026-07-12T00:00:00Z",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```


标签页: SITE_01 / WORKORDERSTATUS


```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
```

```
  "work_order_id": "WO-001",
  "status": "Running",
  "reason": "In production",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

如何避免 Topic 爆炸

Tier0 设计了三种固定的 Topic 类型，`Metric`，`Action` 和 `State` 来避免 topic 爆炸.

- 将 `Metric` 用于高频实时数据

  将高频测量值和遥测数据统一放在 `Metric` 下，而不是为每个信号细节创建 topic。

标签页: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Temperature_Value
        ├── Temperature_Unit
        ├── Temperature_Status
        └── Speed_Value
```

标签页: BETTER

```
Factory_A
└── SMT_Line_1
    └── Metric
        └── Machine_001
```

消息内容:

```
{
  "temperature": 80,
```

```
  "temperature_unit": "Celsius",
  "temperature_status": "Normal",
  "speed": 300
}
```

- 将 `State` 用于低频状态数据

  将低频状态、快照和当前业务状态放在 `State` 下。避免为作业、批次或订单等变化实体分别创建新 topic。

  标签页: BAD

  ```
  Factory_A
  └── SMT_Line_1
      └── State
          ├── Job_001
          ├── Job_002
          └── Job_003
  ```

  标签页: BETTER

  ```
  Factory_A
  └── SMT_Line_1
      └── State
          └── Machine_001
  ```

  消息内容:

  ```
  {
    "current_job": "Job_003",
    "product": "Product_A"
  }
  ```

- 将 `Action` 用于低频动作数据

将命令、请求和操作员动作放在 `Action` 下。动作输入应作为 topic 的字段，而不是为每个参数创建一个 topic。

标签页：BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Start_Command
        ├── Target_Height
        └── Light_On
```

标签页：BETTER

```
Factory_A
└── SMT_Line_1
    └── Action
        └── StartJob
```

消息内容：

```
{
  "target_height": 12.5,
  "light_on": true,
  "requested_by": "operator"
}
```

最佳实践

连接工业协议

使用 Node-RED 将 OPC UA、Modbus 和其他典型协议设备接入 UNS，并提供真实数据流 JSON 参考。

本节以 OPC UA、Modbus 和 API 等常见数据类型为例，展示如何通过 **数据连接** 将数据源连接到 **UNS**。

OPC UA

配置数据流

1. 进入 Tier0 的 **数据流程**，创建一个 **数据连接**。
2. 在数据流中加入 `inject`，`OPC UA Client` 和 `mqtt out` 节点。
3. 按顺序连接节点，并双击每个节点配置详细信息。

- `inject`
  1. 单击左下角 **add** 添加一条属性。
  2. 按照以下内容配置属性。

```
{
  "msg.payload": "[multiple data tags]",
  "msg.topic": "multiple"
}
```

- `OPC UA Client`

```
"Endpoint": "<OPC UA 服务器地址和端口>", //e.g. opc.tcp://127.0.0.1:4850
"Action": "SUBSCRIBE",
"Interval": "<数据采集频率>" //e.g. 10 秒
```

- `mqtt out`

```
"Server": "<内置 UNS broker>", //与数据流同名
"Topic": "<UNS 模型>",
```

4. 部署并触发数据流。

数据流参考

显示 OPC UA 数据流 JSON


```
[
    {
        "id": "fn_mapper_01",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "ISA95 Mapping (Air Compressor)",
        "func": "// OPC UA NodeId → ISA-95 MQTT Topic Mapping\n\nconst mapping
= {\n    \"ns=2;i=2\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/outlet_temperature\
",\n        field: \"temperature\"\n    },\n    \"ns=2;i=3\": {\n
topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/discharge_pressure\
",\n        field: \"pressure\"\n    },\n    \"ns=2;i=4\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/vibration_rms\",\n
field: \"vibration\"\n    }\n};\n\nlet value = msg.payload;\n\n// OPC UA
structure normalize\nif (value && typeof value === \"object\") {\n    if
(value.value?.value !== undefined) {\n        value = value.value.value;\n
} else if (value.value !== undefined) {\n        value = value.value;\n
}\n}\n\nconst m = mapping[msg.topic];\nif (!m) return null;\n\nmsg.topic =
m.topic;\n\nmsg.payload = {\n    [m.field]: Number(value)\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 340,
        "y": 320,
        "wires": [
            [
                "mqtt_out_01"
            ]
        ]
```

```
    },
    {
        "id": "mqtt_out_01",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "MQTT Air Compressor Metrics",
        "topic": "",
        "qos": "0",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 570,
        "y": 200,
        "wires": []
    },
    {
        "id": "1f242deefa9fdba9",
        "type": "OpcUa-Client",
        "z": "b002a07d1163c3de",
        "endpoint": "298ef5d50e9dc411",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 220,
        "y": 220,
        "wires": [
            [
                "fn_mapper_01"
            ],
            [],
            []
        ]
```

```
    },
    {
        "id": "7be5f021ed1f10ef",
        "type": "inject",
        "z": "b002a07d1163c3de",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "multiple",
        "payload": "[{\"nodeId\":\"ns=2;i=2\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=3\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=4\",\"datatype\":\"Float\"}]",
        "payloadType": "json",
        "x": 110,
        "y": 160,
        "wires": [
            [
                "1f242deefa9fdba9"
            ]
        ]
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
```

```
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "298ef5d50e9dc411",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4841",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "a003de8af8e62c75",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```

Modbus

配置数据流

1. 在数据流中加入 `modbus-read` 和 `mqtt out` 节点。
2. 双击节点配置信息。
    - `modbus-read`
        - **Settings**
            - **FC**：选择 Modbus 功能码。例如，FC 3 表示读取保持寄存器。
            - **Address**：设置起始寄存器地址。例如，从 0 开始读取。
            - **Quantity**：设置要连续读取的寄存器数量。
            - **Poll Rate**：设置 Modbus 轮询频率。例如，每 10 秒读取一次。
            - **Server**：选择或创建 Modbus 服务器，配置 IP、端口和协议。
        - **Server**
            - **Host**：Modbus 服务器 IP。
            - **Port**：Modbus 服务器端口。

- `mqtt out`

```
"Server": "<内置 UNS broker>", //与数据流同名
"Topic": "<UNS 模型>",
```

3. 部署数据流。

数据流参考

显示 Modbus 数据流 JSON


```
[
    {
        "id": "d3becb19e0d762f4",
        "type": "modbus-read",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic": "",
        "showStatusActivities": false,
        "logIOActivities": false,
        "showErrors": false,
        "showWarnings": true,
        "unitid": "1",
        "dataType": "HoldingRegister",
        "adr": "0",
        "quantity": "1",
        "rate": "10",
        "rateUnit": "s",
        "delayOnStart": false,
        "startDelayTime": "",
        "server": "c665b74952f6cf5a",
        "useIOFile": false,
        "ioFile": "",
        "useIOForPayload": false,
        "emptyMsgOnFail": false,
        "x": 130,
        "y": 500,
        "wires": [
            [
                "2987638967f632b4",
                "09b8538e09d0766a"
            ],
            []
        ]
```

```
    },
    {
        "id": "b068c119928c029a",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic":
"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_A
ir_System/Air_Compressor_Station/Air_Compressor_01/State/compressor_status",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 510,
        "y": 540,
        "wires": []
    },
    {
        "id": "2987638967f632b4",
        "type": "debug",
        "z": "b002a07d1163c3de",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 260,
        "y": 600,
        "wires": []
    },
    {
        "id": "09b8538e09d0766a",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "function 1",
        "func": "const value = msg.payload;\n\n// 1️⃣ verify\nif (value ===
undefined || value === null) {\n    node.warn(\"Empty Modbus payload\");\n
return null;\n}\n\n// 2️⃣ convert to number\nconst status_code =
Number(value);\n\n// 3️⃣ mapping to status\nconst statusMap = {\n    0:
\"maintenance\",\n    1: \"running\",\n    2: \"standby\",\n    3:
\"debug\",\n    4: \"fault\"\n};\n\n// 4️⃣ filtering\nif (!(status_code in
statusMap)) {\n    node.warn(\"Invalid status_code: \" + status_code);\n
return null;\n}\n\n// 5️⃣ contextualized output\nmsg.payload = {\n
status_code: status_code,\n    status_label: statusMap[status_code],\n
```

```
_valid: true,\n    _ts: new Date().toISOString()\n};\n\n// 6️⃣ optional  set
topic\nmsg.topic = \"v1/aircompressor01/state\";\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 500,
        "wires": [
            [
                "b068c119928c029a"
            ]
        ]
    },
    {
        "id": "c665b74952f6cf5a",
        "type": "modbus-client",
        "name": "modbus",
        "clienttype": "tcp",
        "bufferCommands": true,
        "stateLogEnabled": false,
        "queueLogEnabled": false,
        "failureLogEnabled": true,
        "tcpHost": "172.31.151.237",
        "tcpPort": "5020",
        "tcpType": "DEFAULT",
        "serialPort": "/dev/ttyUSB",
        "serialType": "RTU-BUFFERD",
        "serialBaudrate": 9600,
        "serialDatabits": 8,
        "serialStopbits": 1,
        "serialParity": "none",
        "serialConnectionDelay": 100,
        "serialAsciiResponseStartDelimiter": "0x3A",
        "unit_id": 1,
        "commandDelay": 1,
        "clientTimeout": 1000,
        "reconnectOnTimeout": true,
        "reconnectTimeout": 2000,
        "parallelUnitIdsAllowed": true,
        "showErrors": false,
        "showWarnings": true,
        "showLogs": true
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
```

```
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "5cc203d0788c324d",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-modbus": "5.43.0"
        }
    }
]
```

API

配置数据流

1. 在数据流中加入 `inject`，`http request` 和 `mqtt out` 节点。
2. 双击节点配置信息。
    - `http request`

      ```
      "Method": "<API method>",
      "URL": "<API URL>",
      "Return": "a parsed JSON object",
      "Headers": "<API headers>"
      ```

    - `mqtt out`

      ```
      ```

```
"Server": "<内置 UNS broker>", //与数据流同名
"Topic": "<UNS 模型>",
```

3. 部署数据流。

数据流参考

显示 API 数据流 JSON


```
[
    {
        "id": "46273fcbb98afa11",
        "type": "inject",
        "z": "932355c6886aaf27",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 170,
        "y": 380,
        "wires": [
            [
                "3aca156076ad6dd1"
            ]
        ]
    },
    {
        "id": "3aca156076ad6dd1",
        "type": "http request",
        "z": "932355c6886aaf27",
        "name": "",
        "method": "GET",
        "ret": "obj",
```

```
        "paytoqs": "ignore",
        "url": "http://127.0.0.1:1880/api/wms/picking-zone-a/status",
        "tls": "",
        "persist": false,
        "proxy": "",
        "insecureHTTPParser": false,
        "authType": "",
        "senderr": false,
        "headers": [],
        "x": 380,
        "y": 380,
        "wires": [
            [
                "83fd65cf177c2ddf"
            ]
        ]
    },
    {
        "id": "83fd65cf177c2ddf",
        "type": "mqtt out",
        "z": "932355c6886aaf27",
        "name": "",
        "topic": "v1/Suzhou_Plant/Assembly-Area-1/Line-01/Station-
Screwing/Metric/torque",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djm245hxt9d9",
        "x": 780,
        "y": 440,
        "wires": []
    },
    {
        "id": "broker-djm245hxt9d9",
        "type": "mqtt-broker",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355039328257392",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
```

```
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    }
]
```

节点使用建议

- 发布到 **UNS** 前，可以按需拖入 **function** 节点处理数据。
- 可以使用 **debug** 节点现场检查数据。
- 节点详情见 Node-RED 数据流。

最佳实践

安全与可靠性

Tier0 身份认证、数据访问、部署可用性与恢复机制概览。

Tier0 支持从**安全**和**可靠性**两个方面来确保数据安全。

- **安全**：控制谁能访问平台、应用和数据；
- **可靠性**：发生故障后服务能否继续，以及数据能否恢复。

各版本的安全范围

版本	安全能力范围
云服务版	提供平台访问控制，以及需要身份认证的 MQTT 和 API 数据访问。
企业版	除平台和数据访问控制外，还提供适用于私有部署的部署可用性、备份与恢复方案。
Edge（开源版）	用于本地数据连接与建模，内置 MQTT Broker 支持匿名订阅。

各版本定位请参阅选择合适的产品版本。

> ⓘ **Tier0 云服务版安全说明**
> 有关云服务版生产服务的网络边界、租户隔离、数据保护和安全运维措施，请参阅 **Tier0 云服务版网络与数据安全说明**。该说明仅适用于 Tier0 云服务版。

安全

身份与访问控制

范围	Tier0 机制	详情
平台访问	• 云服务版通过 Workspace 成员资格控制访问。	用户管理

范围	Tier0 机制	详情
	• 企业版通过用户账号控制访问。	
Agent 访问	Agent 使用 API Key 完成身份认证，并通过 Tier0 CLI 访问和操作平台。	CLI 身份认证
项目与应用访问	Tier0 基于 RBAC 实现应用访问控制。	访问控制

> ⓘ 注意
> Workspace 成员资格不代表自动生成账号，用户仍需自行注册并登录账号。账号注册须经过双重验证。

数据访问控制

访问方式	Tier0 机制	详情
MQTT	• 客户端使用 Tier0 生成的 Credential 凭据完成鉴权。 • TLS 和安全 WebSocket 对传输中的数据进行加密。	MQTT 凭据与连接
REST API	必须使用 Tier0 生成的 Key 才能访问数据。	API 参考

可靠性

部署可用性

企业版目前为中心节点提供两种部署方案：

方案	可用性边界	详情
标准部署	• 数据库支持主备故障切换。 • Enterprise 应用和 EMQX 仍为单点。	标准部署方案
高可用部署	• 增加 Enterprise 主备节点，并明确数据库、缓存、对象存储及运行文件的故障切换机制。 • 不承诺零中断或零数据丢失。	高可用部署方案

> ⓘ 注意
> 详细的可用性限制请参阅 SLA 与高可用边界。

备份与恢复

企业版提供数据库备份和恢复。恢复后需要检查部署状态、执行基础功能验证，并确认能够正常登录平台。

- 备份请参阅备份与恢复
- 恢复验证要求请参阅演练要求。

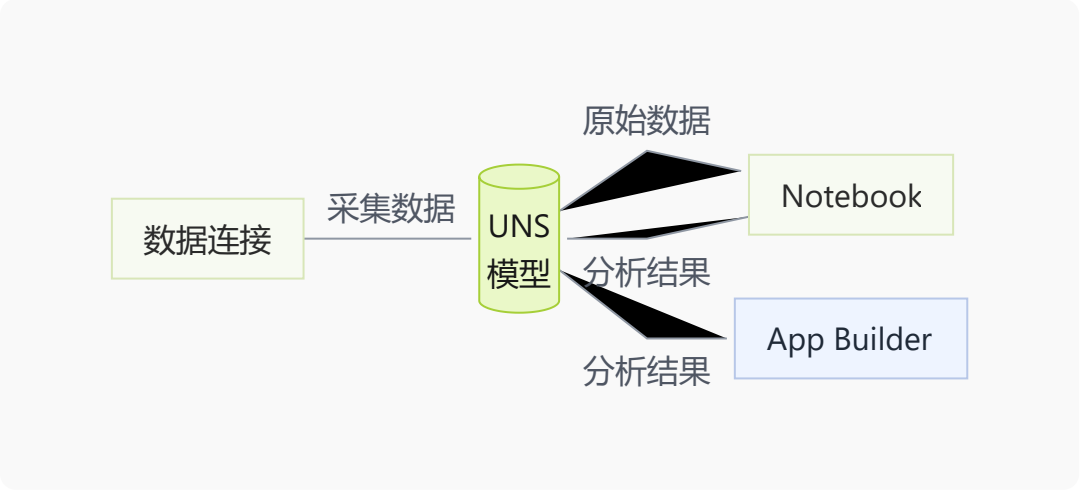
> ⓘ 响应时间
> 一般情况下，响应时间为 3 个工作日。

最佳实践

构建分析应用

使用 Aramco Bowtie 案例（脱敏数据），通过 Notebook 构建分析应用。

Tier0 使用 Marimo Notebook 通过 Python 进行高级数据分析。本指南以 Bowtie 应用作为示例。



示例背景

炼化腐蚀是一个受多种运行因素影响的复杂过程。本示例展示一个实时腐蚀风险评估：结合过程数据估算腐蚀发生概率，并支持主动维护。

获取原始数据

1. 在 Tier0 中进入 **UNS**，导入数据模型。

显示 UNS 模型 JSON

```
{
  "notes":
"type:PATH|TOPIC,topicType:STATE|ACTION|METRIC,schema.type:INTEGER|STRING
|FLOAT|DOUBLE|BOOLEAN|LONG|DATETIME",
  "namespace": [
    {
      "name": "Refinery",
      "type": "PATH",
      "children": [
        {
```

```
          "name": "CDU_Plant",
          "type": "PATH",
          "children": [
            {
              "name": "State",
              "type": "PATH",
              "topicType": "STATE",
              "children": [
                {
                  "name": "Corrosion_Risk",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current corrosion risk state inferred
by Bayesian Network",
                  "enableHistory": "TRUE",
                  "schema": [
                    {
                      "name": "risk_state",
                      "type": "STRING"
                    },
                    {
                      "name": "previous_state",
                      "type": "STRING"
                    },
                    {
                      "name": "model_confidence",
                      "type": "FLOAT"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            },
            {
              "name": "Metric",
              "type": "PATH",
              "topicType": "METRIC",
              "children": [
                {
                  "name": "Corrosion_Monitoring",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Real-time process measurements for CDU
atmospheric overhead corrosion monitoring",
                  "enableHistory": "TRUE",
                  "mockData": "FALSE",
                  "schema": [
                    {
```

```
        "name": "d103_chloride",
        "type": "FLOAT"
      },
      {
        "name": "d103_ph",
        "type": "FLOAT"
      },
      {
        "name": "wash_water_flow",
        "type": "FLOAT"
      },
      {
        "name": "desalter_salt_ptb",
        "type": "FLOAT"
      },
      {
        "name": "desalter_bsw",
        "type": "FLOAT"
      },
      {
        "name": "wash_water_rate",
        "type": "FLOAT"
      },
      {
        "name": "rrd_ph",
        "type": "FLOAT"
      },
      {
        "name": "rrd_chloride",
        "type": "FLOAT"
      },
      {
        "name": "rrd_total_iron",
        "type": "FLOAT"
      },
      {
        "name": "sap_evidence",
        "type": "STRING"
      },
      {
        "name": "injection_quill_fail",
        "type": "BOOLEAN"
      },
      {
        "name": "timestamp",
        "type": "DATETIME"
      }
    ]
  },
  {
    "name": "Corrosion_Analysis",
```

```
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Posterior probabilities generated by
the Bayesian Network",
                    "enableHistory": "TRUE",
                    "mockData": "FALSE",
                    "schema": [
                      {
                        "name": "p_normal",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "p_developing",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "p_confirmed",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "total_corrosion_risk",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "lopc_probability",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "shutdown_probability",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "escalation_probability",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "timestamp",
                        "type": "DATETIME"
                      }
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
}
```

> **For external connections to UNS broker**
> Refer to Connecting MQTT Clients

2 进入 数据流程，创建 数据连接，连接原始数据并发布到 **UNS**。

显示数据连接 JSON


```
[
    {
        "id": "6173469ed5064ccf",
        "type": "tab",
        "label": "corrosion",
        "disabled": false,
        "info": ""
    },
    {
        "id": "ad20b016baf7247f",
        "type": "OpcUa-Client",
        "z": "6173469ed5064ccf",
        "endpoint": "66bd118c33eb12f8",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 280,
        "y": 440,
        "wires": [
            [
                "ac9195bb4863dbdd",
                "5a3d61b2ae40dcb6"
            ],
            [],
            []
```

```
            ]
        },
        {
            "id": "b6746ed066ba0e1a",
            "type": "inject",
            "z": "6173469ed5064ccf",
            "name": "",
            "props": [
                {
                    "p": "payload"
                },
                {
                    "p": "topic",
                    "vt": "str"
                }
            ],
            "repeat": "",
            "crontab": "",
            "once": false,
            "onceDelay": 0.1,
            "topic": "multiple",
            "payload": "[   {     \"name\": \"D103_CHLORIDE\",
\"nodeId\": \"ns=2;i=3\",     \"datatype\": \"Float\"   },   {
\"name\": \"D103_PH\",     \"nodeId\": \"ns=2;i=4\",     \"datatype\":
\"Float\"   },   {     \"name\": \"WASH_WATER_FLOW\",     \"nodeId\":
\"ns=2;i=5\",     \"datatype\": \"Float\"   },   {     \"name\":
\"DESALTER_SALT_PTB\",     \"nodeId\": \"ns=2;i=6\",     \"datatype\":
\"Float\"   },   {     \"name\": \"DESALTER_BSW\",     \"nodeId\":
\"ns=2;i=7\",     \"datatype\": \"Float\"   },   {     \"name\":
\"WASH_WATER_RATE\",     \"nodeId\": \"ns=2;i=8\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_PH\",     \"nodeId\":
\"ns=2;i=9\",     \"datatype\": \"Float\"   },   {     \"name\":
\"RRD_CHLORIDE\",     \"nodeId\": \"ns=2;i=10\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_TOTAL_IRON\",     \"nodeId\":
\"ns=2;i=11\",     \"datatype\": \"Float\"   },   {     \"name\":
\"SAP_EVIDENCE\",     \"nodeId\": \"ns=2;i=12\",     \"datatype\":
\"UInt16\"   },   {     \"name\": \"INJECTION_QUILL_FAIL\",
\"nodeId\": \"ns=2;i=13\",     \"datatype\": \"Boolean\"   } ]",
            "payloadType": "json",
            "x": 90,
            "y": 440,
            "wires": [
                [
                    "ad20b016baf7247f"
                ]
            ]
        },
        {
            "id": "ac9195bb4863dbdd",
            "type": "debug",
            "z": "6173469ed5064ccf",
```

```
            "name": "debug 2",
            "active": true,
            "tosidebar": true,
            "console": false,
            "tostatus": false,
            "complete": "false",
            "statusVal": "",
            "statusType": "auto",
            "x": 490,
            "y": 480,
            "wires": []
        },
        {
            "id": "5a3d61b2ae40dcb6",
            "type": "function",
            "z": "6173469ed5064ccf",
            "name": "function 1",
            "func": "const TOPIC =
\"Refinery/CDU_Plant/Metric/Corrosion_Monitoring\";\n\nconst nodeMap =
{\n    \"ns=2;i=3\": { field: \"d103_chloride\", type: \"float\" },\n
\"ns=2;i=4\": { field: \"d103_ph\", type: \"float\" },\n    \"ns=2;i=5\":
{ field: \"wash_water_flow\", type: \"float\" },\n    \"ns=2;i=6\": {
field: \"desalter_salt_ptb\", type: \"float\" },\n    \"ns=2;i=7\": {
field: \"desalter_bsw\", type: \"float\" },\n    \"ns=2;i=8\": { field:
\"wash_water_rate\", type: \"float\" },\n    \"ns=2;i=9\": { field:
\"rrd_ph\", type: \"float\" },\n    \"ns=2;i=10\": { field:
\"rrd_chloride\", type: \"float\" },\n    \"ns=2;i=11\": { field:
\"rrd_total_iron\", type: \"float\" },\n    \"ns=2;i=12\": { field:
\"sap_evidence\", type: \"sap\" },\n    \"ns=2;i=13\": { field:
\"injection_quill_fail\", type: \"boolean\" }\n};\n\nconst sapMap = {\n
0: \"none\",\n    1: \"work_order\",\n    2:
\"inspection_finding\"\n};\n\nfunction normalizeNodeId(v) {\n    if (!v)
return \"\";\n    if (typeof v === \"string\") return v;\n    if
(v.toString) return v.toString();\n    return \"\";\n}\n\nfunction
extractValue(payload) {\n    // OPC UA DataValue: payload.value.value\n
if (payload && payload.value && payload.value.value !== undefined) {\n
return payload.value.value;\n    }\n\n    // Some OPC UA nodes output
Variant directly.\n    if (payload && payload.value !== undefined) {\n
return payload.value;\n    }\n\n    return payload;\n}\n\nfunction
extractNodeId(msg) {\n    return normalizeNodeId(\n        msg.nodeId
||\n        msg.topic ||\n        msg.payload?.nodeId ||\n
msg.payload?.nodeid\n    );\n}\n\nconst nodeId =
extractNodeId(msg);\nconst spec = nodeMap[nodeId];\n\nif (!spec) {\n
node.warn(`Unknown OPC UA nodeId: ${nodeId}`);\n    return
null;\n}\n\nlet value = extractValue(msg.payload);\n\nif (spec.type ===
\"float\") {\n    value = Number(value);\n} else if (spec.type ===
\"boolean\") {\n    value = Boolean(value);\n} else if (spec.type ===
\"sap\") {\n    value = sapMap[Number(value)] || \"unknown\";\n}\n\nlet
latest = context.get(\"latest\") || {};\nlatest[spec.field] =
value;\nlatest.timestamp = new
Date().toISOString();\ncontext.set(\"latest\", latest);\n\n// Publish
```

```
only after all 11 fields have been received at least once.\nconst
requiredFields = Object.values(nodeMap).map(x => x.field);\nconst ready =
requiredFields.every(field => latest[field] !== undefined);\n\nif
(!ready) {\n    node.status({\n        fill: \"yellow\",\n        shape:
\"ring\",\n        text: `waiting ${Object.keys(latest).length - 1}/11`\n
});\n    return null;\n}\n\nmsg.topic = TOPIC;\nmsg.payload = {\n
d103_chloride: latest.d103_chloride,\n    d103_ph: latest.d103_ph,\n
wash_water_flow: latest.wash_water_flow,\n    desalter_salt_ptb:
latest.desalter_salt_ptb,\n    desalter_bsw: latest.desalter_bsw,\n
wash_water_rate: latest.wash_water_rate,\n    rrd_ph: latest.rrd_ph,\n
rrd_chloride: latest.rrd_chloride,\n    rrd_total_iron:
latest.rrd_total_iron,\n    sap_evidence: latest.sap_evidence,\n
injection_quill_fail: latest.injection_quill_fail,\n    timestamp:
latest.timestamp\n};\n\nnode.status({\n    fill: \"green\",\n    shape:
\"dot\",\n    text: \"published corrosion monitoring\"\n});\n\nreturn
msg;",
          "outputs": 1,
          "timeout": 0,
          "noerr": 0,
          "initialize": "",
          "finalize": "",
          "libs": [],
          "x": 370,
          "y": 580,
          "wires": [
              [
                  "aa15be33a84fb812",
                  "4fdd68dda3ee5c9e"
              ]
          ]
      },
      {
          "id": "aa15be33a84fb812",
          "type": "mqtt out",
          "z": "6173469ed5064ccf",
          "name": "",
          "topic": "",
          "qos": "",
          "retain": "",
          "respTopic": "",
          "contentType": "",
          "userProps": "",
          "correl": "",
          "expiry": "",
          "broker": "broker-djx75cwgiywq",
          "x": 550,
          "y": 580,
          "wires": []
      },
      {
          "id": "4fdd68dda3ee5c9e",
```

```
        "type": "debug",
        "z": "6173469ed5064ccf",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 450,
        "y": 680,
        "wires": []
    },
    {
        "id": "66bd118c33eb12f8",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4850",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "broker-djx75cwgiywq",
        "type": "mqtt-broker",
        "z": "6173469ed5064ccf",
        "name": "corrosion",
        "broker": "emqx",
        "port": "1883",
        "clientid": "357356352615696",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "c5201f1ad482b182",
        "type": "global-config",
```

```
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```

在 Notebook 中构建分析应用

1. 在 Tier0 中进入 **Notebook**，创建一个新的 notebook。
2. 打开该 notebook，添加以下 cell 来分析 **UNS** 中的数据。
    - 单元 1 - 导入所需依赖，并配置 MQTT broker、topics 和运行时设置。

显示代码

```
@app.cell
def _():
    """Imports and runtime configuration."""
    import json
    import math
    import os
    import threading
    import time
    from datetime import datetime, timezone

    import marimo as mo
    import pandas as pd
    import paho.mqtt.client as mqtt

    MQTT_BROKER_HOST = os.getenv("TIER0_MQTT_HOST", "enterprise-new-
dev.tier0.dev")
    MQTT_BROKER_PORT = int(os.getenv("TIER0_MQTT_PORT", "1883"))
    MQTT_USERNAME = os.getenv("TIER0_MQTT_USERNAME",
"357890055321856")
    MQTT_PASSWORD = os.getenv("TIER0_MQTT_PASSWORD",
"caa6b34242002a3dff4948dd73ce8a")
    MQTT_CLIENT_ID = os.getenv(
        "TIER0_MQTT_CLIENT_ID",
        "357890055321856_bn",
    )

    SOURCE_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Monitoring"
    ANALYSIS_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Analysis"
```

```
    RISK_TOPIC = "Refinery/CDU_Plant/State/Corrosion_Risk"

    SOURCE_QOS = int(os.getenv("TIER0_SOURCE_QOS", "1"))
    OUTPUT_QOS = int(os.getenv("TIER0_OUTPUT_QOS", "1"))
    AUTOSTART = os.getenv("TIER0_BN_AUTOSTART", "true").lower() in {
        "1", "true", "yes", "on"
    }
    ROOT_CONFIDENCE = float(os.getenv("TIER0_BN_ROOT_CONFIDENCE",
"0.85"))

    def utc_now_iso() -> str:
        return (
            datetime.now(timezone.utc)
            .isoformat(timespec="milliseconds")
            .replace("+00:00", "Z")
        )

    mo.md(
        """
        # CDU Corrosion Bayesian Analysis

        This notebook subscribes to a complete corrosion-monitoring
snapshot,
        performs one Bayesian inference for each new source message,
and publishes
        the analysis results back to the UNS.
        """
    )
    return (
        ANALYSIS_TOPIC,
        AUTOSTART,
        MQTT_BROKER_HOST,
        MQTT_BROKER_PORT,
        MQTT_CLIENT_ID,
        MQTT_PASSWORD,
        MQTT_USERNAME,
        OUTPUT_QOS,
        RISK_TOPIC,
        ROOT_CONFIDENCE,
        SOURCE_QOS,
        SOURCE_TOPIC,
        datetime,
        json,
        math,
        mo,
        mqtt,
        os,
        pd,
        threading,
        time,
        timezone,
```

```
        utc_now_iso,
    )
```

> 外部连接 UNS broker
> 参考 连接 MQTT 客户端

- 单元 2 - 定义 Bayesian Network 拓扑、状态空间、CPT 生成器和概率规则。

显示代码

```python
@app.cell
def _(math, pd):
    """Static Bayesian-network definition and CPT generators."""
    import itertools

    from pgmpy.factors.discrete import TabularCPD
    from pgmpy.inference import VariableElimination
    from pgmpy.models import DiscreteBayesianNetwork

    STATE_SPACES = {
        "STATE_3": ["LOW", "NORMAL", "HIGH"],
        "STATE_YN": ["NO", "YES"],
        "PB2_3STATE": ["FAIL", "IL_OK", "SL_OK"],
        "TOP_EVENT_STATE": ["NORMAL", "DEVELOPING", "CONFIRMED"],
    }

    NODE_STATE_SPACE = {
        "T1_HighChlorideLoad": "STATE_3",
        "T2_ChemProtectionFail": "STATE_3",
        "T3_WashWaterLow": "STATE_3",
        "PB1_Desalter_OK": "STATE_YN",
        "PB2_WashQty_State": "PB2_3STATE",
        "PB3_Neutralization_OK": "STATE_YN",
        "MB1_ESD_OK": "STATE_YN",
        "MB2_TSV_OK": "STATE_YN",
        "TE_CorrosionState": "TOP_EVENT_STATE",
        "EV_TOTIRON_High": "STATE_YN",
        "EV_SAP_Positive": "STATE_YN",
        "EV_QuillFailure": "STATE_YN",
        "C1_LOPC": "STATE_YN",
        "C2_UnplannedShutdown": "STATE_YN",
        "C3_Escalation": "STATE_YN",
    }

    EDGES = [
```

```
        ("T1_HighChlorideLoad", "TE_CorrosionState"),
        ("T2_ChemProtectionFail", "TE_CorrosionState"),
        ("T3_WashWaterLow", "TE_CorrosionState"),
        ("PB1_Desalter_OK", "TE_CorrosionState"),
        ("PB2_WashQty_State", "TE_CorrosionState"),
        ("PB3_Neutralization_OK", "TE_CorrosionState"),
        ("TE_CorrosionState", "EV_TOTIRON_High"),
        ("TE_CorrosionState", "EV_SAP_Positive"),
        ("TE_CorrosionState", "EV_QuillFailure"),
        ("TE_CorrosionState", "C1_LOPC"),
        ("MB1_ESD_OK", "C1_LOPC"),
        ("MB2_TSV_OK", "C1_LOPC"),
        ("TE_CorrosionState", "C2_UnplannedShutdown"),
        ("MB1_ESD_OK", "C2_UnplannedShutdown"),
        ("C1_LOPC", "C3_Escalation"),
        ("MB1_ESD_OK", "C3_Escalation"),
        ("MB2_TSV_OK", "C3_Escalation"),
    ]

    def states_for(node_id: str) -> list[str]:
        return STATE_SPACES[NODE_STATE_SPACE[node_id]]

    def softmax(values: list[float]) -> list[float]:
        maximum = max(values)
        exps = [math.exp(value - maximum) for value in values]
        total = sum(exps)
        return [value / total for value in exps]

    THREAT_SCORE = {"LOW": 0.0, "NORMAL": 1.0, "HIGH": 2.0}
    PB1_OK_BONUS = {"NO": 0.8, "YES": -0.6}
    PB3_OK_BONUS = {"NO": 1.0, "YES": -0.7}
    PB2_STATE_BONUS = {"FAIL": 1.2, "IL_OK": 0.3, "SL_OK": -0.6}

    def top_event_distribution(t1, t2, t3, pb1, pb2, pb3) ->
dict[str, float]:
        score = (
            THREAT_SCORE[t1]
            + THREAT_SCORE[t2]
            + THREAT_SCORE[t3]
            + PB1_OK_BONUS[pb1]
            + PB2_STATE_BONUS[pb2]
            + PB3_OK_BONUS[pb3]
        )
        probabilities = softmax(
            [
                2.2 - score,
                -0.2 + 0.6 * score,
                -1.2 + 0.8 * score,
            ]
        )
```

```
        return dict(zip(STATE_SPACES["TOP_EVENT_STATE"],
probabilities))

    TE_PARENTS = [
        "T1_HighChlorideLoad",
        "T2_ChemProtectionFail",
        "T3_WashWaterLow",
        "PB1_Desalter_OK",
        "PB2_WashQty_State",
        "PB3_Neutralization_OK",
    ]

    TE_PARENT_SPACES = [states_for(parent) for parent in TE_PARENTS]
    te_rows = []
    for combination in itertools.product(*TE_PARENT_SPACES):
        distribution = top_event_distribution(*combination)
        te_rows.append(
            [
                *combination,
                distribution["NORMAL"],
                distribution["DEVELOPING"],
                distribution["CONFIRMED"],
            ]
        )
    TE_CPT_DF = pd.DataFrame(
        te_rows,
        columns=TE_PARENTS
        + ["P_NORMAL", "P_DEVELOPING", "P_CONFIRMED"],
    )

    def evidence_yes_probability(te_state: str, kind: str) -> float:
        mappings = {
            "TOTIRON": {"NORMAL": 0.15, "DEVELOPING": 0.65,
"CONFIRMED": 0.85},
            "SAP": {"NORMAL": 0.08, "DEVELOPING": 0.35, "CONFIRMED":
0.75},
            "QUILL": {"NORMAL": 0.03, "DEVELOPING": 0.15,
"CONFIRMED": 0.55},
        }
        return mappings[kind][te_state]

    def lopc_yes_probability(te: str, mb1: str, mb2: str) -> float:
        base = {"NORMAL": 0.02, "DEVELOPING": 0.08, "CONFIRMED":
0.25}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0) + (0.18
if mb2 == "NO" else 0))

    def shutdown_yes_probability(te: str, mb1: str) -> float:
        base = {"NORMAL": 0.05, "DEVELOPING": 0.35, "CONFIRMED":
0.60}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0))
```

```
    def escalation_yes_probability(lopc: str, mb1: str, mb2: str) ->
float:
        if lopc == "NO":
            return 0.01
        return min(0.98, 0.10 + (0.18 if mb1 == "NO" else 0) + (0.35
if mb2 == "NO" else 0))

    def peaked_prior(node_id: str, center_state: str, confidence:
float) -> list[float]:
        states = states_for(node_id)
        confidence = min(max(float(confidence), 0.0), 1.0)
        if center_state not in states:
            return [1.0 / len(states)] * len(states)
        if len(states) == 1:
            return [1.0]
        remainder = (1.0 - confidence) / (len(states) - 1)
        probabilities = [remainder] * len(states)
        probabilities[states.index(center_state)] = confidence
        return probabilities

    def binary_cpd(child: str, parent: str, yes_probability_fn):
        parent_states = states_for(parent)
        yes_values = [float(yes_probability_fn(state)) for state in
parent_states]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=[parent],
            evidence_card=[len(parent_states)],
            state_names={child: states_for(child), parent:
parent_states},
        )

    def binary_multi_cpd(child: str, parents: list[str],
yes_probability_fn):
        parent_spaces = [states_for(parent) for parent in parents]
        combinations = list(itertools.product(*parent_spaces))
        yes_values = [float(yes_probability_fn(*combination)) for
combination in combinations]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=parents,
            evidence_card=[len(space) for space in parent_spaces],
            state_names={
                child: states_for(child),
```

```
                **{parent: states_for(parent) for parent in
parents},
            },
        )

    return (
        DiscreteBayesianNetwork,
        EDGES,
        NODE_STATE_SPACE,
        STATE_SPACES,
        TE_CPT_DF,
        TE_PARENTS,
        TabularCPD,
        VariableElimination,
        binary_cpd,
        binary_multi_cpd,
        escalation_yes_probability,
        evidence_yes_probability,
        itertools,
        lopc_yes_probability,
        peaked_prior,
        shutdown_yes_probability,
        states_for,
    )
```

- 单元 3 - 校验传入的 MQTT 消息内容，并将连续过程值转换为 Bayesian Network 状态。

显示代码

```
@app.cell
def _(ROOT_CONFIDENCE, utc_now_iso):
    """Source validation, discretization, and one-snapshot
analysis."""
    import hashlib

    REQUIRED_SOURCE_FIELDS = [
        "d103_chloride",
        "d103_ph",
        "wash_water_flow",
        "desalter_salt_ptb",
        "desalter_bsw",
        "wash_water_rate",
        "rrd_ph",
        "rrd_chloride",
        "rrd_total_iron",
        "sap_evidence",
```

```
        "injection_quill_fail",
        "timestamp",
    ]

    def parse_boolean(value) -> bool:
        if isinstance(value, bool):
            return value
        if isinstance(value, (int, float)):
            return bool(value)
        if isinstance(value, str):
            normalized = value.strip().lower()
            if normalized in {"true", "1", "yes", "y", "on"}:
                return True
            if normalized in {"false", "0", "no", "n", "off"}:
                return False
        raise ValueError(f"invalid boolean value: {value!r}")

    def normalize_sap(value) -> str:
        normalized = str(value).strip().lower()
        if normalized in {"0", "none", "normal", "", "nan"}:
            return "None"
        if normalized in {"1", "work_order", "work order", "minor",
"warning"}:
            return "Minor corrosion observed"
        if normalized in {
            "2",
            "inspection_finding",
            "inspection finding",
            "confirmed",
            "abnormal",
        }:
            return "Confirmed corrosion / thickness loss"
        raise ValueError(f"unsupported sap_evidence value:
{value!r}")

    def normalize_source_payload(payload: dict) -> dict:
        if not isinstance(payload, dict):
            raise TypeError("source MQTT payload must be a JSON
object")

        lower = {str(key).lower(): value for key, value in
payload.items()}
        missing = [field for field in REQUIRED_SOURCE_FIELDS if
field not in lower]
        if missing:
            raise ValueError(f"source payload is missing fields:
{missing}")

        normalized = {
            "d103_chloride": float(lower["d103_chloride"]),
            "d103_ph": float(lower["d103_ph"]),
```

```
            "wash_water_flow": float(lower["wash_water_flow"]),
            "desalter_salt_ptb": float(lower["desalter_salt_ptb"]),
            "desalter_bsw": float(lower["desalter_bsw"]),
            "wash_water_rate": float(lower["wash_water_rate"]),
            "rrd_ph": float(lower["rrd_ph"]),
            "rrd_chloride": float(lower["rrd_chloride"]),
            "rrd_total_iron": float(lower["rrd_total_iron"]),
            "sap_evidence": normalize_sap(lower["sap_evidence"]),
            "injection_quill_fail":
parse_boolean(lower["injection_quill_fail"]),
            "timestamp": str(lower["timestamp"]).strip(),
        }

        numeric_ranges = {
            "d103_chloride": (0.0, 1000.0),
            "d103_ph": (0.0, 14.0),
            "wash_water_flow": (0.0, 10000.0),
            "desalter_salt_ptb": (0.0, 1000.0),
            "desalter_bsw": (0.0, 100.0),
            "wash_water_rate": (0.0, 100.0),
            "rrd_ph": (0.0, 14.0),
            "rrd_chloride": (0.0, 10000.0),
            "rrd_total_iron": (0.0, 10000.0),
        }
        for field, (minimum, maximum) in numeric_ranges.items():
            value = normalized[field]
            if not minimum <= value <= maximum:
                raise ValueError(f"{field} out of accepted range:
{value}")

        if not normalized["timestamp"]:
            raise ValueError("timestamp cannot be empty")
        return normalized

    def source_message_id(payload: dict) -> str:
        # Timestamp is the primary event key. Hash protects against
two different
        # snapshots accidentally sharing the same timestamp.
        ordered = "|".join(str(payload[field]) for field in
REQUIRED_SOURCE_FIELDS)
        return hashlib.sha256(ordered.encode("utf-8")).hexdigest()

    def discretize_source(payload: dict) -> tuple[dict, dict]:
        t1 = (
            "LOW"
            if payload["d103_chloride"] < 5.0
            else "NORMAL"
            if payload["d103_chloride"] <= 20.0
            else "HIGH"
        )
```

```
        chemical_failure_score = int(payload["d103_ph"] < 5.5) +
int(
            payload["d103_chloride"] > 10.0
        )
        t2 = (
            "HIGH"
            if chemical_failure_score == 2
            else "NORMAL"
            if chemical_failure_score == 1
            else "LOW"
        )

        t3 = (
            "HIGH"
            if payload["wash_water_flow"] < 6.0
            else "NORMAL"
            if payload["wash_water_flow"] <= 10.0
            else "LOW"
        )

        pb1 = (
            "YES"
            if payload["desalter_salt_ptb"] <= 1.0
            and payload["desalter_bsw"] <= 0.2
            else "NO"
        )

        pb2 = (
            "FAIL"
            if payload["wash_water_rate"] < 4.0
            else "IL_OK"
            if payload["wash_water_rate"] < 6.0
            else "SL_OK"
        )

        pb3 = (
            "YES"
            if payload["rrd_ph"] >= 5.5 and payload["rrd_chloride"]
<= 10.0
            else "NO"
        )

        root_centers = {
            "T1_HighChlorideLoad": t1,
            "T2_ChemProtectionFail": t2,
            "T3_WashWaterLow": t3,
            "PB1_Desalter_OK": pb1,
            "PB2_WashQty_State": pb2,
            "PB3_Neutralization_OK": pb3,
            # The source model currently has no MB fields. Keep both
mitigations
```

```
            # available by default, matching the previous notebook
UI defaults.
            "MB1_ESD_OK": "YES",
            "MB2_TSV_OK": "YES",
        }

        evidence = {
            "EV_TOTIRON_High": "YES" if payload["rrd_total_iron"] >=
10.0 else "NO",
            "EV_SAP_Positive": "YES" if payload["sap_evidence"] !=
"None" else "NO",
            "EV_QuillFailure": "YES" if
payload["injection_quill_fail"] else "NO",
        }
        return root_centers, evidence

    return (
        REQUIRED_SOURCE_FIELDS,
        discretize_source,
        normalize_source_payload,
        parse_boolean,
        source_message_id,
    )
```

- 单元 4 - 基于当前快照构建 Bayesian Network，执行推理并生成输出消息内容。

显示代码

```
@app.cell
def _(
    DiscreteBayesianNetwork,
    EDGES,
    ROOT_CONFIDENCE,
    STATE_SPACES,
    TE_CPT_DF,
    TE_PARENTS,
    TabularCPD,
    VariableElimination,
    binary_cpd,
    binary_multi_cpd,
    discretize_source,
    escalation_yes_probability,
    evidence_yes_probability,
    itertools,
    lopc_yes_probability,
    peaked_prior,
    shutdown_yes_probability,
```

```
    states_for,
    utc_now_iso,
):
    """Build a fresh BN for the current snapshot and execute
inference."""

    def factor_probability(factor, variable: str, state: str) ->
float:
        states = factor.state_names[variable]
        values = list(map(float, factor.values))
        return float(values[states.index(state)])

    def build_model(root_centers: dict):
        model = DiscreteBayesianNetwork(EDGES)

        root_cpds = []
        for node_id, center_state in root_centers.items():
            states = states_for(node_id)
            probabilities = peaked_prior(node_id, center_state,
ROOT_CONFIDENCE)
            root_cpds.append(
                TabularCPD(
                    variable=node_id,
                    variable_card=len(states),
                    values=[[probability] for probability in
probabilities],
                    state_names={node_id: states},
                )
            )

        te_values = [
            TE_CPT_DF["P_NORMAL"].astype(float).tolist(),
            TE_CPT_DF["P_DEVELOPING"].astype(float).tolist(),
            TE_CPT_DF["P_CONFIRMED"].astype(float).tolist(),
        ]
        te_cpd = TabularCPD(
            variable="TE_CorrosionState",
            variable_card=3,
            values=te_values,
            evidence=TE_PARENTS,
            evidence_card=[len(states_for(parent)) for parent in
TE_PARENTS],
            state_names={
                "TE_CorrosionState":
STATE_SPACES["TOP_EVENT_STATE"],
                **{parent: states_for(parent) for parent in
TE_PARENTS},
            },
        )

        evidence_cpds = [
```

```
            binary_cpd(
                "EV_TOTIRON_High",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"TOTIRON"),
            ),
            binary_cpd(
                "EV_SAP_Positive",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"SAP"),
            ),
            binary_cpd(
                "EV_QuillFailure",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"QUILL"),
            ),
        ]

        consequence_cpds = [
            binary_multi_cpd(
                "C1_LOPC",
                ["TE_CorrosionState", "MB1_ESD_OK", "MB2_TSV_OK"],
                lopc_yes_probability,
            ),
            binary_multi_cpd(
                "C2_UnplannedShutdown",
                ["TE_CorrosionState", "MB1_ESD_OK"],
                shutdown_yes_probability,
            ),
            binary_multi_cpd(
                "C3_Escalation",
                ["C1_LOPC", "MB1_ESD_OK", "MB2_TSV_OK"],
                escalation_yes_probability,
            ),
        ]

        model.add_cpds(*root_cpds, te_cpd, *evidence_cpds,
*consequence_cpds)
        if not model.check_model():
            raise RuntimeError("Bayesian network model validation
failed")
        return model

    def analyze_snapshot(payload: dict, previous_risk_state: str |
None = None) -> dict:
        root_centers, evidence = discretize_source(payload)
        model = build_model(root_centers)
        inference = VariableElimination(model)
```

```
        q_te = inference.query(["TE_CorrosionState"],
evidence=evidence, show_progress=False)
        q_c1 = inference.query(["C1_LOPC"], evidence=evidence,
show_progress=False)
        q_c2 = inference.query(["C2_UnplannedShutdown"],
evidence=evidence, show_progress=False)
        q_c3 = inference.query(["C3_Escalation"], evidence=evidence,
show_progress=False)

        p_normal = factor_probability(q_te, "TE_CorrosionState",
"NORMAL")
        p_developing = factor_probability(q_te, "TE_CorrosionState",
"DEVELOPING")
        p_confirmed = factor_probability(q_te, "TE_CorrosionState",
"CONFIRMED")
        probabilities = {
            "NORMAL": p_normal,
            "DEVELOPING": p_developing,
            "CONFIRMED": p_confirmed,
        }
        risk_state = max(probabilities, key=probabilities.get)
        analysis_timestamp = utc_now_iso()

        analysis_payload = {
            "p_normal": p_normal,
            "p_developing": p_developing,
            "p_confirmed": p_confirmed,
            "total_corrosion_risk": p_developing + p_confirmed,
            "lopc_probability": factor_probability(q_c1, "C1_LOPC",
"YES"),
            "shutdown_probability": factor_probability(
                q_c2, "C2_UnplannedShutdown", "YES"
            ),
            "escalation_probability": factor_probability(q_c3,
"C3_Escalation", "YES"),
            "timestamp": analysis_timestamp,
        }

        risk_payload = {
            "risk_state": risk_state,
            "previous_state": previous_risk_state or risk_state,
            "model_confidence": max(probabilities.values()),
            "timestamp": analysis_timestamp,
        }

        return {
            "source_timestamp": payload["timestamp"],
            "root_states": root_centers,
            "evidence": evidence,
            "analysis_payload": analysis_payload,
            "risk_payload": risk_payload,
```

```
            "risk_state": risk_state,
        }

    return analyze_snapshot, build_model, factor_probability
```

- 单元 5 - 订阅源 topic，处理每个传入快照，执行 Bayesian 分析并发布结果。

显示代码

```
@app.cell
def _(
    ANALYSIS_TOPIC,
    AUTOSTART,
    MQTT_BROKER_HOST,
    MQTT_BROKER_PORT,
    MQTT_CLIENT_ID,
    MQTT_PASSWORD,
    MQTT_USERNAME,
    OUTPUT_QOS,
    RISK_TOPIC,
    SOURCE_QOS,
    SOURCE_TOPIC,
    analyze_snapshot,
    json,
    mqtt,
    normalize_source_payload,
    source_message_id,
    threading,
    utc_now_iso,
):
    """Long-running MQTT subscriber and publisher service."""

    class CorrosionBNService:
        def __init__(self):
            self.lock = threading.RLock()
            self.client = None
            self.running = False
            self.connected = False
            self.last_message_id = None
            self.last_source_timestamp = None
            self.last_analysis_timestamp = None
            self.previous_risk_state = None
            self.processed_count = 0
            self.duplicate_count = 0
            self.error_count = 0
            self.last_error = None
            self.last_result = None
```

```
            self.connection_rc = None

        def _create_client(self):
            # Use MQTT 3.1.1 and callback API v1 for broad
compatibility with
            # EMQX and different paho-mqtt releases bundled in
Notebook images.
            kwargs = {
                "client_id": MQTT_CLIENT_ID,
                "clean_session": True,
                "protocol": mqtt.MQTTv311,
                "transport": "tcp",
            }
            try:
                client = mqtt.Client(

callback_api_version=mqtt.CallbackAPIVersion.VERSION1,
                    **kwargs,
                )
            except (AttributeError, TypeError):
                client = mqtt.Client(**kwargs)

            client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
            client.on_connect = self._on_connect
            client.on_disconnect = self._on_disconnect
            client.on_message = self._on_message
            client.reconnect_delay_set(min_delay=1, max_delay=30)
            client.enable_logger()
            return client

        def _on_connect(self, client, userdata, flags, rc,
properties=None):
            code = int(rc)
            with self.lock:
                self.connected = code == 0
                self.connection_rc = code
                self.last_error = None if code == 0 else f"MQTT
connection rejected, rc={code}"
            if code == 0:
                sub_rc, mid = client.subscribe(SOURCE_TOPIC,
qos=SOURCE_QOS)
                if sub_rc != mqtt.MQTT_ERR_SUCCESS:
                    with self.lock:
                        self.last_error = f"MQTT subscribe failed,
rc={sub_rc}"
                    return
                print(f"[BN SERVICE] connected and subscribed to
{SOURCE_TOPIC}")
            else:
                print(f"[BN SERVICE ERROR] connection rejected rc=
{code}")
```

```
        def _on_disconnect(self, client, userdata, rc,
properties=None):
            with self.lock:
                self.connected = False
                self.connection_rc = int(rc) if rc is not None else
None
                if self.running and rc:
                    self.last_error = f"MQTT disconnected
unexpectedly, rc={rc}"
            print(f"[BN SERVICE] disconnected: {rc}")

        def _publish_json(self, topic: str, payload: dict):
            info = self.client.publish(
                topic,
                payload=json.dumps(payload, separators=(",", ":"),
ensure_ascii=False),
                qos=OUTPUT_QOS,
                retain=False,
            )
            info.wait_for_publish(timeout=10)
            if info.rc != mqtt.MQTT_ERR_SUCCESS:
                raise RuntimeError(f"MQTT publish failed for
{topic}: rc={info.rc}")

        def _on_message(self, client, userdata, msg):
            try:
                raw_payload = json.loads(msg.payload.decode("utf-
8"))
                payload = normalize_source_payload(raw_payload)
                message_id = source_message_id(payload)

                with self.lock:
                    if message_id == self.last_message_id:
                        self.duplicate_count += 1
                        return
                    previous_risk_state = self.previous_risk_state

                result = analyze_snapshot(payload,
previous_risk_state)

                # Publish only after both payloads have been
generated successfully.
                self._publish_json(ANALYSIS_TOPIC,
result["analysis_payload"])
                self._publish_json(RISK_TOPIC,
result["risk_payload"])

                with self.lock:
                    self.last_message_id = message_id
```

```
                    self.last_source_timestamp =
payload["timestamp"]
                    self.last_analysis_timestamp =
result["analysis_payload"]["timestamp"]
                    self.previous_risk_state = result["risk_state"]
                    self.processed_count += 1
                    self.last_result = result
                    self.last_error = None

                print(
                    "[BN SERVICE] processed",
                    payload["timestamp"],
                    "->",
                    result["risk_state"],
                )
            except Exception as exc:
                with self.lock:
                    self.error_count += 1
                    self.last_error = f"{type(exc).__name__}: {exc}"
                print("[BN SERVICE ERROR]", self.last_error)

        def start(self):
            with self.lock:
                if self.running:
                    return False
                if not MQTT_PASSWORD:
                    raise RuntimeError(
                        "TIER0_MQTT_PASSWORD is empty. Set it before
starting the service."
                    )
                self.client = self._create_client()
                self.running = True
            try:
                # Synchronous connect surfaces DNS, socket, and
authentication
                # failures immediately instead of leaving
running=True forever.
                rc = self.client.connect(
                    MQTT_BROKER_HOST,
                    MQTT_BROKER_PORT,
                    keepalive=30,
                )
                if rc != mqtt.MQTT_ERR_SUCCESS:
                    raise RuntimeError(f"MQTT connect() failed, rc=
{rc}")
                self.client.loop_start()
                return True
            except Exception as exc:
                with self.lock:
                    self.running = False
                    self.connected = False
```

```
                    self.last_error = f"{type(exc).__name__}: {exc}"
                    self.client = None
                raise

        def stop(self):
            with self.lock:
                if not self.running:
                    return False
                client = self.client
                self.running = False
                self.connected = False
                self.client = None
            try:
                client.disconnect()
            finally:
                client.loop_stop()
            return True

        def restart(self):
            self.stop()
            return self.start()

        def status(self) -> dict:
            with self.lock:
                return {
                    "running": self.running,
                    "connected": self.connected,
                    "connection_rc": self.connection_rc,
                    "client_id": MQTT_CLIENT_ID,
                    "broker": f"{MQTT_BROKER_HOST}:
{MQTT_BROKER_PORT}",
                    "source_topic": SOURCE_TOPIC,
                    "analysis_topic": ANALYSIS_TOPIC,
                    "risk_topic": RISK_TOPIC,
                    "processed_count": self.processed_count,
                    "duplicate_count": self.duplicate_count,
                    "error_count": self.error_count,
                    "last_source_timestamp":
self.last_source_timestamp,
                    "last_analysis_timestamp":
self.last_analysis_timestamp,
                    "previous_risk_state": self.previous_risk_state,
                    "last_error": self.last_error,
                }

    # Reuse a previous service instance when the cell is re-
executed, preventing
    # duplicate subscribers inside the same Python process.
    existing = globals().get("_CORROSION_BN_SERVICE")
    if existing is not None:
        try:
```

```
            existing.stop()
        except Exception:
            pass

    CORROSION_BN_SERVICE = CorrosionBNService()
    globals()["_CORROSION_BN_SERVICE"] = CORROSION_BN_SERVICE

    AUTOSTART_ERROR = None
    if AUTOSTART:
        try:
            CORROSION_BN_SERVICE.start()
        except Exception as exc:
            AUTOSTART_ERROR = f"{type(exc).__name__}: {exc}"

    return AUTOSTART_ERROR, CORROSION_BN_SERVICE, CorrosionBNService
```

- 单元 6 - 显示 MQTT 连接状态和服务统计信息。

显示代码

```
@app.cell
def _(AUTOSTART_ERROR, CORROSION_BN_SERVICE, mo, pd):
    """Service status and usage instructions."""
    status = CORROSION_BN_SERVICE.status()

    blocks = [
        mo.md("## MQTT analysis service"),
        pd.DataFrame([status]),
    ]
    if status.get("last_error"):
        blocks.append(mo.md(f"### Connection / processing
error\n`{status['last_error']}`"))
    elif status.get("running") and not status.get("connected"):
        blocks.append(mo.md("### Connecting\nThe MQTT client has
started but has not completed the broker connection yet."))

    if AUTOSTART_ERROR:
        blocks.append(
            mo.md(
                f"""
                ### Service not started

                `{AUTOSTART_ERROR}`

                Set the MQTT password in the Notebook environment
and run this cell again:
```

```
                ~~~python
                import os
                # MQTT credentials are already configured in this
notebook.
                ~~~
                """
            )
        )
    else:
        blocks.append(
            mo.md(
                """
                The service starts automatically and processes one
complete source
                snapshot for every new MQTT message. No manual
analysis trigger or SQL
                polling is required.

                Available controls in Python:

                ~~~python
                CORROSION_BN_SERVICE.status()
                CORROSION_BN_SERVICE.stop()
                CORROSION_BN_SERVICE.start()
                CORROSION_BN_SERVICE.restart()
                ~~~
                """
            )
        )

    mo.vstack(blocks)
    return status,
```

- 单元 7 - 显示最新的 Bayesian 分析结果。

显示代码

```
@app.cell
def _(CORROSION_BN_SERVICE, mo):
    """Display the most recent analysis result when one is
available."""
    latest = CORROSION_BN_SERVICE.last_result
    if latest is None:
        mo.md("### Latest result\nNo source message has been
processed yet.")
    else:
        mo.vstack(
```

```
            [
                mo.md("### Latest source snapshot result"),
                mo.md(f"- Source timestamp:
`{latest['source_timestamp']}`"),
                mo.md(f"- Risk state: `{latest['risk_state']}`"),
                mo.md("#### Root states"),
                latest["root_states"],
                mo.md("#### Evidence"),
                latest["evidence"],
                mo.md("#### Corrosion analysis payload"),
                latest["analysis_payload"],
                mo.md("#### Corrosion risk payload"),
                latest["risk_payload"],
            ]
        )
    return


if __name__ == "__main__":
    app.run()
```

3 运行所有单元，然后进入 **UNS** 检查结果。

构建 Bow-tie 应用

1 在 Tier0 中进入 **Builder**。

2 在对话框中输入应用需求，并开始构建。

> ⚠ 必须说明的内容
> 请告诉 agent 使用 UNS 中的数据，并提供具体 topic。否则 agent 可能难以展示正确的数据。

> 如果你在另一个 Tier0 上构建应用
> 使用不同的 **clientId**（添加不同后缀）连接 UNS broker，避免 client ID 冲突。

显示 Builder prompt

```
为 CDU 常压塔顶腐蚀构建一个现代工业 Bow-Tie Analysis 应用。
```

仅使用以下 UNS topic 作为数据源：

- Refinery/CDU_Plant/Metric/Corrosion_Monitoring
- Refinery/CDU_Plant/Metric/Corrosion_Analysis
- Refinery/CDU_Plant/State/Corrosion_Risk

当新的 UNS 消息到达时，应用必须自动更新。

该应用应帮助操作员理解当前腐蚀风险、相关威胁、屏障状态以及可能后果。

创建一个单页 dashboard，包含：

1. 将 Bow-Tie 图作为主区域：
- 左侧显示威胁
- 在威胁与顶事件之间显示预防性屏障
- 以腐蚀作为中心顶事件
- 在顶事件之后显示缓解性屏障
- 右侧显示后果

2. 显著展示风险摘要，包括：
- 当前风险状态
- 模型置信度
- 正常、发展中、已确认三类概率
- 总腐蚀风险

3. 一个紧凑的实时过程数据面板，显示 Corrosion_Monitoring 中的关键值，包括 chloride、pH、wash water flow、desalter performance、wash water rate、total iron、SAP evidence 和 injection quill status。

4. 后果指标包括：
- 一次包容失效
- 非计划停机
- 升级风险

5. 视觉状态规则：
- 绿色表示正常或健康状态
- 琥珀色/黄色表示风险发展中或警告状态
- 红色表示风险已确认、屏障失效或严重后果
- 灰色表示数据不可用
- 蓝色仅用于交互和信息高亮

6. 交互：
- 选择威胁、屏障或后果时，应显示其当前状态、相关过程值和简短说明。
- 当新的 UNS 数据到达时，自动更新界面。
- 显示最新数据时间戳，以及内置 SDK 提供的连接/订阅状态。

设计要求：

在整个应用中保持现有视觉设计语言和样式。

使用现代工业运营 dashboard 风格：

- 使用深蓝灰色调，而不是纯黑色。
- 使用带有细微渐变的分层面板。
- 使用卡片式布局，圆角较大，约 16-20px。
- 使用细边框做分隔，避免厚重阴影。
- 使用柔和层次感和克制的发光效果。
- 保持清晰间距、充足内边距和一致对齐。
- 正文使用 IBM Plex Sans。
- 标题使用 Space Grotesk。
- 使用紧凑、信息密集且层级清晰的卡片。
- 使用增加字距的大写 section label。
- 呈现专业的炼化厂 / SCADA 界面，而不是商业分析 dashboard。
- 使用扁平插图和 SVG 图形，避免拟物元素。
- 使用平滑、细微的过渡和 hover 效果。

整体外观应清晰、现代、技术化，并适合运营场景。

避免：
- 纯黑背景
- 玻璃拟态
- 赛博朋克风格
- 霓虹灯效
- 过度装饰效果
- 商业分析 dashboard 风格

使用清晰、专业的炼化运营风格，保持明确层级、紧凑卡片、可读标签和响应式布局。

严格使用对应 UNS 模型中定义的 payload 字段。不要虚构额外字段、计算值或 mock 数据。

3 经过几轮对话，确认应用完成后，部署应用。

4 进入 应用中心，打开应用并检查。

最佳实践

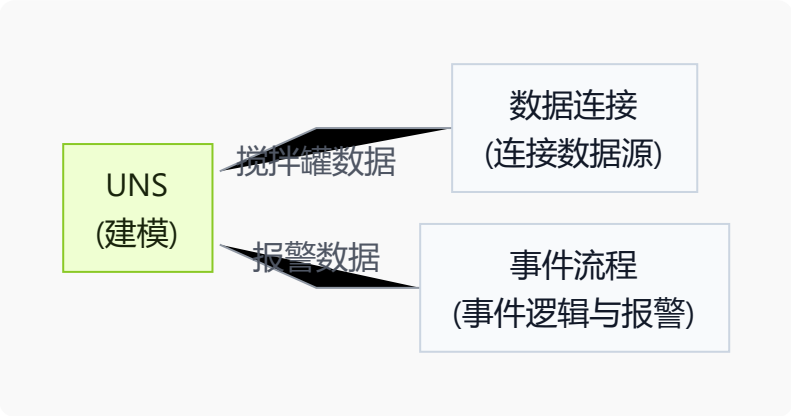
用 Agent 构建车间工作流

使用 UNS Agent 构建覆盖数据建模、数据连接和数据事件的工作流。

UNS Agent 支持通过自然语言对话简单快速对工厂数据进行建模，连接机器数据并针对数据构建报警事件。

工作流背景

一个搅拌罐在加热材料的同时完成混合。构建数据模型并采集温度、水位和加热器状态，用于判断搅拌罐是否过热并触发报警。



如何构建工作流

1. 在 **UNS** 中使用 `full_access` 与 UNS Agent 开始对话。
2. 输入提示词来构建 UNS 模型。

   ```
   创建一个表示搅拌罐温度、水位和加热器状态的数据模型，温度和水位放在一个 metric topic 中，加热器状态放在一个 state topic 中。
   ```

3. 确认模型完整后，输入提示词，让 Agent 将对应数据连接到模型。

   ```
   创建一个数据连接，向这 2 个 topic 发送数据，并模拟合理的数据。
   ```

> 🚀 当你有真实数据源时
> 告诉 Agent 你的数据源信息，让它完成连接。

4 输入提示词，按照以下逻辑创建事件。

- 低水位预警：当水位低于 20% 且加热器开启时触发。
- 干烧报警：当水位低于 15%、温度高于 90°C 且加热器开启时触发。

```
为以下报警逻辑创建一个事件流程：
  - Warning：当 level < 20 且 heater_status = true 时触发。
  - Critical：当 level < 15、temperature > 90 且 heater_status = true 时触发。
  - 当 level > 25 或 heater_status = false 时清除当前报警。
在现有 UNS 模型下创建一个新 topic 来接收报警结果。输出中包含报警等级、消息、是否激活和时间戳。
```

5 在 **UNS** 中检查结果。

> ⓘ 注意
> 每一步完成后，都可以继续和 Agent 对话来优化结果。

最佳实践

使用数字孪生展示数据

在 3D 模型和场景中实时查看 UNS 数据。

Tier0 的数字孪生功能用于在逼真的 3D 模型和场景中实时查看 UNS 数据。 本节会把一个车间场景构建出来。该车间包含以下设备：

- 2 台车床
- 1 台在车床之间工作的机械臂
- 1 个进料缓存站和 1 个出料缓存区
- 1 台质量检测设备
- 1 套统一控制系统
- 2 个原料货架和 2 个成品货架
- 1 台 AGV

从车间采集数据

将所有数据采集到 **UNS**，包括控制机器动作的数据、表示机器状态的数据，以及车间环境数据。

1 在 **UNS** 中构建数据模型。

查看 UNS 模型 JSON


```
{
"namespace": [
    {
    "name": "Metric",
    "alias": "metric_151a7edd68bf4269a229",
    "type": "PATH",
    "children": [
        {
        "name": "RobotArm",
        "alias": "Robot_0_9d5dc450dc1f4c0c9a86",
        "type": "TOPIC",
        "topicType": "METRIC",
        "extendProperties": {
            "J0_Cur": "0"
```

```
        },
        "fields": [
            {
            "name": "J1_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J1_Init",
            "type": "FLOAT"
            },
            {
            "name": "J2_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J2_Init",
            "type": "FLOAT"
            },
            {
            "name": "J3_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J3_Init",
            "type": "FLOAT"
            },
            {
            "name": "J4_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J4_Init",
            "type": "FLOAT"
            },
            {
            "name": "J5_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J5_Init",
            "type": "FLOAT"
            },
            {
            "name": "J6_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J6_Init",
            "type": "FLOAT"
            },
            {
```

```
            "name": "Model_Name",
            "type": "STRING"
            },
            {
            "name": "Max_Payload_kg",
            "type": "FLOAT"
            },
            {
            "name": "Reach_Radius_mm",
            "type": "FLOAT"
            },
            {
            "name": "Gripper_Force_N",
            "type": "FLOAT"
            }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
        },
        {
        "name": "ProLathe-01",
        "alias": "ProLathe_ef325e0a9db047d6abac",
        "type": "TOPIC",
        "topicType": "METRIC",
        "fields": [
            {
            "name": "J_Changer_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Door_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Finger_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Spindle_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_TailStock_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_X_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Y_Cur",
```

```
            "type": "FLOAT"
            },
            {
            "name": "J_Z_Cur",
            "type": "FLOAT"
            },
            {
            "name": "Model_Name",
            "type": "STRING"
            },
            {
            "name": "Max_Placement_Rate_cph",
            "type": "INTEGER"
            },
            {
            "name": "Placement_Speed_cph",
            "type": "INTEGER"
            },
            {
            "name": "Feeder_Usage_pct",
            "type": "INTEGER"
            },
            {
            "name": "Machine_State_code",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "ProLathe-02",
        "alias": "ProLathe2_ef325e0a9db047d6abac",
        "type": "TOPIC",
        "topicType": "METRIC",
        "fields": [
            {
            "name": "J_Changer_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Door_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Finger_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Spindle_Cur",
            "type": "FLOAT"
```

```
        },
        {
        "name": "J_TailStock_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J_X_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J_Y_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J_Z_Cur",
        "type": "FLOAT"
        },
        {
        "name": "Model_Name",
        "type": "STRING"
        },
        {
        "name": "Max_Placement_Rate_cph",
        "type": "INTEGER"
        },
        {
        "name": "Placement_Speed_cph",
        "type": "INTEGER"
        },
        {
        "name": "Feeder_Usage_pct",
        "type": "INTEGER"
        },
        {
        "name": "Machine_State_code",
        "type": "INTEGER"
        }
    ],
    "enableHistory": "FALSE",
    "mockData": "FALSE"
    },
    {
    "name": "AGV",
    "alias": "AGV_7eddee0054a0",
    "type": "TOPIC",
    "topicType": "METRIC",
    "fields": [
        {
        "name": "Position_X_Cur",
        "type": "FLOAT"
        },
```

```
{
"name": "Position_X_Init",
"type": "FLOAT"
},
{
"name": "Position_Y_Cur",
"type": "FLOAT"
},
{
"name": "Position_Y_Init",
"type": "FLOAT"
},
{
"name": "Rotation_Y_Cur",
"type": "FLOAT"
},
{
"name": "Rotation_Y_Init",
"type": "FLOAT"
},
{
"name": "Tray_Height_Cur",
"type": "FLOAT"
},
{
"name": "Tray_Height_Init",
"type": "FLOAT"
},
{
"name": "State_Cur",
"type": "STRING"
},
{
"name": "State_Init",
"type": "STRING"
},
{
"name": "Position_Z_Cur",
"type": "FLOAT"
},
{
"name": "Position_Z_Init",
"type": "FLOAT"
},
{
"name": "Model_Name",
"type": "STRING"
},
{
"name": "Max_Load_kg",
"type": "FLOAT"
```

```
        },
        {
        "name": "Task_Progress_pct",
        "type": "INTEGER"
        },
        {
        "name": "Remaining_Distance_m",
        "type": "FLOAT"
        },
        {
        "name": "AGV_State_code",
        "type": "INTEGER"
        }
    ],
    "enableHistory": "FALSE",
    "mockData": "FALSE"
    },
    {
    "name": "RobotArm-1",
    "alias": "RobotArm_1_b9f06e21346d",
    "type": "TOPIC",
    "topicType": "METRIC",
    "fields": [
        {
        "name": "J1_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J1_Init",
        "type": "FLOAT"
        },
        {
        "name": "J2_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J2_Init",
        "type": "FLOAT"
        },
        {
        "name": "J3_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J3_Init",
        "type": "FLOAT"
        },
        {
        "name": "J4_Cur",
        "type": "FLOAT"
        },
```

```
            {
            "name": "J4_Init",
            "type": "FLOAT"
            },
            {
            "name": "J5_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J5_Init",
            "type": "FLOAT"
            },
            {
            "name": "J6_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J6_Init",
            "type": "FLOAT"
            }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
        }
    ]
    },
    {
    "name": "State",
    "alias": "state_3912d0df3fee",
    "type": "PATH",
    "children": [
        {
        "name": "RawMaterialStorage",
        "alias": "RawMaterialS_c3307f680405",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
            {
            "name": "StockAmount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "ProductStorage",
        "alias": "ProductStora_3c145f140bd9",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
```

```
            {
            "name": "StockAmount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "InboundBuffer",
        "alias": "LoaderBuffer_9feece9b3231",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
            {
            "name": "Amount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "OutboundBuffer",
        "alias": "OutboundBuff_75433b98fde6",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
            {
            "name": "Amount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        }
    ]
    }
]
}
```


2 使用 数据连接 将数据源连接到 UNS 模型。

- Robot Arm

查看机械臂数据连接 JSON

```
[
    {
        "id": "03933a7eb1a7b992",
        "type": "function",
        "z": "8daeb61ebd9e5f52",
        "name": "Cache Data",
        "func": "const CONFIG = {\n    fields: ['Joint_1', 'Joint_2',
'Joint_3', 'Joint_4', 'Joint_5', 'Joint_6'],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        time_stamp:
'time_stamp',\n        quality: 'quality'\n    }\n};\n\nlet cache =
flow.get('dataCache') || { instant: {}, metadata: {} };\n\nconst
topicParts = msg.topic.split('/');\nconst fieldName =
topicParts[topicParts.length - 1];\n\nlet payload = msg.payload;\nif
(typeof payload === 'string') {\n    try {\n        payload =
JSON.parse(payload);\n    } catch (e) {\n        return null;\n
}\n}\n\nlet value = payload[CONFIG.payloadFields.value];\nif (value ===
undefined) return null;\n\nconst metadata = {\n    timeStamp:
payload[CONFIG.payloadFields.timeStamp],\n    time_stamp:
payload[CONFIG.payloadFields.time_stamp],\n    quality:
payload[CONFIG.payloadFields.quality],\n    name:
payload[CONFIG.payloadFields.name],\n    initialValue:
payload[CONFIG.payloadFields.initialValue]\n};\n\nif
(CONFIG.fields.includes(fieldName)) {\n    cache.instant[fieldName] =
value;\n    cache.metadata[fieldName] = metadata;\n} else {\n    return
null;\n}\n\nflow.set('dataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 880,
        "y": 500,
        "wires": []
    },
    {
        "id": "0c4dbdcbe5e3db92",
        "type": "mqtt in",
        "z": "8daeb61ebd9e5f52",
        "name": "Subscribe Joint_1~6",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/Transporter_Kawasaki_BX100N/Mo
tor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "c11decb07a68083d",
        "nl": false,
        "rap": true,
        "rh": 0,
```

```
        "inputs": 0,
        "x": 610,
        "y": 480,
        "wires": [
            [
                "03933a7eb1a7b992"
            ]
        ]
    },
    {
        "id": "ba42bca5aa9dc561",
        "type": "inject",
        "z": "8daeb61ebd9e5f52",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "vt": "str",
                "p": "topic"
            }
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 520,
        "y": 600,
        "wires": [
            [
                "0fc94449c377ad9a"
            ]
        ]
    },
    {
        "id": "0fc94449c377ad9a",
        "type": "function",
        "z": "8daeb61ebd9e5f52",
        "name": "Aggregate & Publish (Flat)",
        "func": "const CONFIG = {\n    fieldMapping: {\n
'Joint_1': 'J1',\n        'Joint_2': 'J2',\n        'Joint_3': 'J3',\n
'Joint_4': 'J4',\n        'Joint_5': 'J5',\n        'Joint_6': 'J6'\n
},\n    missingStrategy: 'lastValue',\n    outputTopic: 'robot/state',\n
windowMs: 100\n};\n\nconst FIELDS =
Object.keys(CONFIG.fieldMapping);\n\nlet cache = flow.get('dataCache') ||
{ instant: {}, metadata: {} };\nlet lastValues = flow.get('lastValues')
|| {};\nlet lastMetadata = flow.get('lastMetadata') || {};\n\nlet output
```

```
= {\n    timestamp: new Date().toISOString()\n};\n\nfor (const field of
FIELDS) {\n    const shortName = CONFIG.fieldMapping[field];\n    \n
if (cache.instant[field] !== undefined) {\n        output[shortName +
'_Cur'] = cache.instant[field];\n        output[shortName + '_Init'] =
cache.metadata[field] ? cache.metadata[field].initialValue : 0;\n
output[shortName + '_Qly'] = cache.metadata[field] ?
cache.metadata[field].quality : 0;\n        \n        lastValues[field] =
cache.instant[field];\n        lastMetadata[field] =
cache.metadata[field];\n    } else {\n        let fallbackValue =
CONFIG.missingStrategy === 'lastValue' \n            ? (lastValues[field]
!== undefined ? lastValues[field] : 0) \n            : 0;\n        \n
output[shortName + '_Cur'] = fallbackValue;\n        output[shortName +
'_Init'] = lastMetadata[field] ? lastMetadata[field].initialValue : 0;\n
output[shortName + '_Qly'] = -1;\n    }\n}\n\nmsg.payload =
output;\nmsg.topic = CONFIG.outputTopic;\n\nflow.set('dataCache', {
instant: {}, metadata: {} });\nflow.set('lastValues',
lastValues);\nflow.set('lastMetadata', lastMetadata);\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 760,
        "y": 600,
        "wires": [
            [
                "c9eee2264fe78846",
                "a685279629e73d9a",
                "4ce075fda1d3d993",
                "8e8779918eab6f89"
            ]
        ]
    },
    {
        "id": "c9eee2264fe78846",
        "type": "mqtt out",
        "z": "8daeb61ebd9e5f52",
        "name": "Publish robot/state",
        "topic": "Metric/RobotArm",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "9836929a70c74929",
        "x": 1070,
        "y": 540,
        "wires": []
```

```
    },
    {
        "id": "c11decb07a68083d",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "9836929a70c74929",
        "type": "mqtt-broker",
        "z": "8daeb61ebd9e5f52",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "3364626855530 88&336463047745872",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
```

```
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```

- AGV

查看 AGV 数据连接 JSON

```
[
    {
        "id": "fa46cf31be8d751e",
        "type": "tab",
        "label": "Metric/AGV",
        "disabled": false,
        "info": ""
    },
    {
        "id": "1f0e9d8c7b6a5a41",
        "type": "mqtt in",
        "z": "fa46cf31be8d751e",
        "name": "Subscribe AGV telemetry",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/AGV_zOnron/Motor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "c11decb07a68083d",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 190,
        "y": 380,
        "wires": [
            [
                "2a1b3c4d5e6f7081"
            ]
```

```
        ]
    },
    {
        "id": "2a1b3c4d5e6f7081",
        "type": "function",
        "z": "fa46cf31be8d751e",
        "name": "Cache AGV telemetry",
        "func": "const CONFIG = {\n    fields: [\n        'Position_X',\n
'Position_Y',\n        'Position_Z',\n        'Rotation_Y',\n
'Tray_Height',\n        'State'\n    ],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        timeStampAlt:
'time_stamp'\n    }\n};\n\nlet cache = flow.get('agvDataCache') || {
instant: {}, metadata: {} };\n\nconst topicParts = (msg.topic ||
'').split('/');\nconst fieldName = topicParts[topicParts.length - 1];\nif
(!CONFIG.fields.includes(fieldName)) {\n    return null;\n}\n\nlet
payload = msg.payload;\nif (typeof payload === 'string') {\n    try {\n
payload = JSON.parse(payload);\n    } catch (error) {\n        return
null;\n    }\n}\n\nif (typeof payload !== 'object' || payload === null)
{\n    return null;\n}\n\nconst currentValue =
payload[CONFIG.payloadFields.value];\nif (currentValue === undefined) {\n
return null;\n}\n\ncache.instant[fieldName] =
currentValue;\ncache.metadata[fieldName] = {\n    name:
payload[CONFIG.payloadFields.name] || fieldName,\n    initialValue:
payload[CONFIG.payloadFields.initialValue],\n    timeStamp:
payload[CONFIG.payloadFields.timeStampAlt] ??
payload[CONFIG.payloadFields.timeStamp] ??
0\n};\n\nflow.set('agvDataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 470,
        "y": 380,
        "wires": []
    },
    {
        "id": "3b2c1d0e9f8a7b61",
        "type": "inject",
        "z": "fa46cf31be8d751e",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
```

```
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 150,
        "y": 500,
        "wires": [
            [
                "4c3d2e1f0a9b8c71"
            ]
        ]
    },
    {
        "id": "4c3d2e1f0a9b8c71",
        "type": "function",
        "z": "fa46cf31be8d751e",
        "name": "Aggregate AGV UNS payload",
        "func": "const CONFIG = {\n    fields: [\n        'Position_X',\n
'Position_Y',\n        'Position_Z',\n        'Rotation_Y',\n
'Tray_Height',\n        'State'\n    ],\n    outputTopic: 'Metric/AGV',\n
defaultNumeric: 0,\n    defaultString: ''\n};\n\nlet cache =
flow.get('agvDataCache') || { instant: {}, metadata: {} };\nlet
lastValues = flow.get('agvLastValues') || {};\nlet lastMetadata =
flow.get('agvLastMetadata') || {};\nlet lastTimestamp =
flow.get('agvLastTimestamp') || 0;\n\nlet output = {};\nlet
candidateTimestamps = [];\n\nfor (const field of CONFIG.fields) {\n
const hasCurrentFrameValue =
Object.prototype.hasOwnProperty.call(cache.instant, field);\n    const
metadata = hasCurrentFrameValue ? cache.metadata[field] :
lastMetadata[field];\n    const currentValue = hasCurrentFrameValue ?
cache.instant[field] : lastValues[field];\n    const isStringField =
field === 'State';\n\n    output[field + '_Cur'] = currentValue !==
undefined\n        ? currentValue\n        : (isStringField ?
CONFIG.defaultString : CONFIG.defaultNumeric);\n    output[field +
'_Init'] = metadata && metadata.initialValue !== undefined\n        ?
metadata.initialValue\n        : (isStringField ? CONFIG.defaultString :
CONFIG.defaultNumeric);\n\n    if (hasCurrentFrameValue) {\n
lastValues[field] = cache.instant[field];\n        lastMetadata[field] =
cache.metadata[field];\n\n        const rawTime = cache.metadata[field] ?
Number(cache.metadata[field].timeStamp) : NaN;\n        if
(!Number.isNaN(rawTime)) {\n
candidateTimestamps.push(rawTime);\n        }\n    }\n}\n\nif
(candidateTimestamps.length > 0) {\n    lastTimestamp =
Math.max(...candidateTimestamps);\n}\n\noutput.timestamp =
lastTimestamp;\n\nmsg.topic = CONFIG.outputTopic;\nmsg.payload =
output;\n\nflow.set('agvDataCache', { instant: {}, metadata: {}
});\nflow.set('agvLastValues', lastValues);\nflow.set('agvLastMetadata',
```

```
lastMetadata);\nflow.set('agvLastTimestamp', lastTimestamp);\n\nreturn
msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 460,
        "y": 500,
        "wires": [
            [
                "6d5e4f3a2b1c0d91",
                "7c6b5a4d3e2f1098"
            ]
        ]
    },
    {
        "id": "6d5e4f3a2b1c0d91",
        "type": "mqtt out",
        "z": "fa46cf31be8d751e",
        "name": "Publish Metric/AGV",
        "topic": "Metric/AGV",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "f2d431c2d2264341",
        "x": 710,
        "y": 400,
        "wires": []
    },
    {
        "id": "7c6b5a4d3e2f1098",
        "type": "debug",
        "z": "fa46cf31be8d751e",
        "name": "Debug AGV payload",
        "active": true,
        "tosidebar": true,
        "console": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 740,
        "y": 580,
        "wires": []
    },
```

```
    {
        "id": "c11decb07a68083d",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "f2d431c2d2264341",
        "type": "mqtt-broker",
        "z": "fa46cf31be8d751e",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "336462685553088&341668146888752",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
```

```
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```

- Lathe 1

查看车床 1 数据连接 JSON

```
[
    {
        "id": "88fed08889390f28",
        "type": "tab",
        "label": "Metric/ProLathe-01",
        "disabled": false,
        "info": ""
    },
    {
        "id": "f2bae08de3b55bb0",
        "type": "mqtt in",
        "z": "88fed08889390f28",
        "name": "Subscribe ProLathe joints",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/porcess_prolathe_gs_260ms_1/Mo
tor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "6f8ade8021b85933",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 590,
        "y": 340,
        "wires": [
            [
                "ac84eab3cf5bb664",
                "8a8a5572465dac5b"
```

```
            ]
        ]
    },
    {
        "id": "ac84eab3cf5bb664",
        "type": "function",
        "z": "88fed08889390f28",
        "name": "Cache Data",
        "func": "const CONFIG = {\n    fields: [\"JointChanger\",
\"JointDoors\", \"JointFinger\", \"JointSpindle\", \"JointTailStock\",
\"Joint_X\", \"Joint_Y\", \"Joint_Z\"],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        time_stamp:
'time_stamp',\n        quality: 'quality'\n    }\n};\n\nlet cache =
flow.get('dataCache') || { instant: {}, metadata: {} };\n\nconst
topicParts = msg.topic.split('/');\nconst fieldName =
topicParts[topicParts.length - 1];\n\nlet payload = msg.payload;\nif
(typeof payload === 'string') {\n    try {\n        payload =
JSON.parse(payload);\n    } catch (e) {\n        return null;\n
}\n}\n\nlet value = payload[CONFIG.payloadFields.value];\nif (value ===
undefined) return null;\n\nconst metadata = {\n    timeStamp:
payload[CONFIG.payloadFields.timeStamp],\n    time_stamp:
payload[CONFIG.payloadFields.time_stamp],\n    quality:
payload[CONFIG.payloadFields.quality],\n    name:
payload[CONFIG.payloadFields.name],\n    initialValue:
payload[CONFIG.payloadFields.initialValue]\n};\n\nif
(CONFIG.fields.includes(fieldName)) {\n    cache.instant[fieldName] =
value;\n    cache.metadata[fieldName] = metadata;\n} else {\n    return
null;\n}\n\nflow.set('dataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 860,
        "y": 360,
        "wires": []
    },
    {
        "id": "ca4b6b2146df5093",
        "type": "inject",
        "z": "88fed08889390f28",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "vt": "str",
                "p": "topic"
```

```
            }
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 500,
        "y": 460,
        "wires": [
            [
                "e47cbdc7b7dae23f"
            ]
        ]
    },
    {
        "id": "e47cbdc7b7dae23f",
        "type": "function",
        "z": "88fed08889390f28",
        "name": "Aggregate & Publish (Flat)",
        "func": "const CONFIG = {\n    fieldMapping: {\n
\"JointChanger\": \"J_Changer\",\n    \"JointDoors\": \"J_Door\",\n
\"JointFinger\": \"J_Finger\",\n    \"JointSpindle\": \"J_Spindle\",\n
\"JointTailStock\": \"J_TailStock\",\n    \"Joint_X\": \"J_X\",\n
\"Joint_Y\": \"J_Y\",\n    \"Joint_Z\": \"J_Z\"\n},\n    missingStrategy:
'lastValue',\n    outputTopic: \"Metric/porcess_prolathe_gs_260ms_1\",\n
windowMs: 100\n};\n\nconst FIELDS =
Object.keys(CONFIG.fieldMapping);\n\nlet cache = flow.get('dataCache') ||
{ instant: {}, metadata: {} };\nlet lastValues = flow.get('lastValues')
|| {};\nlet lastMetadata = flow.get('lastMetadata') || {};\nlet output =
{};\n\noutput.timestamp = new Date().toISOString();\n\nfor (const field
of FIELDS) {\n    const shortName = CONFIG.fieldMapping[field];\n\n    if
(cache.instant[field] !== undefined) {\n        output[shortName +
'_Cur'] = Number(cache.instant[field]);\n        output[shortName +
'_Init'] = cache.metadata[field] && cache.metadata[field].initialValue
!== undefined\n            ? Number(cache.metadata[field].initialValue)\n
: 0;\n        lastValues[field] = cache.instant[field];\n
lastMetadata[field] = cache.metadata[field] || {};\n    } else {\n
let fallbackValue = CONFIG.missingStrategy === 'lastValue'\n            ?
(lastValues[field] !== undefined ? lastValues[field] : 0)\n            :
0;\n        output[shortName + '_Cur'] = Number(fallbackValue);\n
output[shortName + '_Init'] = lastMetadata[field] &&
lastMetadata[field].initialValue !== undefined\n            ?
Number(lastMetadata[field].initialValue)\n            : 0;\n
}\n}\n\nmsg.payload = output;\nmsg.topic =
CONFIG.outputTopic;\n\nflow.set('dataCache', { instant: {}, metadata: {}
});\nflow.set('lastValues', lastValues);\nflow.set('lastMetadata',
lastMetadata);\n\nreturn msg;",
        "outputs": 1,
```

```
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 740,
        "y": 460,
        "wires": [
            [
                "ad22bd2f8b228324"
            ]
        ]
    },
    {
        "id": "ad22bd2f8b228324",
        "type": "mqtt out",
        "z": "88fed08889390f28",
        "name": "Metric/Prolathe",
        "topic": "Metric/ProLathe-01",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "c14ef68b0e3b48ac",
        "x": 1060,
        "y": 360,
        "wires": []
    },
    {
        "id": "8a8a5572465dac5b",
        "type": "debug",
        "z": "88fed08889390f28",
        "name": "Debug Output",
        "active": true,
        "tosidebar": true,
        "console": false,
        "complete": "payload",
        "statusVal": "",
        "statusType": "auto",
        "x": 980,
        "y": 540,
        "wires": []
    },
    {
        "id": "6f8ade8021b85933",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
```

```
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "c14ef68b0e3b48ac",
        "type": "mqtt-broker",
        "z": "88fed08889390f28",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "3364626855530 88&337035452420768",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
```

```
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```


- Lathe 2

查看车床 2 数据连接 JSON


```
[
    {
        "id": "c974bfc5fbc79b0e",
        "type": "tab",
        "label": "Metric/ProLathe-02",
        "disabled": false,
        "info": ""
    },
    {
        "id": "2ff367f08cbe470f",
        "type": "mqtt in",
        "z": "c974bfc5fbc79b0e",
        "name": "Subscribe ProLathe joints",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/porcess_prolathe_gs_260ms_2/Mo
tor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "6f8ade8021b85933",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 590,
        "y": 340,
        "wires": [
            [
                "7c54f7852fb5430e"
            ]
        ]
    },
    {
        "id": "7c54f7852fb5430e",
        "type": "function",
```

```
        "z": "c974bfc5fbc79b0e",
        "name": "Cache Data",
        "func": "const CONFIG = {\n    fields: [\"JointChanger\",
\"JointDoors\", \"JointFinger\", \"JointSpindle\", \"JointTailStock\",
\"Joint_X\", \"Joint_Y\", \"Joint_Z\"],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        time_stamp:
'time_stamp',\n        quality: 'quality'\n    }\n};\n\nlet cache =
flow.get('dataCache') || { instant: {}, metadata: {} };\n\nconst
topicParts = msg.topic.split('/');\nconst fieldName =
topicParts[topicParts.length - 1];\n\nlet payload = msg.payload;\nif
(typeof payload === 'string') {\n    try {\n        payload =
JSON.parse(payload);\n    } catch (e) {\n        return null;\n
}\n}\n\nlet value = payload[CONFIG.payloadFields.value];\nif (value ===
undefined) return null;\n\nconst metadata = {\n    timeStamp:
payload[CONFIG.payloadFields.timeStamp],\n    time_stamp:
payload[CONFIG.payloadFields.time_stamp],\n    quality:
payload[CONFIG.payloadFields.quality],\n    name:
payload[CONFIG.payloadFields.name],\n    initialValue:
payload[CONFIG.payloadFields.initialValue]\n};\n\nif
(CONFIG.fields.includes(fieldName)) {\n    cache.instant[fieldName] =
value;\n    cache.metadata[fieldName] = metadata;\n} else {\n    return
null;\n}\n\nflow.set('dataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 860,
        "y": 360,
        "wires": []
    },
    {
        "id": "a69b0f994c614213",
        "type": "inject",
        "z": "c974bfc5fbc79b0e",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "vt": "str",
                "p": "topic"
            }
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
```

```
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 500,
        "y": 460,
        "wires": [
            [
                "a182dc40725e4801"
            ]
        ]
    },
    {
        "id": "a182dc40725e4801",
        "type": "function",
        "z": "c974bfc5fbc79b0e",
        "name": "Aggregate & Publish (Flat)",
        "func": "const CONFIG = {\n    fieldMapping: {\n
\"JointChanger\": \"J_Changer\",\n    \"JointDoors\": \"J_Door\",\n
\"JointFinger\": \"J_Finger\",\n    \"JointSpindle\": \"J_Spindle\",\n
\"JointTailStock\": \"J_TailStock\",\n    \"Joint_X\": \"J_X\",\n
\"Joint_Y\": \"J_Y\",\n    \"Joint_Z\": \"J_Z\"\n},\n    missingStrategy:
'lastValue',\n    outputTopic: \"Metric/porcess_prolathe_gs_260ms_2\",\n
windowMs: 100\n};\n\nconst FIELDS =
Object.keys(CONFIG.fieldMapping);\n\nlet cache = flow.get('dataCache') ||
{ instant: {}, metadata: {} };\nlet lastValues = flow.get('lastValues')
|| {};\nlet lastMetadata = flow.get('lastMetadata') || {};\nlet output =
{};\n\noutput.timestamp = new Date().toISOString();\n\nfor (const field
of FIELDS) {\n    const shortName = CONFIG.fieldMapping[field];\n\n    if
(cache.instant[field] !== undefined) {\n        output[shortName +
'_Cur'] = Number(cache.instant[field]);\n        output[shortName +
'_Init'] = cache.metadata[field] && cache.metadata[field].initialValue
!== undefined\n            ? Number(cache.metadata[field].initialValue)\n
: 0;\n        lastValues[field] = cache.instant[field];\n
lastMetadata[field] = cache.metadata[field] || {};\n    } else {\n
let fallbackValue = CONFIG.missingStrategy === 'lastValue'\n            ?
(lastValues[field] !== undefined ? lastValues[field] : 0)\n            :
0;\n        output[shortName + '_Cur'] = Number(fallbackValue);\n
output[shortName + '_Init'] = lastMetadata[field] &&
lastMetadata[field].initialValue !== undefined\n            ?
Number(lastMetadata[field].initialValue)\n            : 0;\n
}\n}\n\nmsg.payload = output;\n\nmsg.payload.Model_Name =
\"Turbo\";\nmsg.payload.Max_Carry_Weight_kg = 15.0;\n\nmsg.topic =
CONFIG.outputTopic;\n\nflow.set('dataCache', { instant: {}, metadata: {}
});\nflow.set('lastValues', lastValues);\nflow.set('lastMetadata',
lastMetadata);\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
```

```
        "x": 740,
        "y": 460,
        "wires": [
            [
                "b4e1817aa3744ab9",
                "84c88149d8d24eae"
            ]
        ]
    },
    {
        "id": "b4e1817aa3744ab9",
        "type": "mqtt out",
        "z": "c974bfc5fbc79b0e",
        "name": "Metric/Prolathe2",
        "topic": "Metric/ProLathe-02",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "6befe0f78bf64759",
        "x": 1060,
        "y": 360,
        "wires": []
    },
    {
        "id": "84c88149d8d24eae",
        "type": "debug",
        "z": "c974bfc5fbc79b0e",
        "name": "Debug Output",
        "active": true,
        "tosidebar": true,
        "console": false,
        "complete": "payload",
        "statusVal": "",
        "statusType": "auto",
        "x": 980,
        "y": 540,
        "wires": []
    },
    {
        "id": "6f8ade8021b85933",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
```

```
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "6befe0f78bf64759",
        "type": "mqtt-broker",
        "z": "c974bfc5fbc79b0e",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "336462685553088&338098843755632",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
```

```
        "userProps": "",
        "sessionExpiry": ""
    }
]
```


可视化车间场景

1. 在 **模型库** 中导入车间中的模型，将 UNS 数据映射到模型，并创建对应实例。
2. 进入 **场景**，从 **实例库** 将车间中的实例添加到场景中。
3. 移动模型到车间中的对应位置，并设置环境光照等细节。

参考资料

API 参考

Tier0 的 API 集成要点和机器可读的 OpenAPI 参考。

快速开始

- 使用 `https://<your-tier0-host>/openapi/v1` 作为基础 URL。
- 除 `/assets/files/public` 外，所有 OpenAPI 端点都要发送 `x-api-key`。标记为 `public` 的文件无需 API 密钥即可下载。
- 先调用 `POST /auth/whoami`，确认 API 密钥对应的身份和实际权限。

> ⓘ **OpenAPI 参考**
> 打开交互式 **API 参考**，查看请求架构、响应架构和可执行请求。

处理结果

即使返回 HTTP `200`，UNS 批量操作也可能部分失败。在认定操作成功前，请检查每一项结果。

Flow 中的自定义 HTTP In 路由

对于在 **Source Flow** 和 **Event Flow** 中开发的 API，输入和响应由已部署的 Flow 定义。

- **Source Flow** 使用 `/flow/source/`，默认公开。暴露敏感行为前，请添加入口或 Flow 级访问控制。
- **Event Flow** 使用 `/flow/event/`，需要具备 `flow.read` 权限的 `x-api-key`。

> ⓘ **Source Flow 或 Event Flow 中的自定义 API 示例**
> 例如 API URL 为 `/integrations/echo`，并且需要用 `HTTP In` 节点开发该 API：
>
> - `HTTP In` 节点中的路由为 `/integrations/echo`，其余部分由 Tier0 补全。
> - 外部调用 URL 为 `https://tier0.example.com/flow/source/integrations/echo` 或 `https://tier0.example.com/flow/event/integrations/echo`。

故障排除

- `401` / `403`：检查 API 密钥、权限范围和所有者权限。
- `404`：检查 HTTP 方法、路由前缀、节点 URL 和部署状态。
- `No response`：确保每条 Flow 路径都到达 `HTTP Response` 节点，且没有丢弃原始请求和响应属性。

参考资料

CLI 命令参考

CLI 和 Agent Skill 命令参考。

搜索命令、参数或示例...

全局参数 这些参数可以用于任何 `tier0` 命令。

`--json` 输出原始 JSON `--debug` 打印 HTTP 请求/响应详情

基础操作

`tier0 doctor`

诊断本地 Tier0 CLI 连接和认证

用法

```
tier0 doctor
```

说明

- 检查本地配置、OpenAPI 连通性、API key 是否存在以及 API key 身份。

示例

```
tier0 doctor
```

`tier0 config`

查看或更新配置

用法

```
tier0 config
```

参数

`--base-url`
设置平台 base URL

`--api-key`
设置 API key（可替代 tier0 login）

`--lang`
已弃用的兼容参数；CLI 输出仅支持英文
已弃用

示例

```
tier0 config
```

```
tier0 config --base-url https://tier0.dev
```

```
tier0 config --api-key sk-per-xxxxxx
```

`tier0 login`

通过 Device Flow 认证

用法

```
tier0 login
```

参数

`--no-wait`

打印授权 URL 后退出，不进行轮询

`--setup-code`

使用已有 setup code 轮询，而不是创建新的 setup code

`--base-url`

覆盖平台 base URL

说明

- 未使用 `--no-wait` 时，CLI 每 5 秒轮询一次，最长 10 分钟。
- 使用 `--no-wait` `--json` 时，CLI 会返回 `verification_url` 、 `setup_code` 和 `expires_in` 。

示例

```
tier0 login
```

```
tier0 login --no-wait --json
```

```
tier0 login --setup-code <setup_code>
```

`tier0 api`

直接调用 API endpoint

用法

```
tier0 api <endpoint>
```

参数

`--body`

请求体 JSON 字符串

`--body-file`

从文件读取请求体

`--method`

HTTP 方法（GET|POST|PUT|DELETE）
默认值：`POST`

`--dry-run`

预览请求但不执行

说明

- `--body` 和 `--body-file` 互斥。

示例

```
tier0 api /openapi/v1/uns/browse --body '{"path":"/"}'
```

```
tier0 api /openapi/v1/uns/read --body '{"topics":["demo"]}'
```

```
tier0 api /openapi/v1/uns/write --body-file body.json
```

`tier0 upgrade`

将 CLI 升级到最新版本

用法

```
tier0 upgrade
```

参数

`--dry-run`
检查更新但不安装

`-v, --version`
升级到指定版本（默认 latest）

示例

```
tier0 upgrade
```

```
tier0 upgrade --dry-run
```

`tier0 uninstall`

卸载 tier0 CLI

用法

```
tier0 uninstall
```

参数

`--purge`
同时删除配置文件（凭据）

`--remove-skills`
同时从检测到的 AI agents 中移除 Tier0 Skill

`--keep-skills`
保留 agent skills（已弃用；现在默认如此）
已弃用

说明

- 默认保留 Agent Skills 和配置。
- CLI 卸载时会删除本地内置 skills 目录。

示例

```
tier0 uninstall
```

```
tier0 uninstall --purge
```

```
tier0 uninstall --remove-skills
```

```
tier0 uninstall --purge --remove-skills
```

`tier0 version`

显示版本

用法

```
tier0 version
```

示例

```
tier0 version
```

技能

`tier0 skills install`

安装或修复 CLI 内置的 Skill

用法

```
tier0 skills install
```

参数

`--force`
用内置基线替换当前启用的 Skill

`--no-sync`
仅本地安装，不同步到检测到的 AI agents

示例

```
tier0 skills install
```

```
tier0 skills install --force
```

`tier0 skills list`

列出已安装的 skills

别名: ls

用法

```
tier0 skills list
```

示例

```
tier0 skills list
```

`tier0 skills update`

将 skills 升级到最新版本

别名: upgrade

用法

```
tier0 skills update
```

参数

`--dry-run`
检查更新但不安装

示例

```
tier0 skills update
```

```
tier0 skills update --dry-run
```

`tier0 skills version`

显示 skills 版本信息

别名: ver

用法

```
tier0 skills version
```

示例

```
tier0 skills version
```

`tier0 skills status`

显示当前启用和内置 Skill 状态

用法

```
tier0 skills status
```

示例

```
tier0 skills status
```

`tier0 skills sync`

将当前启用的 Skill 同步到检测到的 AI agents

用法

```
tier0 skills sync
```

示例

```
tier0 skills sync
```

认证

`tier0 auth whoami`

显示当前 API key 身份

用法

```
tier0 auth whoami
```

说明

- 显示用户、workspace、key 名称、key 类型、角色和权限。

示例

```
tier0 auth whoami
```

```
tier0 auth whoami --json
```

UNS

`tier0 uns browse`

浏览 UNS namespace 树

用法

```
tier0 uns browse
```

参数

`-p, --path`
要浏览的 UNS 树路径
默认值: `/`

`-d, --max-depth`
最大递归深度（0 = 不限制）
默认值: `1`

`--include-metadata`
包含节点元数据

`--include-leaf-value`
包含叶子节点值

示例

```
tier0 uns browse
```

```
tier0 uns browse --path /devices
```

```
tier0 uns browse --path / --max-depth 2
```

`tier0 uns search`

搜索 UNS topics

用法

```
tier0 uns search
```

参数

`-k, --keyword`
按名称关键字搜索

`--path-prefix`
按路径前缀过滤
默认值：`/`

`--topic-type`
按 topic type 过滤

`--page`
页码
默认值：`1`

`-l, --size`
每页数量（最大结果数）
默认值：`20`

`--include-metadata`
包含节点元数据

`--include-leaf-value`
包含叶子节点值

示例

```
tier0 uns search --keyword temp
```

```
tier0 uns search --path-prefix /devices --size 50
```

```
tier0 uns search --keyword temp --include-metadata
```

`tier0 uns read`

读取 UNS topics 的当前值

用法

```
tier0 uns read [topic...]
```

```
tier0 uns read --topic <topic>
```

参数

`-t, --topic`
要读取的 topic 名称（可重复；也接受位置参数）
可重复

`--include-metadata`
包含 topic 元数据（topicType、fields、description）

`--include-leaf-value`
包含叶子节点值

说明

- 必须通过位置参数或 `--topic` 提供至少一个 topic。

示例

```
tier0 uns read demo
```

```
tier0 uns read --topic demo
```

```

```

```
tier0 uns read temp humidity
```

```
tier0 uns read --topic sensor1 --include-metadata
```

`tier0 uns write`

向 UNS topic 写入值

用法

```
tier0 uns write
```

参数

`-t, --topic`
要写入的 topic 名称（必填）
必填

`-v, --value`
JSON 值字符串（与 --file 互斥）

`-f, --file`
从文件读取值（与 --value 互斥）

`--qos`
MQTT QoS 等级（0/1/2）
默认值：`0`

`--retain`
设置 MQTT retain flag

`--dry-run`
预览请求但不执行

说明

- `--value` 和 `--file` 互斥。
- 值必须是符合 topic fields 的有效 JSON object。

示例

```
tier0 uns write --topic demo --value '{"temp":25}'
```

```
tier0 uns write --topic demo --file payload.json
```

```
tier0 uns write --topic sensor1 --value '{"on":true}' --qos 1 --retain
```

`tier0 uns history`

查询 topics 历史数据

用法

```
tier0 uns history
```

参数

`-t, --topic`

topic 名称（可重复，必填）
必填 可重复

`--start`

开始时间：相对时间（-1h/-30m/-7d）、ISO 8601 或 'now'（必填）
必填

`--end`

结束时间：相对时间、ISO 8601 或 'now'（默认 now）
默认值：`now`

`--page`

页码
默认值：`1`

`-l, --size`

每页数量（最大数据点数）

默认值：`100`

`--interval`

聚合间隔（例如 1m、1h、1d）

`--fn`

聚合函数（avg/max/min/sum/count）

`--field`

聚合字段名

示例

```
tier0 uns history -t demo --start -1h
```

```
tier0 uns history -t demo --start -24h --end now --fn avg --interval 1h
```

```
tier0 uns history -t demo --start 2026-01-01T00:00:00Z --end 2026-01-02T00:00:00Z
```

`tier0 uns create`

创建 UNS namespace 节点

用法

```
tier0 uns create
```

参数

`-t, --topic`

topic 路径或叶子名称（不使用 --file 时必填）

`--parent`
父级路径前缀（可选，会与 --topic 组合）

`--type`
节点类型：'path'（文件夹）或 'topic'（数据点）

`-d, --display-name`
显示名称

`--description`
描述

`--alias`
别名

`-f, --file`
从 JSON 文件读取 namespace 定义（{"namespace":[]} 或裸数组）

`--topic-type`
已弃用：topic type 现在由路径推导（叶子节点前的 Metric/Action/State 文件夹）
已弃用

`--fields`
schema fields JSON 数组

`--fields-file`
从文件读取 schema fields JSON 数组

`--dry-run`
预览请求但不执行

说明

- 不使用 `--file` 时，必须提供 `--topic` 和 `--type`。
- `--fields` 和 `--fields-file` 互斥。
- 对于 topic 节点，叶子节点前一级路径必须是 Metric、Action 或 State。

示例

```
tier0 uns create --topic Plant/Line1/Metric/Temperature --type topic
```

```
```

```
tier0 uns create --parent Factory1/Line1/Station1 --topic
Metric/ProductionCount --type topic
```

```
tier0 uns create --topic Plant/Line1 --type path --display-name 'Line 1'
```

```
tier0 uns create --file namespace.json
```

`tier0 uns update`

更新 UNS topic 元数据

用法

```
tier0 uns update
```

参数

`-p, --path`
要更新的 topic 路径（必填）
必填

`-n, --name`
新名称

`--alias`
新别名

`--description`
新描述

`--clear-description`
清空描述，不依赖空 shell 参数

`-d, --display-name`

新的显示名称

`--extend-properties`

扩展属性 JSON 对象

`--fields`

schema fields JSON 数组

`--fields-file`

从文件读取 schema fields JSON 数组

`--update-mask`

要更新的字段（可重复，例如 name、description、fields）
可重复

`--dry-run`

预览请求但不执行

说明

- `--description` 和 `--clear-description` 互斥。
- `--fields` 和 `--fields-file` 互斥。
- 至少需要提供一个要更新的字段。

示例

```
tier0 uns update --path Plant/Line1/Metric/Temperature --display-name
'Line 1 Temp'
```

```
tier0 uns update --path Plant/Line1 --description 'Production line 1' --
update-mask description
```

```
tier0 uns update --path Plant/Line1/Metric/Temperature --fields
'[{"name":"temp","type":"float","unit":"C"}]' --update-mask fields
```

`tier0 uns delete`

删除 UNS 节点

用法

```
tier0 uns delete
```

参数

`-p, --path`
要删除的节点路径（可重复，必填）
必填 可重复

`--hard`
硬删除（不可恢复）

`-y, --yes`
确认高风险操作（必填）
必填

`--dry-run`
预览请求但不执行

说明

- 默认执行软删除，可以恢复。
- `--hard` 不可恢复。
- 高风险操作需要 `--yes`，除非使用 `--dry-run`。

示例

```
tier0 uns delete --path factory/line1/sensor/temp --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --path
factory/line1/sensor/humi --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --hard --yes
```

`tier0 uns restore`

从历史记录恢复 topic

用法

```
tier0 uns restore
```

参数

`-p, --path`
要恢复的 topic 路径（必填）
必填

`-y, --yes`
确认高风险操作（必填）
必填

`--dry-run`
预览请求但不执行

说明

- 高风险操作需要 `--yes`，除非使用 `--dry-run`。

示例

```
tier0 uns restore --path Plant/Line1/Metric/Temperature --yes
```

Node-RED 流程

`tier0 flow list`

列出 flows

别名: ls

用法

```
tier0 flow list
```

参数

`-k, --keyword`
按名称关键字过滤

`-t, --type`
按类型过滤（SourceFlow/EventFlow）

`--source`
只显示 SourceFlow

`--event`
只显示 EventFlow

说明

- `--source` 和 `--event` 互斥。
- `--type` 不能与 `--source` 或 `--event` 同时使用。

示例

```
tier0 flow list
```

```
tier0 flow list --source --json
```

`tier0 flow get`

获取 flow 详情

用法

```
tier0 flow get --id <id>
```

```
tier0 flow get <id>
```

参数

`--id`

Flow ID

默认值：`0`

说明

- 必须通过 `--id` 或位置参数提供正数 Flow ID。

示例

```
tier0 flow get --id 1 --json
```

```
tier0 flow get 1 --json
```

`tier0 flow create`

创建新 flow

用法

```
tier0 flow create
```

参数

`-n, --name`

Flow 名称（必填）

必填

`-t, --type`

Flow 类型：SourceFlow | EventFlow（必填）

必填

`--source`

设置类型为 SourceFlow

`--event`

设置类型为 EventFlow

`--desc`

描述

`--template`

初始模板 JSON 字符串

`--template-file`

从文件读取初始模板

`--dry-run`

预览请求但不执行

说明

- 必须且只能使用 `--type`、`--source` 或 `--event` 之一选择 Flow 类型。
- `--template` 和 `--template-file` 互斥。

示例

```
tier0 flow create --name "modbus-collector" --source --desc "Modbus TCP
collector"
```

```
tier0 flow create --name "alert-handler" --event --desc "Temperature
alarm processor"
```

`tier0 flow update`

更新 flow 元数据

用法

```
tier0 flow update --id <id>
```

```
tier0 flow update <id>
```

参数

`--id`
Flow ID（必填）
默认值：`0`

`-n, --name`
新名称

`--desc`
新描述

`--template`
新模板 JSON 字符串

`--template-file`
从文件读取新模板

`--favorite`
标记为收藏

`--unfavorite`
取消收藏

`--dry-run`
预览请求但不执行

说明

- 必须通过 `--id` 或位置参数提供正数 Flow ID。
- `--template` 和 `--template-file` 互斥。

- `--favorite` 和 `--unfavorite` 互斥。
- 至少需要提供一个要更新的字段。

示例

```
tier0 flow update --id 1 --name "line1-collector"
```

```
tier0 flow update --id 1 --favorite
```

`tier0 flow data`

获取 Node-RED canvas JSON

用法

```
tier0 flow data --id <id>
```

```
tier0 flow data <id>
```

参数

`--id`
Flow ID
默认值：`0`

`-o, --out`
保存输出到文件

说明

- 必须通过 `--id` 或位置参数提供正数 Flow ID。

示例

```
tier0 flow data --id 1 --out flows.json
```

`tier0 flow deploy`

部署 Node-RED canvas JSON

用法

```
tier0 flow deploy --id <id>
```

```
tier0 flow deploy <id>
```

参数

`--id`

Flow ID（必填）
默认值：`0`

`--flows-json`

Node-RED 画布 JSON 字符串

`-f, --flows-file`

从文件读取 Node-RED 画布 JSON（推荐）

`-y, --yes`

确认高风险操作（必填）
必填

`--dry-run`

预览请求但不执行

说明

- 必须通过 `--id` 或位置参数提供正数 Flow ID。
- `--flows-json` 和 `--flows-file` 互斥。
- 部署会替换该 Flow 中所有现有 Node-RED 节点。

- 高风险操作需要 `--yes`，除非使用 `--dry-run`。

示例

```
tier0 flow deploy --id 1 -f flows.json --dry-run --json
```

```
tier0 flow deploy --id 1 -f flows.json --yes
```

`tier0 flow delete`

删除 flow

别名: del, rm

用法

```
tier0 flow delete --id <id>
```

```
tier0 flow delete <id>[,<id>...]
```

参数

`--id`
要删除的 Flow ID（可重复）
可重复

`-y, --yes`
确认高风险操作（必填）
必填

`--dry-run`
预览请求但不执行

说明

- 位置参数可以包含逗号分隔的 Flow ID。
- 删除会停止 Node-RED 容器，且无法撤销。
- 高风险操作需要 `--yes`，除非使用 `--dry-run`。

示例

```
tier0 flow delete --id 1 --yes
```

`tier0 flow nodes`

列出可用的 Node-RED 节点类型

用法

```
tier0 flow nodes [source|event]
```

```
tier0 flow nodes --type <SourceFlow|EventFlow>
```

```
tier0 flow nodes --source
```

```
tier0 flow nodes --event
```

参数

`-t, --type`
Flow 类型（SourceFlow/EventFlow）

`--source`
显示 SourceFlow 节点

`--event`

显示 EventFlow 节点

说明

- 最多接受一个位置参数形式的 Flow 类型。
- `--source` 和 `--event` 互斥。
- 不要将位置参数/类型输入与 `--source` 或 `--event` 同时使用。

示例

```
tier0 flow nodes source --json
```

```
tier0 flow nodes --source --json
```

```
tier0 flow nodes --event --json
```

文件资源

`tier0 assets upload`

上传文件到 Tier0 object storage

用法

```
tier0 assets upload <local-file>
```

参数

`--business`

业务场景

默认值：`attachment`

`--use-by`

使用范围：user|workspace|platform
默认值：`workspace`

`--visibility`

可见性：public|private
默认值：`private`

`--app-instance-id`

AI 应用实例 ID

`--session-id`

AI 会话 ID

`--multipart-size`

分片大小（例如 10MB，最小 5MB）
默认值：`CLI-defined default`

`--concurrency`

分片上传并发数（并行分片数量）
默认值：`4`

`--resume`

恢复中断的分片上传

`--abort`

中止中断的分片上传并清理其状态

说明

- 必须提供且只能提供一个本地文件参数。
- 超过 100MB 的文件会自动使用分片上传。
- CLI 不强制限制最大文件大小；服务器会按套餐和配额限制处理。

示例

```
tier0 assets upload ./report.csv
```

```
tier0 assets upload ./report.csv --visibility public --business
attachment
```

```
tier0 assets upload ./report.csv --use-by workspace --visibility private
```

```
tier0 assets upload ./large-backup.tar.gz
```

```
tier0 assets upload ./large-backup.tar.gz --resume
```

```
tier0 assets upload ./large-backup.tar.gz --abort
```

```
tier0 assets upload ./large-backup.tar.gz --multipart-size 20MB --
concurrency 8
```

`tier0 assets download`

从 Tier0 object storage 下载文件

用法

```
tier0 assets download
```

参数

`--file-path`
上传返回的文件路径
必填

`-o, --output`

输出文件路径（默认 stdout）

`--content-disposition`

自定义 Content-Disposition

示例

```
tier0 assets download --file-path workspace/.../report.csv -o
./report.csv
```

```
tier0 assets download --file-path workspace/.../report.csv
```

`tier0 assets url`

获取 Tier0 object storage 文件访问 URL

用法

```
tier0 assets url
```

参数

`--file-path`

上传返回的文件路径

必填

`--expired-sec`

预签名 URL 过期秒数

默认值：`3600`

`--content-disposition`

自定义 Content-Disposition

示例

```
tier0 assets url --file-path workspace/.../report.csv
```

```
tier0 assets url --file-path workspace/.../report.csv --expired-sec 300
```

`tier0 assets delete`

删除 Tier0 object storage 文件

用法

```
tier0 assets delete
```

参数

`--file-path`
上传返回的文件路径
必填

`-y, --yes`
确认高风险操作（必填）
必填

`--dry-run`
预览请求但不执行

说明

- 删除无法撤销。
- 高风险操作需要 `--yes`，除非使用 `--dry-run`。

示例

```
tier0 assets delete --file-path workspace/.../report.csv --dry-run --
json
```

```
tier0 assets delete --file-path workspace/.../report.csv --yes
```

参考资料

内置 Builder Skills

Tier0 Builder 内置 AI Skills 的参考。

Tier0 App Builder 当前包含 7 个 active 的内置 AI Skills。这些 Skills 帮助 agent 将需求转化为工业应用、对齐 Tier0 平台脚手架、修复 UI 问题，并保持预览环境稳定。

`app-srs`

含义：新建工业企业应用的需求分析入口。适用于 WMS、MES、ERP、CRM、QMS、EAM、审批、库存、供应商、客户、排程等企业工作流。

示例：

```
使用 $app-srs 为汽车零部件工厂设计一个质量异常管理系统。
```

也可以用自然语言触发：

```
生成一个仓库管理应用，包含收货、上架、拣货、发货和库存盘点。
```

它会做什么：

1. 分析业务角色、工作流、状态和异常。
2. 检查需求是否足够清晰。
3. 生成完整 SRS。
4. 默认保存到 `specs/spec.md`。
5. 除非用户只要求写需求，否则默认把结果交给 `app-builder`。

`app-builder`

含义：实际构建应用的核心 Skill。

示例：

```
使用 $app-builder 根据 specs/spec.md 构建第一版应用。
```

内部工作类型：agent 会根据 prompt 和当前项目状态决定使用哪种工作类型。

- **First Release**：根据 SRS 构建首版应用。
- **Delta Feature**：为已有应用增加较大的模块或工作流。
- **Light Edit**：处理小字段、文案、样式或校验调整，不运行完整构建流程。

它会检查什么：

- 可见入口。
- 可执行的用户路径。
- 数据或状态变化。
- 角色权限。
- 构建和预览验证。

> 提示
> 如果首次构建时没有 `specs/spec.md`，agent 会先用 `app-srs` 创建一个。

`align-platform-app`

含义：用于将已有应用对齐到 Tier0 平台脚手架的迁移 Skill。

```
使用 $align-platform-app 将这个旧 Vite React 项目迁移到 Tier0 平台脚手架。
```

适用来源：

- Vite、CRA、Next.js、React Router、静态 HTML 和全栈 Node/React 应用。
- 旧版 Tier0 应用。
- 依赖 LocalStorage、IndexedDB、Zustand persist、内存 mock 或模拟数据的应用。

它会做什么：

- 对齐 Tier0 TanStack Start 脚手架。
- 将持久业务数据迁移到 Drizzle/PostgreSQL。
- 对齐路由、API、service 和数据库设置。
- 移除应用自己实现的登录、退出和角色切换流程。
- 接入 Gateway 管理的身份和角色。
- 保留 `/api/health`、`/api/manifest` 和 `artifact.toml` 等平台契约。
- 验证 preview 和生产启动。

`migrate-node-red-app`

含义：用于将 Node-RED 应用对齐到 Tier0 平台脚手架的迁移 Skill，通常与 `align-platform-app` 一起使用。

示例：

```
使用 $align-platform-app 和 $migrate-node-red-app 将这个 Node-RED 后端迁移成
Tier0 TanStack Start 应用。
```

它会做什么：

- 盘点 flow、subflow、function node、change node 和 switch node。
- 分析 HTTP、MQTT、数据库、定时任务和外部 API 调用。
- 建立迁移映射表。
- 区分 CRUD、工作流、调度、集成和临时模拟行为。
- 将业务逻辑迁移到 services。
- 将接口迁移到 API routes。
- 将状态迁移到 PostgreSQL。
- 将长任务和集成迁移到 workers、queues 或外部服务。
- 分别验证浏览器、CRUD、工作流和集成切换。

> ⚠ 警告
> 它不会单独处理普通 preview 或打包问题。

`ui-repair`

含义：用于已有应用的修复 Skill。它处理 UI 质量问题，不用于首次生成应用。

示例：

```
使用 $ui-repair 修复订单编辑弹窗在小屏幕上的溢出和表单错位。
```

适合的问题：

- 表单或必填标记错位。
- Dialog 或 drawer 溢出。
- 意外的横向滚动。
- Select 菜单或下拉菜单被裁剪。
- 表格或图表布局异常。
- 页面层级弱、视觉过平。
- UI 语言混杂。
- PDA、kiosk、看板和工业界面布局问题。

> ⚠ 警告
> 该 Skill 应定位最小责任组件并修复受影响的界面，不应重建整个应用。复杂响应式问题请使用 `responsive-craft`。

`responsive-craft`

含义：用于响应式布局审计、新建响应式页面和多断点预览的 Skill。

调用模式：

- **audit**：审计并修复已有响应式布局。
- **build**：从零构建响应式页面。
- **preview**：启动多断点实时预览。

示例：

```
使用 $responsive-craft audit 修复这个 dashboard 的移动端布局。
```

```
使用 $responsive-craft build 创建一个适配手机、平板和桌面的库存工作台。
```

```
使用 $responsive-craft preview 展示 375、768、1024 和 1440 px 断点。
```

执行方式：

- **Adaptive**：快速推进，并在出现设计分叉时确认。
- **Guided**：先为每个组件编写响应式行为规格，再实现。

`preview-runtime-stability`

含义：用于应用开发过程中 preview 失败的运行时排障 Skill。它不构建业务功能。

示例：

```
使用 $preview-runtime-stability 诊断 "Preview loading timed out"。
```

适合的问题：

- `node_modules` 缺失或安装不完整。
- npm `ERESOLVE`。
- Vite 或 TanStack Start 启动失败。
- 端口 `5173` 冲突。
- `/api/health` 失败。
- SDK SSR 模块错误。
- `artifact.toml` preview 端口不一致。

- HMR 期间频繁整页刷新。
- 旧 Node 或 Vite 进程未退出。

标准流程：

1. 检查依赖安装。
2. 检查端口 `5173`。
3. 探测健康接口。
4. preview 健康时复用。
5. preview 不健康时清理旧 preview。
6. 重启并验证端口、健康接口和 HMR 行为。

Skill 选择指南

- 需求不清晰或需要稳定规格文档时，先使用 `app-srs`。
- 实现范围已经清楚时，使用 `app-builder`。
- 迁移工作使用 `align-platform-app` 和 `migrate-node-red-app`。
- 应用已存在且问题属于视觉、响应式或运行时问题时，使用 `ui-repair`、`responsive-craft` 或 `preview-runtime-stability`。

参考资料

用户管理

Tier0 通过成员资格、License 和自定义用户账号，由购买方管理平台使用资格与访问权限。

Tier0 用户权限管理和访问控制依据不同版本通过不同方式实现。

- 云服务版通过 Workspace 成员邀请管理使用资格；
- 企业版由管理员分配用户账号和角色，控制用户可访问的功能模块。

访问控制

标签页：云服务版

- Workspace 创建者或现有成员通过电话或邮箱向新成员发起邀请。新成员收到邀请码或邀请链接后，注册账号并加入 Workspace。
- 发起邀请时配置新成员的坐席类型（Operator 或 Builder），决定新成员可访问的功能范围。
    - **Builder**：可访问所有功能模块。
    - **Operator**：仅可访问应用中心。
- 账号注册须经过双重验证。

> ⚠ 警告
> 获得 Workspace 成员资格后，用户仍需注册账号才能使用平台。

> ⓘ 注意
> 注册账号后，用户也可以自行申请新建 Workspace。

标签页：企业版

- 管理员使用默认账号首次登录后，修改密码，并在设置中添加用户账号、配置角色。
- 企业通过分配账号决定谁可以使用平台，通过角色决定用户可访问的功能范围：
    - **Admin**：可访问所有功能模块。

- **Operator**：仅可访问应用中心。

> ⓘ 注意
> 用户使用管理员创建的账号登录平台，在所配置角色的权限范围内使用功能。

账号管理

标签页：云服务版

操作	谁	描述
更改席位	创建者 / Builder	• 创建者可更改所有成员的席位。 • Builder 仅可更改 Operator 的席位。
删除 Workspace	创建者	删除 Workspace 后，所有数据立即清除且无法恢复。
转移 Workspace	创建者	—
移除成员	创建者 / Builder	• 创建者可移除所有成员。 • Builder 仅可移除 Operator。
删除账号	所有成员	• 创建者必须先删除或转移 Workspace。 • 成员账号会立刻退出所有 Workspace。 • 账号数据立即清除且无法恢复。

标签页：企业版

- Admin 可以重置密码、编辑用户角色、禁用账号、删除账号。
- 初始默认账号角色不可编辑，且不能被禁用或删除。
- 账号删除后，所有数据立即清除且无法恢复。

参考资料

Tier0 云服务版网络与数据安全说明

Tier0 云服务版生产 SaaS 服务的网络边界、租户隔离、数据保护与安全运维措施。

适用范围： Tier0 云服务版生产 SaaS 服务，部署于 AWS 新加坡区域。
文档日期： 2026 年 10 月 22 日

Tier0 云服务版通过网络访问控制、租户隔离、身份权限管理、数据加密及备份恢复等措施，降低网络攻击和未经授权访问的风险。本说明介绍平台的安全防护措施，以及限制安全事件向其他租户、平台内部服务和客户网络扩散的控制边界。

> ⓘ **Edge 版本**
> 本说明仅适用于 Tier0 云服务版。Edge 的安装说明请参阅安装。

1 网络边界与访问控制

Tier0 云服务版面向公网的 Web 与 API 访问采用 TLS 加密，并通过统一的边缘入口和应用网关接入平台。核心服务入口配置源站访问校验，对不符合校验要求的直接访问请求予以拒绝，降低绕过边缘入口访问源站的风险。

生产环境与开发、预发布环境在云账号、网络、集群和数据库等层面实施隔离，并分别管理访问凭据。生产管理访问通过经过身份认证的专用网络通道进行。

2 租户隔离与攻击扩散控制

2.1 工作负载隔离

租户工作负载运行在独立的 Kubernetes 命名空间中，并通过专用节点池和调度约束，与平台服务及管理组件分离。该机制用于限制租户应用与平台组件之间的直接接触范围，降低单个应用发生安全事件时对其他服务的影响。

2.2 网络隔离

租户命名空间的入站和出站通信采用默认拒绝、显式放行的网络策略。策略由平台自动配置并同步，由集群网络组件执行。

入站通信仅向同租户工作负载及承担必要业务功能的平台服务开放。出站通信按规则开放集群域名解析、同租户服务、指定平台网关、消息服务及所分配的数据库端点。公网访问受网络策略约

束；除明确放行的业务依赖外，策略限制对私有网络地址的访问，并阻断对云实例元数据地址的直接访问。

上述措施用于限制未经授权的跨租户连接，以及从租户工作负载向平台内部网络的横向移动。单个租户应用发生安全事件，不会因此自动获得访问其他租户或平台管理组件的权限。已获授权的业务连接仍受相应网络规则和身份权限约束。

2.3 数据与身份隔离

租户应用数据在数据库层面实施逻辑隔离。分配独立数据库单元的场景，同时通过网络访问规则限制连接来源。

平台按服务职责配置访问权限，将工作负载的资源访问范围限定于所需功能。网络隔离与身份授权共同限制受影响工作负载能够访问的资源范围。

2.4 客户网络接入边界

本节所述租户隔离措施适用于 Tier0 云服务版平台内部。客户通过连接器、网关或其他集成方式接入内部系统时，客户网络的访问边界还取决于实际接入方式、路由、防火墙规则及所授予的凭据权限。

客户接入时应仅开放业务必需的目标地址、端口和账户权限，并对接入组件与其他内部系统实施网络分区。平台内部隔离不能替代客户侧的网络访问控制；双方应结合具体接入方案确定并核验访问范围，以限制经由已授权连接向客户其他系统扩散的风险。

3 数据保护

平台对数据传输、存储和访问实施相应保护：

- 公网 Web 与 API 通信，以及对托管数据库和缓存的访问，采用 TLS 加密。
- 托管数据库、缓存及对象存储启用静态加密。
- 对象存储配置公共访问阻断，并通过加密要求、HTTPS 访问要求和资源权限限制数据访问。
- 租户数据访问同时受数据库隔离和身份权限控制约束。

4 备份与恢复

Tier0 云服务版对生产数据库及纳入备份范围的服务级数据库执行备份，备份数据存储于专用、访问受限的对象存储中，并按相应保留策略管理。

备份任务使用独立身份，权限限定于指定存储位置所需的读写操作，不授予备份对象删除权限。备份到期清理由存储生命周期规则执行，以减少备份任务凭据泄露后被用于删除恢复数据的风险。

备份完成后执行完整性校验，并通过隔离环境中的恢复验证检查备份可用性。应用配置纳入版本控制，构建产物保存在私有镜像仓库，为服务恢复和版本回退提供支持。

5 管理访问与生产变更

生产管理访问使用具名身份，并按最小权限原则分配授权。涉及敏感操作的控制台身份实施强制多因素认证。

日常巡检使用受限的只读身份，可查看工作负载状态、日志和事件，不能读取敏感凭据、修改资源或执行远程命令。生产变更通过单独授权的身份和流程执行。

基础设施配置、部署清单和运维脚本纳入版本控制。生产变更需明确目标、影响范围、验证步骤和回滚方案，并记录执行身份、配置差异及验证结果，以支持追溯和恢复。

6 监控与安全事件处置

平台集中采集生产环境的运行指标、日志和健康状态，并将异常告警发送至运维响应渠道。监控范围包括服务可用性、错误率、数据库健康、证书有效期、备份任务状态及集群运行状态。

发生安全事件时，处置工作以控制影响范围为优先，并保留相关日志、配置和事件时间线。在完成必要的隔离与排查后恢复服务，随后开展事件复盘并跟踪整改。事件处置期间的生产变更按既定流程授权和留痕。

7 安全联系

有关安全评估、客户尽调或疑似漏洞，请联系 Tier0 客户经理或安全团队。涉及非公开的架构与控制细节，可在适当的保密安排下提供。首次联系时，请勿发送访问凭据、私钥或客户敏感数据。

参考资料

ISA-95 设备层级

使用 ISA-95 建模物理资产和控制系统时的参考层级。

ISA-95 设备层级描述了组织内物理资产和控制系统的结构方式。

它分为以下层级：

- **Enterprise**：最高层级，代表整个组织。它包含整体业务流程和决策实体，通常由 ERP 系统支持。
- **Site**：代表特定地理位置或工厂。每个 site 可以有自己的管理系统和本地生产控制。
- **Area**：site 内的运营分区，通常是包装、装配或仓储等功能区域。
- **Process Cell**：area 内的基本制造单元。它包含生产特定产品或执行特定功能所需的全部设备。
- **Unit**：process cell 中的独立部分，负责混合、加热或装配等特定功能。
- **Equipment Module**：unit 内执行特定任务的模块化设备。例如，在混合单元中，该模块可以是加热元件。
- **Control Module**：最低层级，代表传感器、泵、执行器或 PID 控制回路等单个设备或控制器，负责直接控制物理过程。

参考资料

标准部署方案

Tier0 Enterprise 四虚拟机标准部署参考方案。

方案目标

简化版不使用 Docker Swarm，只部署单台 `enterprise` 应用节点，总计 `4` 台虚拟机：

- `1` 台 `enterprise` 节点承载 `tier0-enterprise + EMQX`
- `2` 台数据库节点组成 `PostgreSQL + TimescaleDB` 主从集群
- `1` 台 `Witness` 节点参与数据库监督、仲裁和故障切换

简化版目标是降低部署复杂度。代价是应用层和 EMQX 均为单点，只有数据库层保留 `DB VIP + Witness` 高可用。

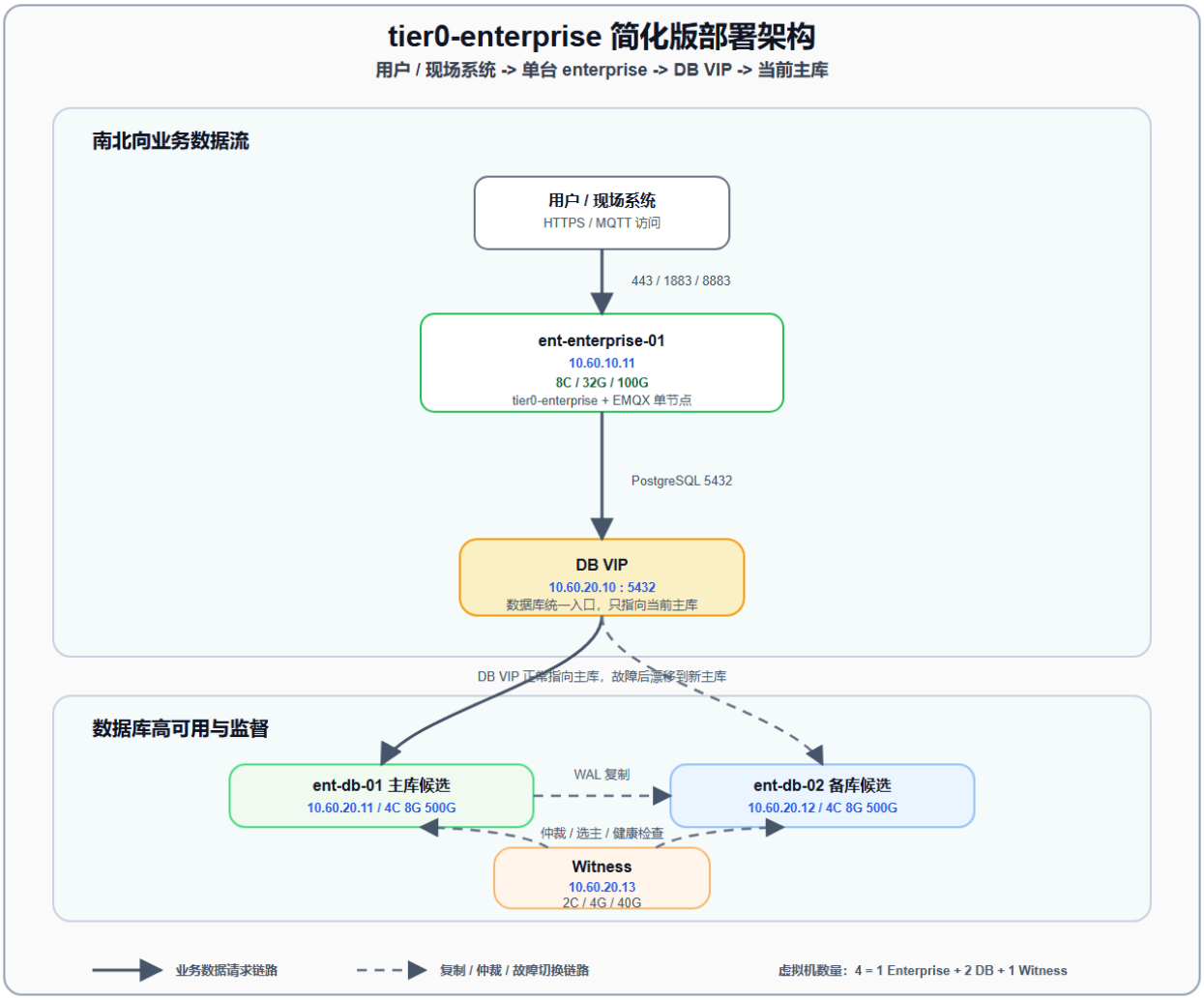
规划 IP

以下 IP 为规划占位，实施前必须替换为现场真实网段。

类型	名称	规划 IP	说明
应用节点	`ent-enterprise-01`	`10.60.10.11`	单台 enterprise，承载 `tier0-enterprise + EMQX`
逻辑入口	`DB VIP`	`10.60.20.10`	数据库统一入口，只指向当前主库
数据库节点	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB 主库候选
数据库节点	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB 备库候选

类型	名称	规划 IP	说明
仲裁节点	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` / 仲裁监督节点

总体拓扑



核心链路：

- 业务访问路径：用户 / 现场系统 -> `ent-enterprise-01` -> `DB VIP` -> 当前主库
- `tier0-enterprise` 和 `EMQX` 均部署在 `ent-enterprise-01`
- 数据库仍通过 `DB VIP` 访问，不直连 `ent-db-01/02`
- `Witness` 只参与数据库仲裁、选主和健康检查，不进入业务访问链路

数据库监督与故障切换

阶段	链路	说明
正常访问	`ent-enterprise-01 -> DB VIP -> 当前主库`	应用始终只连接 `10.60.20.10:5432`
正常复制	`当前主库 -> 当前备库`	PostgreSQL WAL 流复制
正常监督	`DB-01/DB-02 -> Witness`	repmgrd 上报状态，Witness 参与仲裁
主库故障	`备库 -> Witness`	备库请求仲裁，避免脑裂
备库提升	`备库 promote`	Witness 确认后，备库提升为新主库
VIP 漂移	`DB VIP -> 新主库`	`10.60.20.10` 绑定到新主库
应用恢复	`ent-enterprise-01 -> DB VIP -> 新主库`	应用连接池重连后恢复读写

节点配置清单

节点	角色	vCPU	内存	存储	系统盘建议	数据盘建议	主要组件
`ent-enterprise-01`	单节点 enterprise	`8`	`32G`	`100G`	单块系统盘 `/` 100G	不单独拆分	tier0-enterprise、EMQX、Nginx/HAProxy 可选
`ent-db-01`	PostgreSQL/TSDB 主库候选	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G、`/pgwal`	PostgreSQL、TimescaleDB

节点	角色	vCPU	内存	存储	系统盘建议	数据盘建议	主要组件
						40G、`/backup-staging` 20G	、repmgr、keepalived
`ent-db-02`	PostgreSQL/TSDB 备库候选	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived
`ent-db-witness-01`	Witness/仲裁监督	`2`	`4G`	`40G`	`/` 40G	不单独规划	repmgr witness、监控 agent、审计日志

配置汇总清单

类别	数量	单台规格	小计 vCPU	小计内存	小计存储
enterprise 应用节点	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB 节点	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`

类别	数量	单台规格	小计 vCPU	小计内存	小计存储
Witness 节点	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
合计	`4`	-	`18C`	`52G`	`1140G`

换算：

- 总虚拟机：`4` 台
- 总 vCPU：`18`
- 总内存：`52G`
- 总存储：`1.14T`

连接方式清单

运维访问

目标	方式	端口	说明
所有虚拟机	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	统一免密实施后使用；未实施前以现场账号为准
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	业务入口
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	只允许 enterprise 节点和运维白名单访问

应用与 EMQX 端口

源	目标	端口	用途
用户 / 现场系统	`ent-enterprise-01`	`80/TCP`	HTTP 入口或跳转 HTTPS，可选

源	目标	端口	用途
用户 / 现场系统	`ent-enterprise-01`	`443/TCP`	HTTPS 业务入口
设备 / 现场系统	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT 明文，可选
设备 / 现场系统	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS，建议
设备 / 现场系统	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket，可选
设备 / 现场系统	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS，可选
运维网段	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard，仅允许运维白名单
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL 业务访问

数据库与监督通信

源	目标	端口	用途
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL 流复制
`ent-db-02`	`ent-db-01`	`5432/TCP`	故障恢复后反向复制
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr metadata / witness 通信
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP 漂移，需允许协议 `112`

服务部署建议

enterprise 应用节点

- 不部署 Docker Swarm
- `tier0-enterprise` 和 `EMQX` 部署在同一台 `ent-enterprise-01`
- `ent-enterprise-01` 是应用层单点，故障时需要人工恢复或从备份重建
- 节点只有 `100G` 单系统盘，不适合承载长期业务数据
- `tier0-enterprise` 业务数据必须进入 PostgreSQL/TimescaleDB
- 如有上传、导入导出、报表等文件，可保存在本机并纳入备份；如果后续要扩展为标准版，应迁移到 NFS/NAS 或对象存储

EMQX 单节点

- EMQX 在简化版中为单节点有状态服务
- 数据目录固定在 `ent-enterprise-01` 本地盘，例如 `/data/emqx`
- 配置、证书、插件、规则引擎配置需要纳入备份
- 不提供消息层高可用；如不能接受 EMQX 单点，应使用标准版三节点 EMQX 集群模式

PostgreSQL + TimescaleDB

- PostgreSQL： `16` 或现场认证版本
- TimescaleDB：选择与 PostgreSQL 主版本匹配的稳定版本
- 主从复制：物理流复制
- 故障切换： `repmgrd + witness`
- 数据库入口： `DB VIP`
- 连接池：可在 `ent-enterprise-01` 部署 PgBouncer，避免数据库连接数过高

Witness

- Witness 只放 `repmgr witness` 元数据和监督逻辑
- Witness 不承载业务 SQL
- Witness 故障不影响当前主库继续服务，但会降低自动切换可信度

端口与防火墙基线

端口/协议	放行范围	用途
`22/TCP`	运维网段到 4 台 VM	SSH
`80/TCP`	用户网段到 `ent-enterprise-01`	HTTP 跳转或内网明文入口
`443/TCP`	用户网段到 `ent-enterprise-01`	HTTPS 业务入口
`1883/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX MQTT 明文，可选
`8883/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX MQTT TLS，建议
`8083/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX WebSocket，可选
`8084/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX WebSocket TLS，可选
`18083/TCP`	运维网段到 `ent-enterprise-01`	EMQX Dashboard，仅白名单
`5432/TCP`	enterprise 到 `DB VIP`；DB/Witness 内部互通	PostgreSQL / repmgr
`112/VRRP`	DB 节点间	DB VIP 漂移
`9100/TCP`	监控系统到 4 台 VM	node exporter，可选

高可用能力与故障边界

故障场景	预期行为	业务影响
`ent-enterprise-01` 故障	应用和 EMQX 均中断，需要人工恢复或从备份重建	应用层整体中断

故障场景	预期行为	业务影响
当前数据库主库故障	备库经 Witness 仲裁后提升，DB VIP 漂移	秒级到分钟级写入中断，应用需重连
数据库备库故障	主库继续服务，复制中断告警	无直接业务中断，但失去数据库热备
Witness 故障	当前主库继续服务，自动切换能力下降	无直接业务中断，需优先恢复
DB VIP 漂移失败	数据库实际已切主但入口不可达	业务中断，需手动绑定 VIP

上线前检查清单

检查项	通过标准
4 台 VM 主机名固定	`hostnamectl` 与规划一致
统一 SSH 可用	运维终端可免密访问 4 台 VM
时间同步	4 台 VM 时间偏差小于 `1s`
防火墙放行	第 9 节端口按范围放行
enterprise 服务正常	`tier0-enterprise` 健康检查通过
EMQX 状态确认	EMQX 节点、监听端口和持久化目录正常
enterprise 磁盘水位正常	`/` 使用率低于 `70%`，日志和缓存清理策略已配置
DB 主从正常	`repmgr cluster show` 主备和 witness 正常
DB VIP 正常	enterprise 节点连接 `10.60.20.10:5432` 成功
主库故障演练	备库可提升，DB VIP 可漂移

检查项	通过标准
备份恢复演练	可从备份恢复到测试库

实施顺序

1. 创建 4 台 VM，固定主机名、IP、时间同步和 SSH
2. 初始化系统基线、防火墙、磁盘挂载和内核参数
3. 在 `ent-enterprise-01` 部署 `tier0-enterprise`
4. 在 `ent-enterprise-01` 部署单节点 EMQX
5. 部署 PostgreSQL + TimescaleDB 主备
6. 部署 repmgr witness、repmgrd 和 DB VIP 漂移脚本
7. 执行连通性、EMQX 恢复、数据库故障切换、备份恢复和巡检验证
8. 固化运行文档、账号凭据归档和日常巡检任务

风险与待确认项

- 简化版应用层是单点，`ent-enterprise-01` 故障会导致业务整体不可用
- 简化版 EMQX 是单点，不能提供消息层高可用
- `ent-enterprise-01` 只有 `100G` 单系统盘，必须控制日志、缓存、附件和临时文件
- 如后续需要应用层或 EMQX 高可用，应切换到标准版
- 数据库 RPO/RTO 未确认，当前方案按主从自动切换设计，不能承诺零数据丢失
- `DB VIP` 漂移脚本需要在现场验证，特别是虚拟化平台是否允许 VRRP

参考资料

tier0-enterprise 三节点部署方案V2

Enterprise Fleet Center 六机单活主备容灾方案。

> 文档状态：当前实现方案 适用范围：Enterprise Fleet Center 中心节点 部署形态：六台业务机器 + 一台独立部署器控制机 + 四个同网段 VIP 方案定位：单活主备容灾，不是跨机房、共识选主或零数据丢失的严格高可用集群

1. 方案目标

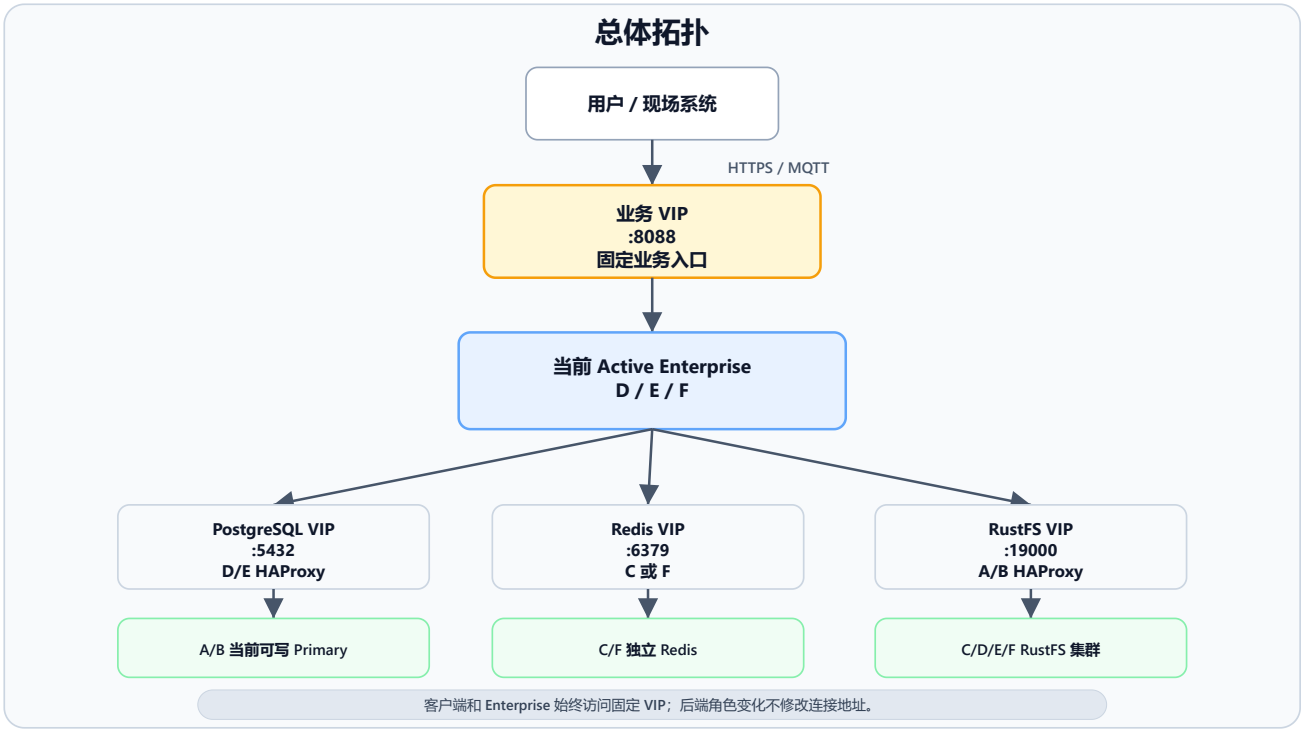
本方案用于在离线或私有化环境中部署一套逻辑上的 Tier0 Enterprise Fleet Center。六台机器 共同承担业务、中间件和代理职责，用户及 Enterprise 内部服务始终访问固定 VIP；单台机器 或单个代理故障后，由 Keepalived、HAProxy、repmgr、RustFS 和 Fleet Agent 分别完成对应 层级的接管。

方案目标如下：

- Fleet Center 中心节点发生单机故障后，业务入口能够漂移到健康 Standby；
- PostgreSQL、Redis、RustFS 均提供固定访问地址，后端节点切换不要求修改 Enterprise `.env`；
- App、Flow、Marimo、License 等本地业务文件能够由 Active 增量同步到 Standby；
- 一份 Cluster License 对应一个逻辑安装实例，切换 Enterprise 节点后不重复激活；
- 通过独立部署器完成六台机器的初始化、角色编排、部署、监控和故障验证；
- 明确数据保护边界，避免把 VIP 漂移、文件同步或双实例误认为完整的数据高可用。

本方案只用于 Fleet Center。Branch Enterprise 仍按普通单机方式安装，不参与六机主备。

2. 总体拓扑



核心请求链路为：

```
用户/现场系统
    ↓
业务 VIP :8088
    ↓
当前 Active Enterprise (D / E / F)
    ├─ PostgreSQL VIP :5432 → D/E HAProxy → A/B 当前可写 Primary
    ├─ Redis VIP      :6379 → C 或 F 独立 Redis
    └─ RustFS VIP    :19000 → A/B HAProxy → C/D/E/F RustFS 集群
```

四个 VIP 是业务配置中的固定入口。Enterprise 切换或中间件后端角色变化后，客户端和 Enterprise 均不修改连接地址。

3. 规划 IP

以下地址只用于展示规划格式，实施时必须替换为现场同一 IPv4 子网中的实际地址。四个 VIP 必须未被 DHCP、静态主机或其他 VRRP 实例占用。

类型	名称	示例 IP	说明
控制面	deployer-controller	`10.60.10.20`	独立部署器，不计入六台业务机器

类型	名称	示例 IP	说明
业务节点	ent-ha-01 / A	`10.60.10.11`	PostgreSQL Primary 等职责
业务节点	ent-ha-02 / B	`10.60.10.12`	PostgreSQL Standby 等职责
业务节点	ent-ha-03 / C	`10.60.10.13`	Witness、Redis-1、RustFS-1
业务节点	ent-ha-04 / D	`10.60.10.14`	Enterprise 初始 Active 等职责
业务节点	ent-ha-05 / E	`10.60.10.15`	Enterprise Standby 等职责
业务节点	ent-ha-06 / F	`10.60.10.16`	Enterprise Standby 等职责
逻辑入口	Business VIP	`10.60.10.101`	Enterprise 统一入口，默认 `8088`
逻辑入口	PostgreSQL VIP	`10.60.10.102`	数据库统一入口，默认 `5432`
逻辑入口	Redis VIP	`10.60.10.103`	Redis 统一入口，默认 `6379`
逻辑入口	RustFS VIP	`10.60.10.104`	S3 统一入口，默认 `19000`

实施前还需要确认：业务网卡名称、IPv4 前缀长度、网关、DNS、NTP、主机名、SSH 用户、磁盘挂载点和防火墙放行范围。VIP、节点 IP 和网卡不能靠脚本猜测后直接用于生产。

4. 六台机器职责



机器	默认部署内容	关键职责
A	PostgreSQL Primary、RustFS HAProxy、Keepalived	初始数据库主库；RustFS 代理候选一
B	PostgreSQL Standby、RustFS HAProxy、Keepalived	数据库流复制备库；RustFS 代理候选二
C	PostgreSQL Witness、Redis-1、RustFS-1	数据库仲裁/状态记录；Redis 与 RustFS 数据节点
D	PG HAProxy、Keepalived、RustFS-2、Enterprise、Fleet Agent、rsync	初始业务 Active；PostgreSQL 代理候选一
E	PG HAProxy、Keepalived、RustFS-3、Enterprise、Fleet Agent、rsync	业务 Standby；PostgreSQL 代理候选二

机器	默认部署内容	关键职责
F	Redis-2、RustFS-4、Enterprise、Fleet Agent、rsync	业务 Standby；第二 Redis 实例

该分配面向单机故障和六机资源均衡。部署器按环境内机器名称/IP 的自然顺序映射 A-F 并 默认勾选角色，提交前仍必须结合现场磁盘和故障域人工复核。

4.1 机器配置清单

扩展能力说明：当前 Enterprise 业务层采用单活主备模式，同一时刻只有业务 VIP 所在节点承载业务请求。当前只支持通过提升单个业务节点的 CPU、内存和磁盘规格进行纵向扩容，不支持通过增加 Enterprise 节点实现多节点负载均衡，也不应按节点数量估算并发能力的线性提升。

六台机器按以下配置准备：A、B 两台 PostgreSQL 数据库节点各配置 `1T`，C、D、E、F 四台中间件/业务节点各配置 `500G`。

机器	节点类型	vCPU	内存	基础存储	系统盘和数据盘建议
A	PostgreSQL Primary 节点	8C	16G	1T	`/` 建议 80G；其余空间用于 PostgreSQL 数据、WAL 和备份暂存
B	PostgreSQL Standby 节点	8C	16G	1T	`/` 建议 80G；其余空间用于 PostgreSQL 数据、WAL 和备份暂存
C	Witness / 中间件节点	4C	8G	500G	系统、Redis 和

机器	节点类型	vCPU	内存	基础存储	系统盘和数据盘建议
					RustFS 数据按固定挂载点分配
D	Enterprise 初始 Active 业务节点	8C	32G	500G	系统、容器、Enterprise 本地同步目录和 RustFS 数据按固定挂载点分配
E	Enterprise Standby 业务节点	8C	32G	500G	系统、容器、Enterprise 本地同步目录和 RustFS 数据按固定挂载点分配
F	Enterprise Standby 业务节点	8C	32G	500G	系统、容器、Enterprise 本地同步目录和 RustFS 数据按固定挂载点分配

基础机器资源汇总如下：

类别	数量	单台规格	小计 vCPU	小计内存	小计基础存储
PostgreSQL 数据库节点	2	`8C / 16G / 1T`	16C	32G	2T
Witness / 中间件节点	1	`4C / 8G / 500G`	4C	8G	500G
Enterprise 业务节点	3	`8C / 32G / 500G`	24C	96G	1.5T
合计	**6**	-	**44C**	**136G**	**4T**

A、B 的 `16G` 内存用于 PostgreSQL/TimescaleDB/pgvector 的缓存和查询，是当前规模的生产基线；数据库量、时序查询或向量检索负载明显增加时再扩到 `32G`。C 同时承载 Witness、Redis 和 RustFS，因此配置 `4C / 8G`，不再按纯 Witness 的轻量规格准备。D、E、F 保持 `8C / 32G` 作为 Enterprise 生产基线，实际并发或 App 负载超出基线时再通过压测扩容。

C、D、E、F 组成 RustFS 四节点集群，四台机器均配置 `500G`。实施时仍需根据文件容量、增长率、保留周期和实际纠删码要求，为系统、Enterprise 本地文件与 RustFS 数据设置固定挂载点和容量预留，并在部署前验证容量、inode、吞吐和重启后自动挂载。

5. 配置汇总

5.1 Enterprise 统一连接配置

D、E、F 的 Enterprise 配置保持一致：

依赖	配置原则
PostgreSQL	连接 PostgreSQL VIP，不写 A/B 节点 IP
Redis	连接 Redis VIP，不写 C/F 节点 IP
文件存储	`FILESTORE_DRIVER=s3`，S3 Endpoint 指向 RustFS VIP
Fleet 身份	三台共享一个逻辑 `installationId`，各自使用独立 `memberId`

依赖	配置原则
Fleet Agent	每套 Enterprise 对应本机一个进程级 Fleet Agent
License	通过业务 VIP 激活一次，Bundle 纳入 HA fileset 同步

`installationId` 和 `memberId` 由安装流程生成和持久化，普通实施人员不应手工编造或在三台机器间复制 `memberId`。

5.2 VIP 与候选节点

VIP	候选节点	健康依据	接管结果
业务 VIP	D、E、F	Fleet Agent readiness、业务容器状态、同步 generation	仅 Owner 启动 Enterprise HA 业务组
PostgreSQL VIP	D、E	HAProxy/Keepalived 状态，HAProxy 只路由可写 Primary	Enterprise 仍使用原 DB 地址
Redis VIP	C、F	Redis `PING` 和本机服务状态	切换到另一独立实例，缓存/会话可能丢失
RustFS VIP	A、B	HAProxy 和 RustFS 后端健康	代理入口漂移，S3 地址不变

6. 组件部署设计

6.1 Enterprise、Fleet Agent 与业务 VIP

- D、E、F 安装同一版本的完整 Enterprise 离线 ZIP；
- Fleet Agent 是宿主机进程级服务，不是业务容器；

- Keepalived 管理业务 VIP，调用 Fleet Agent 的 readiness 接口判断本机是否可以持有 VIP；
- 只有业务 VIP Owner 运行业务组，另外两台保持 Standby；
- Owner 丢失 VIP 后必须停止业务组，恢复节点只能先以 Standby 加入；
- EMQX 随业务组单活运行，不允许绕过 Agent 单独重启或在三台机器上组成另一套选主逻辑。

6.2 PostgreSQL 主备、Witness 与统一入口

- A 初始为 Primary，B 为 Streaming Standby，C 为 repmgr Witness；
- Witness 记录集群状态并参与故障判断，不保存完整业务数据，也不承载业务读写；
- 主库故障时，由 repmgr 流程将健康 Standby 提升为新 Primary；
- D/E HAProxy 根据数据库实际角色，仅把 `5432` 流量发送到当前可写 Primary；
- PostgreSQL VIP 在 D/E 两个代理间漂移，代理故障与数据库主备切换相互解耦；
- 验收必须同时证明“唯一可写 Primary”和“Standby 恢复 streaming”。

Witness 位于只读备库之外，因此不受 Standby 不可写的限制；它参与的是集群观察与仲裁， 不是由 Fleet Agent 向备库写入一套额外的 lease/epoch。

6.3 Redis 双独立实例

- C、F 各运行一个 Redis 单实例；
- 两个 Redis 不组成 Sentinel、Cluster 或主从复制；
- Redis VIP 只提供连接入口切换；当前实例故障后，VIP 漂移到另一健康实例；
- Redis 中只允许保存可丢失或可重建的数据。切换后缓存为空、用户重新登录属于当前设计边界；
- 如果未来 Redis 承载不可丢失状态，必须升级为复制/仲裁方案，不能继续沿用双独立实例假设。

6.4 RustFS 四节点集群

- C、D、E、F 组成 RustFS 四节点分布式对象存储；
- A、B 上的 HAProxy 代理全部健康 RustFS 后端；

- RustFS VIP 在 A/B 之间漂移，Enterprise 始终通过该 VIP 使用 S3；
- 单个 RustFS 节点故障时由集群机制维持服务，单个代理故障时由 VIP 漂移恢复入口；
- 禁止通过 rsync、热拷贝或直接编辑 RustFS 挂载目录来"同步"对象数据；
- RustFS 节点数量、磁盘数和纠删码可用边界以实际部署版本的集群检查结果为准。

6.5 App、Flow、Marimo 与 License 文件同步

Fleet Agent 只同步不能直接放入 PostgreSQL/RustFS、但新 Active 又必须拥有的本地业务文件：

- App 运行目录及 `node_modules`；
- Flow、Marimo、Notebook 等本地运行文件；
- License Bundle；
- 当前实现明确登记的其他 HA fileset。

同步使用系统 `rsync`，通过部署器管理的业务节点专用 SSH 信任网，从 Active 增量发送到 两个 Standby。每次发布生成完整 generation/manifest，Standby 只有确认 generation 完整后 才具备接管资格。VIP 刚切换时，新 Active 不得用旧 generation 反向覆盖其他节点。

PostgreSQL 数据目录、Redis 数据目录和 RustFS 数据目录均不在该同步范围内。

7. 连接方式和网络基线

7.1 访问关系

来源	目标	用途
用户/现场系统	业务 VIP	Web/API 业务访问
D/E/F Enterprise	PostgreSQL VIP	业务数据库读写
D/E/F Enterprise	Redis VIP	缓存和会话
D/E/F Enterprise	RustFS VIP	S3 对象读写
部署器控制机	A-F	SSH、部署、状态采集和运维操作
D/E/F	D/E/F	rsync/SSH 文件增量同步

来源	目标	用途
A/B/C	A/B/C	PostgreSQL 流复制、repmgr 和 Witness 通信
RustFS 成员/代理	C/D/E/F	RustFS 集群和 S3 后端通信
同一 VIP 候选节点	对应候选节点	VRRP 通告与 VIP 漂移

7.2 端口和协议

端口应按"来源→目标"最小化放行，禁止直接把全部管理端口暴露到用户网段。

端口/协议	建议放行范围	用途
`22/TCP`	部署器→A-F；D/E/F 业务同步互访	SSH、部署、rsync
`8088/TCP`	用户网段→业务 VIP	Enterprise 业务入口
`5432/TCP`	D/E/F→PostgreSQL VIP；PG/代理内部互访	PostgreSQL、流复制和代理
`6379/TCP`	D/E/F→Redis VIP；健康检查节点→C/F	Redis 访问和健康检查
`19000/TCP`	D/E/F→RustFS VIP	Enterprise S3 固定入口
`9000/TCP`	A/B→C/D/E/F；RustFS 成员互访	RustFS S3 后端/集群通信，按实际版本确认
`9001/TCP`	运维受控网段→RustFS 管理入口	RustFS 管理端，可选且不得暴露用户网
`19731/TCP`	本机/Keepalived/受控运维网段	Fleet Agent HTTP 健康与控制接口
`18080/TCP`	运维网段→部署器控制机	部署器 HTTPS 页面

端口/协议	建议放行范围	用途
`112/VRRP`	对应 VIP 候选节点互通	Keepalived VIP 漂移，注意它是 IP 协议号而非 TCP/UDP 端口
`1883/8883`	现场设备网段→业务入口，按产品配置选择	MQTT/MQTTS

HAProxy 管理/统计端口、PostgreSQL 角色探测端口及 RustFS 内部端口应以最终部署器生成配置为准，并仅在集群内部按实际需要放行。

8. 故障切换和能力边界

故障场景	预期行为	数据影响/限制
当前 Enterprise Active 故障	业务 VIP 漂移；健康 Standby 校验 generation 后启动业务组	最近一次成功同步后的未同步文件可能丢失
Fleet Agent 故障	readiness 失败，旧业务组停止，业务 VIP 迁移	禁止手工绕过 Agent 启动容器
PostgreSQL Primary 故障	Standby 提升；HAProxy 自动路由新主；PG VIP 不变	切换窗口内连接短暂失败；需确认新备库恢复
一个 PG HAProxy 故障	PostgreSQL VIP 漂移到另一个代理	数据库角色不因此改变
当前 Redis 故障	Redis VIP 漂移到另一独立实例	缓存丢失、会话失效可发生
一个 RustFS 节点故障	其余集群节点继续服务	可容忍范围取决于实际纠删码和磁盘状态
一个 RustFS HAProxy 故障	RustFS VIP 漂移到另一代理	S3 地址不变，存量连接可能重连

故障场景	预期行为	数据影响/限制
部署器控制机故障	已运行数据面继续工作	暂时无法部署、集中监控和运维；控制机本身无自动主备
网络分区	当前方案不保证安全自动选主	可能出现双 VIP/双 Active，必须隔离故障侧并人工处理
任意两台同时故障	不作统一可用性承诺	结果取决于两台的角色组合，需逐场景验证

当前方案没有共识系统、租约、`leaderEpoch` 或 fencing，因此不宣称网络分区下的严格 HA。它解决的是同一二层网络内的常见单机故障和人工恢复成本问题，产品口径应称为"主备容灾"。

9. 备份、监控和巡检

VIP 漂移和 rsync 都不能替代备份。生产环境至少需要：

- PostgreSQL：独立定时备份、保留策略和恢复演练；
- RustFS：按实际集群版本设计对象数据保护、磁盘监控和恢复流程；
- Redis：确认其中仅保存可丢失数据；如果不是，先升级架构再上线；
- Enterprise 本地 fileset：监控最近成功 generation、同步延迟、失败次数和 Standby 完整性；
- 部署器：备份数据卷、TLS/登录材料、SSH/Host Key 配置、完整交付包和校验文件；
- 通用指标：CPU、内存、磁盘容量、inode、磁盘 I/O、网卡错误、容器/服务重启次数；
- HA 指标：四个 VIP Owner、唯一 Enterprise Active、唯一 PostgreSQL Primary、Standby streaming；
- 审计：记录部署、切换、重启、密码查看、License 操作和失败恢复。

日常操作和故障处理见 HA 日常运维手册（仓库 doc/ha/operations-guide.md）。

10. 上线前检查

检查项	通过标准
机器数量和角色	六台业务机器 A-F 与计划一致，部署器未混入业务节点
SSH 和 sudo	部署器可使用受控账号免交互访问 A-F，Host Key 已核验
网络与时间	六台同网段，四 VIP 未占用，网卡/前缀正确，时间偏差满足要求
Docker/Compose	六台初始化成功；无旧容器、端口、网络和数据目录冲突
磁盘	PostgreSQL、RustFS、Enterprise fileset 使用计划挂载点，容量和 inode 正常
PostgreSQL	唯一 Primary、Standby streaming、Witness 在线、通过 PG VIP 可读写
Redis	C/F 实例健康，通过 Redis VIP 可 `PING`，已接受切换后数据不复制的边界
RustFS	四节点均健康，通过 RustFS VIP 可完成对象写入、读取和删除
Enterprise	D/E/F 版本一致，仅业务 VIP Owner 运行业务组
Fleet Agent	三台 Agent 在线，readiness、同步历史和 generation 状态可检查
License	业务 VIP 激活一次；切换后无需重新激活
App 同步	导入包含 `node_modules` 的 App，同步完成后切换仍可运行
防火墙	只放行本方案所需来源、目标和协议
备份	PostgreSQL、RustFS 和部署器备份方案已落地并至少完成一次恢复验证

11. 实施顺序

1. 准备独立部署器控制机、六台业务机器、四个 VIP、网卡和磁盘规划；
2. 校验完整交付包和 Enterprise 离线 ZIP 的 SHA256；
3. 启动部署器，录入 SSH 凭据和六台机器并完成连通性检查；
4. 初始化六台目标机的 Docker/Compose 和离线镜像环境；
5. 创建"标准部署"，确认 A-F 角色分配和四个 VIP；
6. 依次部署 PostgreSQL、Redis、RustFS、HAProxy 和 Keepalived；
7. 建立 D/E/F 专用 rsync 账号和 SSH 信任网；
8. 部署 D/E/F Enterprise 与 Fleet Agent，确认只有 D 初始 Active；
9. 通过业务 VIP 完成一次 License 激活并等待 License 同步成功；
10. 导入测试 App，验证 App、`node_modules`、Flow/Marimo 和 License fileset 同步；
11. 分别执行 Enterprise、PostgreSQL、Redis、RustFS 单节点故障演练；
12. 恢复节点并确认唯一 VIP Owner、唯一 Active、唯一 Primary 和完整同步状态；
13. 完成备份恢复演练、运维交接和现场参数归档。

具体页面操作见 HA 部署安装流程（仓库 doc/ha/deployment-guide.md），AI 自动化验证用例见 HA 环境测试指南（仓库 doc/fleet/ha-test-guide.md）。

12. 验收标准

12.1 功能验收

- 用户仅通过业务 VIP 登录和使用 Enterprise；
- Enterprise 仅通过 PostgreSQL、Redis、RustFS VIP 访问中间件；
- D/E/F 任意时刻只有一个 Active，失去业务 VIP 的节点不运行业务组；
- PostgreSQL 任意时刻只有一个可写 Primary，Witness 不承载业务读写；
- Redis 和 RustFS 后端切换后，Enterprise 无需修改 `.env`；
- License 只激活一次，业务节点切换后不进入激活页；
- 已完成 generation 的 App、Flow、Marimo 和 License 在新 Active 可用；
- 故障恢复节点以 Standby 加入，不用旧数据覆盖新 Active。

12.2 故障演练验收

- Enterprise Active 单机故障后，业务 VIP 自动漂移并恢复访问；
- PostgreSQL Primary 故障后，Standby 成为唯一新主，PG VIP 仍可读写；
- 当前 PG HAProxy 故障后，PG VIP 漂移，Enterprise 数据库连接恢复；
- 当前 Redis 实例故障后，Redis VIP 漂移，业务能够在允许的数据丢失边界内恢复；
- 一个 RustFS 节点或一个 RustFS HAProxy 故障后，通过 RustFS VIP 的对象读写恢复；
- 每次演练后均恢复稳定基线，不遗留双 VIP、双 Active 或双 Primary。

13. 风险和待现场确认项

- 四个 VIP 是否确实位于六台机器可共同持有的同一二层网络；
- 网络设备是否允许 VRRP/GARP，安全策略是否会阻断 VIP 漂移；
- 现场是否完全断网；当前部署器仍有 Harbor 可达性检查，需提前验证路由或代理；
- PostgreSQL、RustFS 和 Enterprise fileset 的容量、增长率、RPO/RTO 目标；
- Redis 中是否存在不能丢失的数据；若存在，当前双独立实例方案不满足要求；
- App/Flow/Marimo 的最大文件量、单次变更量和可接受同步延迟；
- 是否存在跨网段、跨机房、网络分区或任意双机故障的强制可用性要求；
- 生产证书、域名、密码、License、SSH Key 和审计保管责任人；
- 部署器控制机的恢复方式、备份周期和替代运维通道。

上述任一项改变，都可能影响 VIP、角色分配、存储和故障边界，必须先更新方案再实施。

14. 结论

当前方案以六台机器、四个固定 VIP 和单活 Enterprise 为核心，把业务入口、数据库入口、 缓存入口和对象存储入口分别解耦。它能够覆盖同网段内常见的单机故障，并显著减少切换后 修改连接配置、重新激活 License 和手工复制 App 文件的工作量。

同时必须接受三个边界：Redis 不复制数据、业务 fileset 为异步同步、网络分区没有共识仲裁。因此本方案适合定义为 **Fleet Center 六机单活主备容灾方案**，不应对外承诺零中断、零数据 丢失或跨机房严格高可用。

参考资料

SLA 与高可用边界

标准 Tier0 Enterprise 部署的 SLA 建议、高可用边界、验收门槛、演练和加固项。

本文档适用于六虚拟机全离线部署：三台应用 / Swarm 节点、两台 PostgreSQL 高可用节点和一台 witness 节点。

架构基线

层级	当前设计	高可用边界
应用运行时	Enterprise New compose 在固定的 `tier0_app_active_node` 上运行。	`backend`、`redis`、`EMQX`、`SourceFlow`、`EventFlow`、`Marimo` 和 `OPC UA Server` 尚未自动多副本化。
HTTP 入口	三台应用节点运行 keepalived APP VIP + HAProxy。	VIP 可漂移。HAProxy 转发到活动节点的 `8088/TCP`，并检查 `/healthz`。
MQTT / OPC UA	EMQX 使用 `1883/8883/8083/8084`；OPC UA 使用 `4840`。	这些端口直接连接活动节点，不包含在 TCP VIP 代理中，因此不承诺协议入口高可用。
数据库	PostgreSQL / TimescaleDB 流复制 + repmgr + witness + DB VIP。	primary 故障时 standby 可自动提升。异步复制不能承诺严格零 RPO。
备份与恢复	`backup-db.sh` / `restore-db.sh`。	提供可恢复性，但不等同于在线高可用。

推荐 SLA

服务对象	推荐表述	约束
平台 HTTP 入口	从月可用性 `99.5%` 开始。	VIP 故障切换不等同于活动应用节点自动迁移。
数据库访问	运维目标：`RTO <= 2 minutes`。	RPO 取决于复制延迟。同架构历史故障切换测试约为 59-64 秒。
MQTT / OPC UA	当前设计不承诺 HA SLA。	必须先增加 HAProxy TCP 代理、独立 VIP 或服务集群。
备份与恢复	每日备份；重大变更前必须备份。	恢复时间取决于数据量和现场 I/O。

在声明 `99.9%` 或更高可用性之前，需要完成应用自动迁移或多副本部署、协议入口高可用、监控告警和恢复演练。不要把规划能力描述为已交付能力。

验收门槛

每次部署或升级至少需要满足以下条件：

- `verify-materials.sh` 通过，交付包 SHA256 校验通过。
- 三台 Swarm 节点均为 `Ready`。
- `APP VIP /healthz` 和 `/readyz` 返回 `200`，首页和 `/uns` 可访问。
- 活动节点上的七个 compose 服务正在运行。
- `DB VIP:5432` 可访问，primary、standby 和 witness 拓扑正常。
- 默认账号可以实际登录。不要只根据端口、容器或 HTTP 200 响应判断 IAM 初始化成功。
- HTTP、EMQX 端口、WebSocket/WSS 和 OPC UA TCP 冒烟检查通过。

巡检与告警

每天运行以下命令：

```
./bin/verify.sh --site config/site.conf
```

```
./bin/smoke-business-emqx.sh --site config/site.conf
```

至少配置以下告警：

- APP VIP `/healthz` 或 `/readyz` 失败。
- DB VIP 不可达、无 primary、split-brain 风险、复制中断或复制延迟超过阈值。
- keepalived、HAProxy、PostgreSQL 或 repmgrd 状态异常。
- `backend`、`redis`、`EMQX` 等关键容器退出或频繁重启。
- 数据盘使用率超过 `80% / 90%`。
- 最新备份失败或未生成。

演练要求

演练	频率	验证标准
APP VIP 故障切换	交付前和每季度。	停止当前 VIP 节点上的 keepalived 后，`/healthz` 和 `/readyz` 恢复。
DB primary 故障	交付前和每半年。	standby 被提升，DB VIP 恢复，旧 primary 被重建为 standby。
数据库恢复	每月抽样，并在重大变更前执行。	备份可恢复。恢复后 `verify`、冒烟检查和登录验证通过。
物料验证	每次打包构建。	源物料、包 SHA 和解包验证通过。

事件处理原则

1. 先确认影响范围：HTTP、MQTT、OPC UA、数据库以及单节点影响。
2. 优先恢复 VIP、HAProxy、活动应用和数据库 primary。
3. 保留日志、数据目录和备份。不要直接清理或覆盖事故现场。

4. 对于数据库事件，先确定当前 primary，再决定故障切换、重新加入或恢复。
5. 恢复后运行 `verify`、冒烟检查和实际登录验证。

高 SLA 加固项

- 将应用改为多副本，或建立可验证的活动节点自动迁移流程。
- 将 EMQX 集群化，或通过 HAProxy TCP / TLS 和专用 VIP 路由。
- 通过 HAProxy TCP、专用 VIP 或服务集群为 OPC UA 提供高可用入口。
- 使用 PostgreSQL 同步复制，或明确可接受的复制延迟和 RPO。
- 自动化备份计划、异地复制和定期恢复演练。
- 将容器、主机、数据库、日志和黑盒探针接入统一告警。

相关流程：运维 Runbook 和 标准端口清单。

参考资料

标准端口清单

标准六虚拟机离线 Tier0 Enterprise 部署的防火墙端口清单。

下表基于 Enterprise New 六虚拟机离线部署。客户防火墙应只允许来自最小必要源网段的流量，不要将这些端口直接暴露到公网。

来源	目标	端口	协议	用途
控制主机	六台目标 VM	22	TCP	SSH / Ansible
三台应用节点互通	三台应用节点	2377	TCP	Docker Swarm 管理平面
三台应用节点互通	三台应用节点	7946	TCP/UDP	Swarm 节点发现
三台应用节点互通	三台应用节点	4789	UDP	Swarm overlay 网络
三台应用节点	活动应用节点	8088	TCP	HAProxy 转发到 Enterprise New 后端；仅限应用网段
业务网段	APP VIP	80	TCP	平台 HTTP 入口、`/healthz`、`/readyz`
业务网段	APP VIP / 活动应用节点	1883	TCP	MQTT
业务网段	APP VIP / 活动应用节点	8883	TCP	MQTT TLS
业务网段	APP VIP / 活动应用节点	8083	TCP	MQTT WebSocket

来源	目标	端口	协议	用途
业务网段	APP VIP / 活动应用节点	8084	TCP	MQTT WebSocket TLS
业务网段	APP VIP / 活动应用节点	4840	TCP	OPC UA Server
受控应用网段	活动应用节点	8089	TCP	应用中心
动态应用访问网段	活动应用节点	30000-30199	TCP	App Gateway / 动态应用主机端口
应用节点、DB 节点和 Witness	DB VIP / DB 节点	5432	TCP	PostgreSQL / TimescaleDB
DB1、DB2 和 Witness	DB1、DB2 和 Witness	5432	TCP	Streaming replication / repmgr
应用节点	NFS server	2049	TCP/UDP	`NFS_ENABLED=true` 时使用

当前 compose 部署不会对外暴露 Redis、SourceFlow、EventFlow、Marimo 或 EMQX Dashboard。这些服务只由 Docker runtime 网络内的后端调用。

参考资料

运维 Runbook

标准 Tier0 Enterprise 部署的日常巡检、故障切换、备份、恢复、清理和故障排查流程。

本文档基于 Enterprise New 离线应用。

日常巡检

1. 在部署包根目录运行以下命令：

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

2. 确认以下项目：
   - 3 台 Swarm 节点均为 `Ready`。
   - `APP VIP /healthz` 和 `/readyz` 返回 `200`。
   - `DB VIP:5432` 可访问。
   - PostgreSQL primary、standby 和 witness 拓扑正常。
   - `backend`、`redis`、`emqx`、`sourceflow`、`eventflow`、`marimo` 和 `opcua-server` 都运行在活动应用节点上。

应用检查

1. 在活动应用节点上运行以下命令：

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh ps
bash bin/compose.sh logs --tail=200 backend
curl -fsS http://127.0.0.1:8088/healthz
curl -fsS http://127.0.0.1:8088/readyz
```

> ⓘ 注意

不要绕过 `bin/compose.sh` 直接运行原始 `docker compose` 命令。该包装脚本会加载安装器生成的运行时环境。

2. 健康检查解读：

   - `healthz` 失败：先检查 backend 进程、镜像和容器日志。
   - `readyz` 失败：同时检查 DB VIP、Redis 和数据库迁移日志。
   - 首页可访问但无法登录：确认 `.env` 中的 `ADMIN_INITIAL_PASSWORD`，然后检查 backend IAM migration 日志和用户表。不要只根据 HTTP 端口判断部署成功。

协议入口检查

```
nc -vz <APP_VIP> 1883
nc -vz <APP_VIP> 8883
nc -vz <APP_VIP> 8083
nc -vz <APP_VIP> 8084
nc -vz <APP_VIP> 4840
```

> ⓘ 注意
> EMQX 和 OPC UA Server 固定在活动应用节点上。APP VIP 故障切换只保证 `APP VIP:80` HTTP 入口。协议端口未通过 HAProxy TCP 代理路由，因此该设计不承诺协议入口高可用。

数据库检查

在数据库节点上运行：

```
systemctl status tier0-postgresql
systemctl status tier0-repmgrd
sudo -iu postgres psql -Atqc 'select pg_is_in_recovery();'
sudo -iu postgres /usr/local/bin/repmgr -f /etc/repmgr.conf cluster show
```

- `pg_is_in_recovery=false` 表示 primary。
- `pg_is_in_recovery=true` 表示 standby。
- DB VIP 只能位于当前 primary 上。

- 如果 standby 复制中断，或 witness 报告上游错误，先保护现场。不要直接重新运行完整部署。

APP VIP 故障切换

1. 测试前，记录当前持有 VIP 的节点并验证：

```
curl -fsS http://<APP_VIP>/healthz
curl -fsS http://<APP_VIP>/readyz
```

2. 停止当前 VIP 节点上的 `keepalived` 后，确认 VIP 被其他应用节点接管，然后重新测试两个 URL。HAProxy backend 指向活动应用节点的 `8088/TCP`，因此三台应用节点都必须能访问活动节点的 8088 端口。

3. 恢复 `keepalived` 后，再次运行 `verify.sh` 和 smoke checks。演练期间不要停止活动应用容器。

DB Primary 故障切换

故障演练前，创建备份并确认复制健康：

```
./bin/backup-db.sh --site config/site.conf
```

停止当前 primary 上的 `tier0-postgresql` 后，观察 `repmgr` 提升 standby，并确认 DB VIP 接管。目标 RTO 为 2 分钟内。同架构历史测试约为 59-64 秒。

> ⚠ 警告
>
> - 旧 primary 不能直接启动回生产拓扑。先停止 PostgreSQL、repmgrd 和 keepalived；移除残留 DB VIP；再从当前新 primary 执行 basebackup，将该节点注册为 standby，最后重新连接 witness。
> - 故障切换后，不要直接重新运行完整安装器。inventory 中静态的 primary/standby 定义可能扰动当前拓扑。

备份与恢复

- 备份：

```
./bin/backup-db.sh --site config/site.conf
```

- 恢复：

```
./bin/restore-db.sh --site config/site.conf --restore-file /backup/tier0-db/postgres-YYYYmmddTHHMMSS.dump
```

> ⓘ 注意
> 恢复前确认当前 primary。恢复后运行 `verify.sh`、smoke checks，并实际验证默认用户路径的平台登录。

清理与重测

```
./full-cleanup.sh --env install.env --yes
./install.sh --auto --yes
```

> ⚠ 警告
> 完整清理会删除 6 台节点上的 Tier0 运行时数据、Docker/Swarm 数据和数据库备份。它还会在控制容器退出后对控制节点执行最终清理。仅在确认可丢弃的测试环境中使用该入口。如果需要保留客户数据或事故现场，请使用普通 `--cleanup`，并先完成备份。

常见问题

Backend 启动失败

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=300 backend
bash bin/compose.sh ps
```

ⓘ 注意
重点检查 DB URL、DB VIP、Redis、migration 和挂载目录权限。

EMQX 启动失败

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=200 emqx
```

ⓘ 注意
检查证书、配置文件、卷权限，以及端口 `1883/8883/8083/8084` 是否被占用。

repmgrd 反复重启

```
systemctl status tier0-repmgrd
tail -100 /var/log/repmgr/repmgr.log
sudo -iu postgres psql -Atqc 'show shared_preload_libraries;'
```

ⓘ 注意
确认 `shared_preload_libraries` 包含 `repmgr`，并且 systemd unit 使用 `repmgrd --daemonize=false`。

物料校验失败

```
./bin/verify-materials.sh
```

⚠ 警告
不要跳过 SHA256 校验继续部署。请重新获取交付包，或按可追溯流程重新构建物料。

参考资料

OpenAPI 参考附录

Tier0 OpenAPI v1 · 操作: 36 · 架构: 85

操作

API 字段名、示例及源描述沿用随附的英文 OpenAPI 规范。

UNS

读取、写入和管理统一命名空间数据。

`POST` **`/uns/attachments`**

Upload a UNS attachment
Uploads a multipart attachment for a UNS node. Supply a file and at least one of unsId or topic; a valid unsId takes precedence.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments' \
  --header 'x-api-key: <api-key>'
```

`multipart/form-data`

字段	类型	必填	描述
`unsId`	`integer (int64)`	否	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
`topic`	`string`	否	Full owner node path, used when a valid unsId is not supplied.
`fileName`	`string`	否	Original file name.
`sha256`	`string`	否	SHA-256 checksum of the file contents.
`file`	`string (binary)`	是	Binary file contents. The multipart field name must be file.

响应

`200` `400` `401` `403` `404` `409` `413` `500`

成功响应示例

200 · application/json


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "fileName": "string",
    "filePath": "string",
    "fileUrl": "string",
    "contentType": "string"
  }
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	UnsAttachmentUploadResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/attachments/list

List UNS attachments

Lists attachments for a UNS node or topic. Temporary file URLs can expire and must then be requested again.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"topic":"string","page":0,"size":0,"includeFileUrl":false}'
```

application/json

字段	类型	必填	描述
unsId	integer (int64)	否	Numeric UNS node ID.
topic	string	否	Full UNS topic path.
page	integer (int64)	否	Page number, starting at 1.
size	integer (int64)	否	Number of attachments per page.
includeFileUrl	boolean (boolean)	否	Whether to return temporary file URLs. Defaults to true.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	UnsAttachmentListResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/browse

Browse the UNS tree

Returns a UNS subtree. Use an empty path for the workspace root.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/browse' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string","max_depth":0,"include_metadata":false,"include_leaf_value":false}'
```

application/json

字段	类型	必填	描述
path	string	否	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	否	Maximum tree depth. Omit or use 0 for the server default.

字段	类型	必填	描述
include_metadata	boolean (boolean)	否	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	否	Whether returned leaf nodes should include their latest payload.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "tree": [
      "value"
    ]
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	BrowseResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

状态	含义	媒体类型
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/create

Create UNS nodes

Creates UNS nodes and nested subtrees. Check each item result and use the returned topic in subsequent calls.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"namespace":
[{"name":"value","type":"value","children":"value","topicType":"value","fields":"value"}]}'
```

application/json

字段	类型	必填	描述
namespace	array<object>	是	UNS nodes to create. Nest children to create a subtree.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	CreateResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/delete

Delete UNS nodes

Deletes the listed nodes sequentially. By default this is a soft delete; the batch is not rolled back when a later item fails.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"hard_delete":false}'
```

application/json

字段	类型	必填	描述
topics	array<string>	是	UNS topic or path node paths to delete.
hard_delete	boolean (boolean)	否	Permanently delete instead of moving nodes to the recycle state.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	OpenapiEmptyResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–

状态	含义	媒体类型
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/history

Query UNS history
Queries historical topic values over an inclusive RFC 3339 time range. Results can be aggregated or sampled.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/history' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"start_time":"string","end_time":"string","aggregation":
{"interval":"string","function":"string","field":"string","fields":["value"]},"page":0}'
```

application/json

字段	类型	必填	描述
topics	array<string>	是	UNS topic paths whose history should be queried.
start_time	string	是	Inclusive query start in RFC 3339 format.
end_time	string	是	Inclusive query end in RFC 3339 format.
aggregation	object	否	HistoryAggregation. 请参阅架构章节中的引用架构。
page	integer (int64)	否	Page number, starting from 1.
size	integer (int64)	否	Page size; values above the server maximum are capped.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	HistoryResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/read

Read UNS values

Returns value, quality, and timeStamp (VQT) for each requested topic. A failed topic does not discard successful items in the same batch.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/read' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"include_metadata":false,"include_leaf_value":false}'
```

application/json

字段	类型	必填	描述
topics	array<string>	是	UNS topic paths to read.
include_metadata	boolean (boolean)	否	Whether each result should include UNS node metadata.
include_leaf_value	boolean (boolean)	否	Whether leaf nodes should include their latest payload.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	ReadResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/restore

Restore a deleted UNS node

Restores a soft-deleted UNS node. Permanently deleted nodes cannot be restored.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/restore' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string"}'
```

application/json

字段	类型	必填	描述
path	string	是	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	OpenapiEmptyResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/search

Search UNS nodes

Searches UNS node names, aliases, and paths, with optional type and path filters.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/search' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"keyword":"string","topicType":"string","path_prefix":"string","include_metadata":false,"include_
leaf_value":false}'
```

application/json

字段	类型	必填	描述
keyword	string	否	Keyword matched against UNS node names, aliases, and paths.
topicType	string	否	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	否	Only return nodes under this UNS path prefix.
include_metadata	boolean (boolean)	否	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	否	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	否	Page number, starting from 1.
size	integer (int64)	否	Page size.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "objects": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	SearchResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/unsBindFlow

Bind a flow to a UNS node
Binds a flow ID to a UNS node ID.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/unsBindFlow' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"flowId":0}'
```

application/json

字段	类型	必填	描述
unsId	integer (int64)	是	UNS node ID to bind.
flowId	integer (int64)	是	Flow ID to bind.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "unsId": 0,
    "flowId": 0
  }
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	UnsBindFlowResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

状态	含义	媒体类型
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/update

Update a UNS node

Updates an existing UNS node. Empty strings, an empty fields array, and an empty extendProperties object retain existing values.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"path":"string","name":"string","description":"string","displayName":"string","extendProperties":
{}}'
```

application/json

字段	类型	必填	描述
path	string	是	Existing UNS topic or path node to update.
name	string	否	New node name.
description	string	否	Updated node description.
displayName	string	否	Updated display name.
extendProperties	object	否	Updated custom metadata key-value pairs.
fields	array<object>	否	Updated payload schema fields.
alias	string	否	Updated topic alias.
enableHistory	boolean (boolean)	否	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	否	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	OpenapiEmptyResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/write

Write UNS values

Writes values to existing TOPIC nodes; wildcards are not supported. `value` must match the node payload schema. **Batch results:** check each `data.results` item, not only HTTP 200. **Retry:** confirm the

outcome before retrying a write.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/write' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"writes":
[{"topic":"value","value":"value","timeStamp":"value"}],"qos":0,"retain":false}'
```

`application/json`

字段	类型	必填	描述
writes	array<object>	是	Write operations to apply.
qos	integer (int64)	否	MQTT QoS: 0, 1, or 2.
retain	boolean (boolean)	否	Whether the MQTT message should be retained.

响应

200 400 401 403 404 409 413 500

成功响应示例

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

成功响应字段

`200 · application/json`

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	WriteResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	Inspect the success flag of each `data.results` item. HTTP 200 alone does not confirm that every write succeeded.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

文件资源

上传、下载和管理存储文件。

POST /assets/files

Create a direct-upload form

Issues an object-store upload form; it does not transfer file contents. Submit every returned post field unchanged with the file.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"fileName":"string","contentType":"string","size":0,"business":"string","useBy":"string"}'
```

application/json

字段	类型	必填	描述
fileName	string	是	Original file name.
contentType	string	否	MIME content type.

字段	类型	必填	描述
size	integer (int64)	是	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
business	string	否	Business domain or feature associated with the file.
useBy	string	否	Optional usage category: user, workspace, or platform.
visibility	string	否	File visibility, such as private or public.
appInstanceId	string	否	Associated application instance ID.
sessionId	string	否	Caller session ID.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileId": 0,
    "filePath": "string",
    "fileUrl": "string",
    "uploadUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileUploadResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json

状态	含义	媒体类型
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/delete

Delete a file

Deletes a file identified by its server-issued storage path.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"filePath":"string"}'
```

application/json

字段	类型	必填	描述
filePath	string	是	Server-issued storage path returned by an upload API.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "deleted": false
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileDeleteResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/download

Download a private file

Returns file bytes directly. Do not forward x-api-key or other credentials when a client follows a redirect to a cross-origin storage URL.

请求参数

名称	位置	类型	必填	描述
filePath	query	string	是	Server-issued storage path returned by an upload API.
responseContentDisposition	query	string	否	Optional Content-Disposition value for the download response.

响应

302 400 401 403 404 409 413 500

响应状态含义

状态	含义	媒体类型
302	Redirect to the final download URL.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/abort

Abort a multipart upload
Cancels a multipart upload session.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/abort' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string"}'
```

application/json

字段	类型	必填	描述
fileKey	string	是	Object-store key used for the multipart upload.
uploadId	string	是	Object-store multipart upload session ID.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "aborted": 1
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileMultipartAbortResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–

状态	含义	媒体类型
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/complete

Complete a multipart upload
Completes a multipart upload after every uploaded part number and ETag have been supplied.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/complete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","parts":[{"partNumber":"value","etag":"value"}]}'
```

application/json

字段	类型	必填	描述
fileKey	string	是	Object-store key used for the multipart upload.
uploadId	string	是	Object-store multipart upload session ID.
parts	array<object>	是	Uploaded part numbers and their ETags.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "filePath": "string",
    "fileUrl": "string",
    "sizeBytes": 0,
    "expiresAt": 0
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileMultipartCompleteResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/init

Initialize a multipart upload
Creates a multipart upload session and returns its ID, part size, and part count.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/init' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","contentType":"string","size":0,"partSize":0,"business":"string"}'
```

application/json

字段	类型	必填	描述
fileName	string	是	Original file name.
contentType	string	否	File MIME type, for example application/octet-stream.

字段	类型	必填	描述
size	integer (int64)	是	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	否	Preferred part size in bytes. The server may normalize it.
business	string	否	Business domain or feature associated with the file.
useBy	string	否	Optional usage category: user, workspace, or platform.
visibility	string	否	File visibility: private or public.
appInstanceId	string	否	Associated application instance ID.
sessionId	string	否	Caller session ID.
bizId	string	否	Associated business identifier.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileKey": "string",
    "filePath": "string",
    "uploadId": "string",
    "partSize": 0,
    "partCount": 0
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileMultipartInitResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/part-urls

Create multipart upload URLs

Returns presigned PUT URLs for the requested one-based part numbers.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/part-urls' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","partNumbers":[0]}'
```

application/json

字段	类型	必填	描述
fileKey	string	是	Object-store key used for the multipart upload.
uploadId	string	是	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	是	One-based part numbers to sign.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 0,
  "data": {
    "partUrls": [
      "value"
    ]
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileMultipartPartUrlsResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/public

Download a public file

Downloads bytes for a file explicitly marked public. An unguessable path is not authorization; do not mark sensitive files public.

请求参数

名称	位置	类型	必填	描述
`filePath`	query	`string`	是	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	否	Optional Content-Disposition value for the download response.

响应

`200` `400` `404` `409` `413` `500`

响应状态含义

状态	含义	媒体类型
`200`	Returns file bytes directly. Content-Type depends on the file type; no JSON envelope is used.	–
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

`GET` `/assets/files/url`

Create a temporary file URL
Creates a temporary URL for a file.

请求参数

名称	位置	类型	必填	描述
`filePath`	query	`string`	是	Server-issued storage path returned by an upload API.
`expiredSec`	query	`integer (int64)`	否	Requested temporary URL lifetime in seconds.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AssetFileUrlResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/upload

Create a legacy asset upload URL
Legacy compatibility endpoint. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/upload' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","sizeBytes":0,"contentType":"string","size":0,"sha256":"string"}'
```

application/json

字段	类型	必填	描述
fileName	string	是	Original file name.
sizeBytes	integer (int64)	是	File size in bytes.
contentType	string	否	File MIME type, for example application/octet-stream.
size	integer (int64)	否	File size in bytes.
sha256	string	否	Optional SHA-256 checksum of the file content.
storageKey	string	否	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	否	Whether the file should be accessible through the public download endpoint.
business	string	否	Business domain or feature that owns the uploaded file.
useBy	string	否	Usage scenario or owner category for the uploaded file.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "uploadUrl": "string",
    "filePath": "string",
    "fileUrl": "string"
  }
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
msg	string	否	Response message
data	object	否	AssetUploadResp. 请参阅架构章节中的引用架构。

响应状态含义

状态	含义	媒体类型
200	Legacy upload response. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

身份认证

验证当前 OpenAPI 身份与权限。

POST `/auth/whoami`

Inspect the current API key identity
Returns the current API-key identity and its effective permissions. Call this before integration work to verify access.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 0,
  "data": {
    "userID": 0,
    "userName": "string",
    "email": "string",
    "workspaceID": 0,
    "workspaceName": "string"
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	AuthWhoamiResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

状态	含义	媒体类型
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Flow

管理 Source Flow 与 Event Flow 定义。

POST `/flow/create`

Create a flow
Creates a SourceFlow or EventFlow and returns its numeric resource ID.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowName":"string","flowType":"string","description":"string","template":"string"}'
```

application/json

字段	类型	必填	描述
flowName	string	是	Flow name, up to 63 Unicode characters.
flowType	string	是	Flow category: SourceFlow or EventFlow.
description	string	否	Flow description.
template	string	否	Optional flow template JSON.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "brokerID": "string"
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowCreateResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/delete

Delete flows

Deletes the listed flows sequentially; earlier deletions are not rolled back when a later item fails.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"ids":[0]}'
```

application/json

字段	类型	必填	描述
ids	array<integer (int64)>	是	Flow IDs to delete.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowEmptyResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

状态	含义	媒体类型
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/deploy

Deploy a flow

Deploys a complete Node-RED canvas JSON string to a flow.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/deploy' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowsJson":"string"}'
```

application/json

字段	类型	必填	描述
id	integer (int64)	是	Flow ID to deploy.
flowsJson	string	是	Node-RED canvas JSON to deploy.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "flowId": "string"
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowDeployResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/flowdata

Get flow canvas data

Returns the Node-RED canvas revision and node objects for a flow.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/flowdata' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

字段	类型	必填	描述
id	integer (int64)	是	Flow ID.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 0,
  "data": {
    "rev": "string",
    "flows": [
      "value"
    ]
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowDataResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

状态	含义	媒体类型
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/get

Get a flow

Returns flow metadata for a numeric flow ID.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/get' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

字段	类型	必填	描述
id	integer (int64)	是	Flow ID.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "flowId": "string",
    "flowName": "string",
    "flowType": "string",
    "flowStatus": "string"
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowInfo. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/list

List flows

Lists flows, optionally filtered by a keyword or flow category.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","flowType":"string"}'
```

application/json

字段	类型	必填	描述
keyword	string	否	Keyword matched against flow names and descriptions.
flowType	string	否	Flow category filter: SourceFlow or EventFlow.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 200,
  "msg": "success",
  "data": {
    "list": [
      {
        "id": 101,
        "flowId": "flow-101",
        "flowName": "Line 1 collector",
        "flowType": "source",
        "flowStatus": "draft",
        "template": "[]",
        "description": "Collects line telemetry",
        "isFavorite": 2,
        "currentVersionId": 0,
        "currentVersionName": "",
        "currentVersionType": "",
        "createdTime": 1788134400000,
        "updatedTime": 1788134400000
      }
    ]
  }
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowListResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

状态	含义	媒体类型
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/nodes

List available flow node types

Returns the Node-RED node sets available for a flow category.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/nodes' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowType":"string"}'
```

application/json

字段	类型	必填	描述
flowType	string	是	Flow category whose Node-RED node catalog should be returned.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "nodes": [
      "value"
    ]
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowNodesResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/update

Update a flow

Updates flow metadata and settings for a numeric flow ID.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowName":"string","description":"string","template":"string","isFavorite":0}'
```

application/json

字段	类型	必填	描述
id	integer (int64)	是	Flow ID to update.
flowName	string	否	Updated flow name.
description	string	否	Updated flow description.
template	string	否	Updated flow template JSON.
isFavorite	integer (int64)	否	Favorite state: 1 for favorite, 2 for not favorite.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	FlowEmptyResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

状态	含义	媒体类型
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

平台与项目

读取平台信息并管理项目。

POST /info

Get service information
Returns the service name, version, reported capabilities, and configured MQTT broker.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/info' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{}'
```

`application/json`
OpenAPI 未声明此请求体的对象字段。

响应

200 400 401 403 404 409 413 500

响应状态含义

状态	含义	媒体类型
200	A successful response.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

状态	含义	媒体类型
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /launchpad/{projectName}/getMembers

List project members

Lists Launchpad project members. URL-encode projectName when it contains special characters.

请求参数

名称	位置	类型	必填	描述
projectName	path	string	是	Project name in the Launchpad URL. URL-encode special characters.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/launchpad/{projectName}/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"roleKey":"string","roles":
["string"],"updatedAtStart":"string","updatedAtEnd":"string","page":0}'
```

application/json

字段	类型	必填	描述
roleKey	string	否	Role key filter.
roles	array<string>	否	Role filters or the returned role list, depending on the model.
updatedAtStart	string	否	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	否	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	否	Page number, starting at 1.
size	integer (int64)	否	Number of members per page.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	LaunchpadMembersResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /platform/getMembers

List platform members
Lists platform members with optional keyword, role, status, and update-time filters.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/platform/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","roleKey":"string","roles":["string"],"statuses":
["string"],"updatedAtStart":"string"}'
```

application/json

字段	类型	必填	描述
keyword	string	否	Search keyword.
roleKey	string	否	Role key filter.
roles	array<string>	否	Role filters or the returned role list, depending on the model.
statuses	array<string>	否	Member status filters: active or disabled.
updatedAtStart	string	否	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	否	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	否	Page number, starting at 1.
size	integer (int64)	否	Number of members per page.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	PlatformMembersResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

MQTT 身份认证

创建和删除 MQTT 凭据。

POST /mqtt-auth/create

Create an MQTT credential
Creates a normal MQTT credential. The returned password is available only once and must be stored securely immediately.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"name":"string","description":"string","clientIDRandomSuffixEnabled":false}'
```

application/json

字段	类型	必填	描述
name	string	是	MQTT credential name.
description	string	否	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	否	Whether to append a random suffix to the MQTT client ID.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "name": "string",
    "clientID": "string",
    "username": "string",
    "password": "string"
  },
  "msg": "string"
}
```

成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code

字段	类型	必填	描述
data	object	否	MqttAuthCreateResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /mqtt-auth/delete

Delete an MQTT credential

Idempotently deletes a normal MQTT credential. System credentials are not deleted.

请求体

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

字段	类型	必填	描述
id	integer (int64)	是	Resource ID.

响应

200 400 401 403 404 409 413 500

成功响应示例

200 · application/json


```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```


成功响应字段

200 · application/json

字段	类型	必填	描述
code	integer	否	Response code
data	object	否	MqttAuthDeleteResp. 请参阅架构章节中的引用架构。
msg	string	否	Response message

响应状态含义

状态	含义	媒体类型
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

架构

API 字段名、示例及源描述沿用随附的英文 OpenAPI 规范。

AssetFileDeleteReq

字段	类型	必填	描述
`filePath`	`string`	是	Server-issued storage path returned by an upload API.

AssetFileDeleteResp

字段	类型	必填	描述
`deleted`	`boolean`	是	Whether the file was deleted.

AssetFileMultipartAbortReq

字段	类型	必填	描述
`fileKey`	`string`	是	Object-store key used for the multipart upload.
`uploadId`	`string`	是	Object-store multipart upload session ID.

AssetFileMultipartAbortResp

字段	类型	必填	描述
`aborted`	`integer (int32)`	是	Abort result: 1 = aborted (including idempotent replay success), 2 = not aborted.

AssetFileMultipartCompleteReq

字段	类型	必填	描述
`fileKey`	`string`	是	Object-store key used for the multipart upload.
`uploadId`	`string`	是	Object-store multipart upload session ID.
`parts`	`array<object>`	是	Uploaded part numbers and their ETags.

AssetFileMultipartCompleteResp

字段	类型	必填	描述
`filePath`	`string`	是	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	否	File access URL.
`sizeBytes`	`integer (int64)`	是	File size in bytes.
`expiresAt`	`integer (int64)`	否	Expiration time, in Unix milliseconds.

AssetFileMultipartInitReq

字段	类型	必填	描述
fileName	string	是	Original file name.
contentType	string	否	File MIME type, for example application/octet-stream.
size	integer (int64)	是	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	否	Preferred part size in bytes. The server may normalize it.
business	string	否	Business domain or feature associated with the file.
useBy	string	否	Optional usage category: user, workspace, or platform.
visibility	string	否	File visibility: private or public.
appInstanceId	string	否	Associated application instance ID.
sessionId	string	否	Caller session ID.
bizId	string	否	Associated business identifier.

AssetFileMultipartInitResp

字段	类型	必填	描述
fileKey	string	是	Object-store key used for the multipart upload.
filePath	string	是	Server-issued storage path returned by an upload API.
uploadId	string	是	Object-store multipart upload session ID.
partSize	integer (int64)	是	Part size in bytes; the server may normalize it.
partCount	integer (int64)	是	Total number of upload parts.
expiresAt	integer (int64)	是	Expiration time, in Unix milliseconds.

AssetFileMultipartPart

字段	类型	必填	描述
partNumber	integer (int32)	是	Part number, starting at 1.
etag	string	是	ETag returned after uploading this part.

AssetFileMultipartPartUrl

字段	类型	必填	描述
partNumber	integer (int32)	是	Part number, starting at 1.
url	string	是	Presigned PUT URL for this part.
expiresAt	integer (int64)	是	Expiration time, in Unix milliseconds.

AssetFileMultipartPartUrlsReq

字段	类型	必填	描述
fileKey	string	是	Object-store key used for the multipart upload.
uploadId	string	是	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	是	One-based part numbers to sign.

AssetFileMultipartPartUrlsResp

字段	类型	必填	描述
partUrls	array<object>	是	Presigned URLs for the requested parts.

AssetFileUploadReq

字段	类型	必填	描述
fileName	string	是	Original file name.
contentType	string	否	MIME content type.
size	integer (int64)	是	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
business	string	否	Business domain or feature associated with the file.
useBy	string	否	Optional usage category: user, workspace, or platform.
visibility	string	否	File visibility, such as private or public.
appInstanceId	string	否	Associated application instance ID.
sessionId	string	否	Caller session ID.

AssetFileUploadResp

字段	类型	必填	描述
`fileId`	`integer (int64)`	否	File ID.
`filePath`	`string`	是	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	否	File access URL.
`uploadUrl`	`string`	是	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
`expiresAt`	`integer (int64)`	否	Expiration time, in Unix milliseconds.

AssetFileUrlResp

字段	类型	必填	描述
`fileUrl`	`string`	是	File access URL.
`expiresAt`	`integer (int64)`	否	Expiration time, in Unix milliseconds.

AssetUploadReq

AssetUploadReq

字段	类型	必填	描述
`fileName`	`string`	是	Original file name.
`sizeBytes`	`integer (int64)`	是	File size in bytes.
`contentType`	`string`	否	File MIME type, for example application/octet-stream.
`size`	`integer (int64)`	否	File size in bytes.
`sha256`	`string`	否	Optional SHA-256 checksum of the file content.
`storageKey`	`string`	否	Optional object-store key. Omit it to let the server generate one.
`isPublic`	`boolean (boolean)`	否	Whether the file should be accessible through the public download endpoint.
`business`	`string`	否	Business domain or feature that owns the uploaded file.
`useBy`	`string`	否	Usage scenario or owner category for the uploaded file.

AssetUploadResp

AssetUploadResp

字段	类型	必填	描述
`uploadUrl`	`string`	是	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
`filePath`	`string`	是	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	否	File access URL.

AuthWhoamiResp

字段	类型	必填	描述
`userID`	`integer (int64)`	是	Numeric ID of the API key owner.
`userName`	`string`	是	User name.
`email`	`string`	是	Email address.
`workspaceID`	`integer (int64)`	是	Platform workspace ID.
`workspaceName`	`string`	是	Workspace name; may be empty in Enterprise.
`apiKeyName`	`string`	是	Name of the current API key.
`keyPrefix`	`string`	是	API key prefix.
`permissions`	`array<string>`	是	Effective permissions of the current identity.
`roles`	`array<string>`	是	Role filters or the returned role list, depending on the model.
`keyType`	`string`	是	API key ownership type, for example personal.

BrowseReq

BrowseReq

字段	类型	必填	描述
`path`	`string`	否	Root UNS path. Empty means the workspace root.
`max_depth`	`integer (int64)`	否	Maximum tree depth. Omit or use 0 for the server default.
`include_metadata`	`boolean (boolean)`	否	Whether returned nodes should include metadata.
`include_leaf_value`	`boolean (boolean)`	否	Whether returned leaf nodes should include their latest payload.

BrowseResp

BrowseResp

字段	类型	必填	描述
tree	array<object>	是	UNS node tree.

CreateResp

CreateResp

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
results	array<object>	是	Per-item results. Check success and error for each item.

FlowCreateReq

字段	类型	必填	描述
flowName	string	是	Flow name, up to 63 Unicode characters.
flowType	string	是	Flow category: SourceFlow or EventFlow.
description	string	否	Flow description.
template	string	否	Optional flow template JSON.

FlowCreateResp

字段	类型	必填	描述
id	integer (int64)	是	Resource ID.
brokerID	string	是	ID of the MQTT broker configuration node injected when the flow is created.

FlowDataResp

字段	类型	必填	描述
rev	string	是	Node-RED canvas revision; may be an empty string.
flows	array<object>	是	Array of Node-RED canvas node objects.

FlowDeleteReq

字段	类型	必填	描述
ids	array<integer (int64)>	是	Flow IDs to delete.

FlowDeployReq

字段	类型	必填	描述
id	integer (int64)	是	Flow ID to deploy.
flowsJson	string	是	Node-RED canvas JSON to deploy.

FlowDeployResp

字段	类型	必填	描述
flowId	string	是	Flow identifier.

FlowEmptyResp

字段	类型	必填	描述
success	boolean	是	Whether the operation succeeded.

FlowGetReq

字段	类型	必填	描述
id	integer (int64)	是	Flow ID.

FlowInfo

字段	类型	必填	描述
id	integer (int64)	是	Resource ID.
flowId	string	是	Flow identifier.
flowName	string	是	Flow name.
flowType	string	是	Response flow type: source or event.
flowStatus	string	是	Current flow status, for example draft.
template	string	是	Flow template JSON, encoded as a string.
description	string	是	Description.

字段	类型	必填	描述
isFavorite	integer (int64)	是	Favorite state: 1 for favorite, 2 for not favorite.
currentVersionId	integer (int64)	是	Current version ID; 0 if unset.
currentVersionName	string	是	Current version name; empty if unset.
currentVersionType	string	是	Current version type; empty if unset.
createdTime	integer (int64)	是	Creation time, in Unix milliseconds.
updatedTime	integer (int64)	是	Last update time, in Unix milliseconds.

FlowListReq

字段	类型	必填	描述
keyword	string	否	Keyword matched against flow names and descriptions.
flowType	string	否	Flow category filter: SourceFlow or EventFlow.

FlowListResp

字段	类型	必填	描述
list	array<object>	是	Result list.

FlowNodesReq

字段	类型	必填	描述
flowType	string	是	Flow category whose Node-RED node catalog should be returned.

FlowNodesResp

字段	类型	必填	描述
nodes	array<object>	是	Available Node-RED node sets.

FlowUpdateReq

字段	类型	必填	描述
id	integer (int64)	是	Flow ID to update.
flowName	string	否	Updated flow name.
description	string	否	Updated flow description.

字段	类型	必填	描述
`template`	`string`	否	Updated flow template JSON.
`isFavorite`	`integer (int64)`	否	Favorite state: 1 for favorite, 2 for not favorite.

HistoryAggregation

HistoryAggregation

字段	类型	必填	描述
`interval`	`string`	是	Positive aggregation window, such as 1m, 5m, 1h, or 1d.
`function`	`string`	否	Aggregation function: avg, min, max, sum, count, first, or last.
`field`	`string`	否	First-level payload field to aggregate.
`fields`	`array<object>`	否	Per-field aggregation settings.

HistoryAggregationField

HistoryAggregationField

字段	类型	必填	描述
`name`	`string`	是	Payload field to aggregate.
`function`	`string`	否	Function for this field; defaults to the outer aggregation function.

HistoryQueryMeta

HistoryQueryMeta

字段	类型	必填	描述
`aggregated`	`boolean (boolean)`	是	Whether the result was aggregated.
`sampled`	`boolean (boolean)`	是	Whether the result was sampled.
`rawTotal`	`integer (int64)`	是	Total number of raw records.
`returnedTotal`	`integer (int64)`	是	Number of records returned in this response.
`requestedIntervalMs`	`integer (int64)`	否	Requested aggregation interval in milliseconds.
`actualIntervalMs`	`integer (int64)`	否	Actual aggregation interval in milliseconds.
`strategy`	`string`	否	Processing strategy used for the query.

HistoryReq

HistoryReq

字段	类型	必填	描述
topics	array<string>	是	UNS topic paths whose history should be queried.
start_time	string	是	Inclusive query start in RFC 3339 format.
end_time	string	是	Inclusive query end in RFC 3339 format.
aggregation	object	否	HistoryAggregation. 请参阅架构章节中的引用架构。
page	integer (int64)	否	Page number, starting from 1.
size	integer (int64)	否	Page size; values above the server maximum are capped.

HistoryResp

HistoryResp

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
results	array<object>	是	Per-item results. Check success and error for each item.
page	integer (int64)	是	Page number, starting at 1.
size	integer (int64)	是	File size in bytes.
total	integer (int64)	是	Total number of matching records.

InfoReq

```
{
  "type": "object",
  "properties": {}
}
```

LaunchpadGetMembersReq

字段	类型	必填	描述
roleKey	string	否	Role key filter.
roles	array<string>	否	Role filters or the returned role list, depending on the model.
updatedAtStart	string	否	Inclusive update-time lower bound in RFC 3339 format.

字段	类型	必填	描述
updatedAtEnd	string	否	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	否	Page number, starting at 1.
size	integer (int64)	否	Number of members per page.

LaunchpadMemberResp

字段	类型	必填	描述
memberId	string	是	Member identifier.
userId	string	是	User ID.
userName	string	否	User name.
email	string	否	Email address.
roles	array<object>	是	Role filters or the returned role list, depending on the model.
updatedAt	string	否	Last update time.

LaunchpadMemberRoleAppResp

字段	类型	必填	描述
appId	string	是	Application ID, represented as a string.
appName	string	是	Application name.

LaunchpadMemberRoleResp

字段	类型	必填	描述
roleId	string	是	Role ID, represented as a string.
roleKey	string	是	Role key filter.
roleName	string	否	Role display name.
description	string	否	Description.
memberCount	integer (int64)	是	Number of members assigned to the role.
apps	array<object>	否	Applications associated with the role.

LaunchpadMembersResp

字段	类型	必填	描述
list	array<object>	是	Result list.
total	integer (int64)	是	Total number of matching records.
page	integer (int64)	是	Page number, starting at 1.
size	integer (int64)	是	File size in bytes.

MqttAuthCreateReq

字段	类型	必填	描述
name	string	是	MQTT credential name.
description	string	否	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	否	Whether to append a random suffix to the MQTT client ID.

MqttAuthCreateResp

字段	类型	必填	描述
id	integer (int64)	是	Resource ID.
name	string	是	Name.
clientID	string	是	MQTT client ID.
username	string	是	MQTT user name.
password	string	是	Plaintext password. Save it now; it cannot be retrieved again.
clientIDRandomSuffixEnabled	boolean	是	Whether to append a random suffix to the MQTT client ID.

MqttAuthDeleteReq

字段	类型	必填	描述
id	integer (int64)	是	Resource ID.

MqttAuthDeleteResp

字段	类型	必填	描述
success	boolean	是	Whether the operation succeeded.

NamespaceNode

NamespaceNode

字段	类型	必填	描述
name	string	是	Node name, up to 63 Unicode characters. In a nested request, this is the name under its parent.
type	string	否	Node type: PATH or TOPIC.
children	array<object>	否	Child nodes to create under this node.
topicType	string	否	Topic data category: STATE, ACTION, or METRIC.
fields	array<object>	否	Payload schema fields for TOPIC nodes.
description	string	否	Node description, up to 512 Unicode characters.
displayName	string	否	Display name shown in clients.
extendProperties	object	否	Custom metadata key-value pairs.
alias	string	否	Optional topic alias.
enableHistory	boolean (boolean)	否	Whether history storage is enabled for this topic.

NodeCreateReq

NodeCreateReq

字段	类型	必填	描述
namespace	array<object>	是	UNS nodes to create. Nest children to create a subtree.

NodeDeleteReq

NodeDeleteReq

字段	类型	必填	描述
topics	array<string>	是	UNS topic or path node paths to delete.

字段	类型	必填	描述
hard_delete	boolean (boolean)	否	Permanently delete instead of moving nodes to the recycle state.

NodeRedNodeSet

字段	类型	必填	描述
id	string	是	Resource ID.
name	string	是	Name.
types	array<string>	是	Node-RED node types provided by the node set.
enabled	boolean	是	Whether the node set is enabled.
module	string	是	Node-RED module name.
version	string	是	Service or node module version.

NodeRestoreReq

NodeRestoreReq

字段	类型	必填	描述
path	string	是	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

NodeUpdateReq

NodeUpdateReq

字段	类型	必填	描述
path	string	是	Existing UNS topic or path node to update.
name	string	否	New node name.
description	string	否	Updated node description.
displayName	string	否	Updated display name.
extendProperties	object	否	Updated custom metadata key-value pairs.
fields	array<object>	否	Updated payload schema fields.
alias	string	否	Updated topic alias.

字段	类型	必填	描述
enableHistory	boolean (boolean)	否	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	否	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

OpenapiCreateResult

OpenapiCreateResult

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
topic	string	是	Full UNS topic path.
error	object	否	OpenapiItemError. 请参阅架构章节中的引用架构。

OpenapiEmptyResp

OpenapiEmptyResp

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.

OpenapiHistoryItem

OpenapiHistoryItem

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
topic	string	是	Full UNS topic path.
result	object	否	OpenapiHistoryResult. 请参阅架构章节中的引用架构。
error	object	否	OpenapiItemError. 请参阅架构章节中的引用架构。

OpenapiHistoryResult

OpenapiHistoryResult

字段	类型	必填	描述
values	array<object>	是	Historical VQT records.

字段	类型	必填	描述
meta	object	否	HistoryQueryMeta. 请参阅架构章节中的引用架构。

OpenapiHistoryValue

OpenapiHistoryValue

字段	类型	必填	描述
value	object	否	Payload value.
quality	string	是	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	是	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiItemError

OpenapiItemError

字段	类型	必填	描述
code	integer (int64)	是	Status code; in an item error, identifies the failure.
message	string	是	Item-level error message.

OpenapiNodeInfo

OpenapiNodeInfo

字段	类型	必填	描述
id	integer (int64)	是	Resource ID.
path	string	是	Full UNS node path.
name	string	是	Name.
type	string	是	Node type: PATH or TOPIC.
topicType	string	是	Topic category: STATE, ACTION, or METRIC.
description	string	否	Description.
displayName	string	否	Display name.
extendProperties	object	否	Custom node metadata as key-value pairs.
fields	array<object>	否	Payload field definitions.
children	array<object>	否	Child nodes under this node.

字段	类型	必填	描述
alias	string	否	Node alias.
enableHistory	boolean (boolean)	否	Whether history storage is enabled.
bindFlowID	integer (int64)	否	Bound Source Flow ID. 0 means no Source Flow is bound.
bindEventFlowID	integer (int64)	否	Bound Event Flow ID. 0 means no Event Flow is bound.
payload	object	否	OpenapiVQT. 请参阅架构章节中的引用架构。

OpenapiReadItem

OpenapiReadItem

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
topic	string	是	Full UNS topic path.
result	object	否	OpenapiVQT. 请参阅架构章节中的引用架构。
error	object	否	OpenapiItemError. 请参阅架构章节中的引用架构。
metadata	object	否	OpenapiNodeInfo. 请参阅架构章节中的引用架构。

OpenapiVQT

OpenapiVQT

字段	类型	必填	描述
value	object	否	Payload value.
quality	string	是	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	是	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiWriteResult

OpenapiWriteResult

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
topic	string	是	Full UNS topic path.
error	object	否	OpenapiItemError. 请参阅架构章节中的引用架构。

PlatformGetMembersReq

字段	类型	必填	描述
keyword	string	否	Search keyword.
roleKey	string	否	Role key filter.
roles	array<string>	否	Role filters or the returned role list, depending on the model.
statuses	array<string>	否	Member status filters: active or disabled.
updatedAtStart	string	否	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	否	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	否	Page number, starting at 1.
size	integer (int64)	否	Number of members per page.

PlatformMemberResp

字段	类型	必填	描述
userId	string	是	User ID.
userName	string	否	User name.
nickName	string	否	User nickname.
email	string	否	Email address.
status	string	是	Platform member status: active or disabled.
roles	array<object>	是	Role filters or the returned role list, depending on the model.
updatedAt	string	是	Last update time.

PlatformMemberRoleResp

字段	类型	必填	描述
roleId	string	是	Role ID, represented as a string.
roleKey	string	是	Role key filter.
roleName	string	否	Role display name.
description	string	否	Description.

PlatformMembersResp

字段	类型	必填	描述
list	array<object>	是	Result list.
total	integer (int64)	是	Total number of matching records.
page	integer (int64)	是	Page number, starting at 1.
size	integer (int64)	是	File size in bytes.

ReadReq

ReadReq

字段	类型	必填	描述
topics	array<string>	是	UNS topic paths to read.
include_metadata	boolean (boolean)	否	Whether each result should include UNS node metadata.
include_leaf_value	boolean (boolean)	否	Whether leaf nodes should include their latest payload.

ReadResp

ReadResp

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
results	array<object>	是	Per-item results. Check success and error for each item.

SchemaField

SchemaField

字段	类型	必填	描述
name	string	是	Payload field name.
type	string	是	Payload field type: INTEGER, LONG, DOUBLE, FLOAT, STRING, BOOLEAN, or DATETIME.
unit	string	否	Optional engineering unit.

SearchReq

SearchReq

字段	类型	必填	描述
keyword	string	否	Keyword matched against UNS node names, aliases, and paths.
topicType	string	否	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	否	Only return nodes under this UNS path prefix.
include_metadata	boolean (boolean)	否	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	否	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	否	Page number, starting from 1.
size	integer (int64)	否	Page size.

SearchResp

SearchResp

字段	类型	必填	描述
objects	array<object>	是	Matching UNS nodes.
page	integer (int64)	是	Page number, starting at 1.
size	integer (int64)	是	File size in bytes.
total	integer (int64)	是	Total number of matching records.

UnsAttachmentInfo

UnsAttachmentInfo

字段	类型	必填	描述
unsId	integer (int64)	是	Numeric UNS node ID.
fileName	string	否	Original file name.
filePath	string	是	Server-issued storage path returned by an upload API.
fileUrl	string	否	File access URL.
contentType	string	否	File MIME type, for example application/octet-stream.
size	integer (int64)	否	File size in bytes.
sha256	string	否	SHA-256 checksum of the file contents.

字段	类型	必填	描述
createdAt	integer (int64)	否	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	否	File URL expiration time, in Unix milliseconds.

UnsAttachmentListReq

UnsAttachmentListReq

字段	类型	必填	描述
unsId	integer (int64)	否	Numeric UNS node ID.
topic	string	否	Full UNS topic path.
page	integer (int64)	否	Page number, starting at 1.
size	integer (int64)	否	Number of attachments per page.
includeFileUrl	boolean (boolean)	否	Whether to return temporary file URLs. Defaults to true.

UnsAttachmentListResp

UnsAttachmentListResp

字段	类型	必填	描述
unsId	integer (int64)	是	Numeric UNS node ID.
list	array<object>	是	Result list.
total	integer (int64)	是	Total number of matching records.
page	integer (int64)	是	Page number, starting at 1.
size	integer (int64)	是	File size in bytes.

UnsAttachmentUploadReq

UnsAttachmentUploadReq

字段	类型	必填	描述
unsId	integer (int64)	否	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
topic	string	否	Full owner node path, used when a valid unsId is not supplied.
fileName	string	否	Original file name.

字段	类型	必填	描述
sha256	string	否	SHA-256 checksum of the file contents.
file	string (binary)	是	Binary file contents. The multipart field name must be file.

UnsAttachmentUploadResp

UnsAttachmentUploadResp

字段	类型	必填	描述
unsId	integer (int64)	是	Numeric UNS node ID.
fileName	string	否	Original file name.
filePath	string	是	Server-issued storage path returned by an upload API.
fileUrl	string	否	File access URL.
contentType	string	否	File MIME type, for example application/octet-stream.
size	integer (int64)	否	File size in bytes.
sha256	string	否	SHA-256 checksum of the file contents.
createdAt	integer (int64)	否	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	否	File URL expiration time, in Unix milliseconds.

UnsBindFlowReq

UnsBindFlowReq

字段	类型	必填	描述
unsId	integer (int64)	是	UNS node ID to bind.
flowId	integer (int64)	是	Flow ID to bind.

UnsBindFlowResp

UnsBindFlowResp

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
unsId	integer (int64)	是	Numeric UNS node ID.

字段	类型	必填	描述
flowId	integer (int64)	是	Flow identifier.

WriteItem

WriteItem

字段	类型	必填	描述
topic	string	是	UNS topic path to write.
value	object	否	Payload value to write.
timeStamp	integer (int64)	否	Optional Unix timestamp in milliseconds. Omit it to use server time.

WriteReq

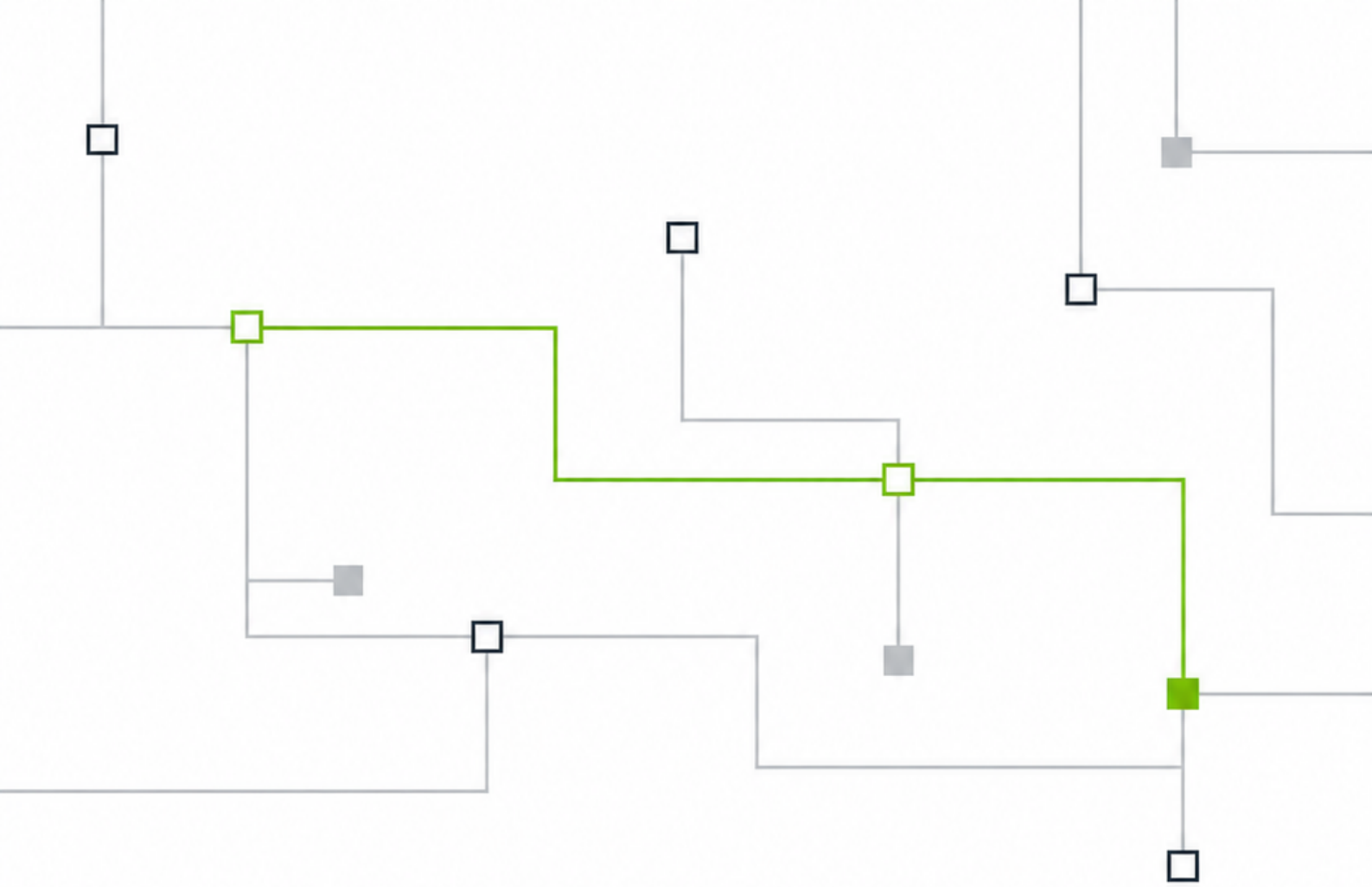
WriteReq

字段	类型	必填	描述
writes	array<object>	是	Write operations to apply.
qos	integer (int64)	否	MQTT QoS: 0, 1, or 2.
retain	boolean (boolean)	否	Whether the MQTT message should be retained.

WriteResp

WriteResp

字段	类型	必填	描述
success	boolean (boolean)	是	Whether the operation succeeded.
results	array<object>	是	Per-item results. Check success and error for each item.



TIER 0

tier0.app